

烟幕干扰弹的投放策略

摘 要

针对无人机投放烟幕干扰弹对来袭导弹实施光学遮蔽的最优策略问题，给出运动学建模、视锥几何判定、以及粒子群联合 Powell 精修的分层优化方法。

建立导弹、无人机、烟幕弹及爆炸后云团的运动学方程，以视锥线判据刻画“全部关键点是否同时被云团遮挡”的几何条件，并提出多弹协同（互补）遮蔽判定；以有效遮蔽时长的时间积分为统一目标函数。

问题一在给定固定参数下求得 FY1 对 M1 的有效遮蔽时长约为 0 s，用于校验模型与程序。问题二以航向角、速度、投放时刻、起爆延时为决策变量，构造 4 维连续优化问题，得到 FY1 单机单弹的最优有效遮蔽时长为 4.895 s。问题三扩展为 8 维（三弹投放时序与起爆延时），得到三弹协同对 M1 的总有效遮蔽时长为 5.555 s。问题四在 12 维空间上做 FY1、FY2、FY3 的协同优化，得到三机协同对 M1 的总有效遮蔽时长为 8.770 s。问题五将协同拦截推广至五机 15 弹对三枚导弹的大规模场景，得到五机对三枚导弹的总有效遮蔽时长为 18.400 s。

敏感性分析表明，关键点采样数 n_c 超过约 100 后有效遮蔽时长趋于收敛；云团下沉速度与无人机速度上限是影响遮蔽时长的主要参数。

关键字： 烟幕干扰弹 粒子群算法 视锥判定 多机协同 运动学建模

一、 问题重述

现代精确制导武器对地面关键目标威胁日增，烟幕干扰等被动遮蔽手段因此得到广泛运用。本文研究的具体场景为：设假目标位于坐标原点，真目标为一竖直圆柱体；来袭导弹以恒定速率直线飞向假目标；我方多架无人机可在不同位置、以不同航向和速度巡飞并投放干扰弹。干扰弹起爆后形成球状烟幕云团，在云团遮挡住导弹观察真目标视线时即形成有效遮蔽。需要在满足物理与战术约束的前提下规划无人机的飞行与投放策略，使对导弹的有效遮蔽时间尽可能长。具体问题如下：

1. 问题一：无人机 FY1 以给定航向、速度和投放/起爆参数投放 1 枚干扰弹，计算其对导弹 M1 的有效遮蔽时长。
2. 问题二：FY1 投放 1 枚干扰弹干扰 M1，优化其飞行方向、速度、投放时刻与起爆延时，使有效遮蔽时长最长。
3. 问题三：FY1 投放 3 枚干扰弹干扰 M1（相邻投放间隔不小于 1 s），优化航向、速度与三弹投放/起爆时序，使总有效遮蔽时长最长，并输出投放方案。
4. 问题四：FY1、FY2、FY3 各投放 1 枚干扰弹协同干扰 M1，优化三机飞行与投放参数，使协同遮蔽时长最长，并输出方案。
5. 问题五：FY1~FY5 五架无人机、每架至多投放 3 枚干扰弹，协同干扰 M1、M2、M3 三枚来袭导弹，优化整体投放策略使总有效遮蔽时长最长，并输出方案。其中总有效遮蔽时长定义为：对每枚来袭导弹，按视锥几何条件（以导弹为顶点、云团为张角）判定其在每一时刻是否被一枚或多枚存活云团完全遮挡所有关键点；将各时刻的判定结果在时间上积分得到该导弹的有效遮蔽时长；最后将三枚导弹各自的有效遮蔽时长求和作为优化目标。

二、 问题分析

五个问题物理机理同源——均以“几何光学视线”判定遮蔽，差别在于决策规模与协同程度。问题一为验证性算例，问题二至五为依次递进的参数优化与协同寻优问题。

问题一参数全部给定。直接将运动学与判定条件代入即可求得遮蔽时长，用于校核模型与程序的正确性。

问题二决策变量为航向角、飞行速度、投放时刻、起爆延时共 4 个连续量。目标函数关于这些变量高度非线性、且存在大片的零遮蔽平台，梯度信息不可靠。需要采用对初值不敏感的群体智能类全局优化方法。

问题三扩展为 3 枚弹同时干扰 M1。三弹的投放时刻与起爆延时形成 8 维决策空间，且需协调三弹的时序以延长总遮蔽窗口。直接从纯随机初始化在 8 维空间搜索效率低，宜先用较易解的问题二得到较好的热启动种子。

问题四引入 FY1、FY2、FY3 三机协同干扰同一导弹，决策维数升至 12。一次性对 12 维做冷启动全局搜索代价过高且易陷入局部最优。可先分解为三个独立的单机子问题获得基线解，再在其邻域作联合精修——该分解使得协同增益（多机云团在时空上互补）能被充分利用。

问题五为五机、每机至多三弹、对三枚导弹的大规模协同拦截，决策维数达 40，且需为每枚弹指派其拦截的目标导弹。沿用问题四的两阶段策略，结合各机对三枚导弹的空间可达性预设拦截分工。

三、 模型假设与符号说明

3.1 模型假设

为简化问题，作出以下假设：

1. 导弹在整个交战过程中以恒定速率 $v_M = 300 \text{ m/s}$ 沿指向假目标（原点）的直线飞行，方向保持不变。
2. 无人机受领任务后作等高度、匀速、直线飞行，其航向角与飞行速度一经确定即不再改变，飞行速度限制在 $[80, 120] \text{ m/s}$ 之间。
3. 干扰弹脱离无人机后、起爆之前作平抛运动：水平方向保持投放瞬间无人机的速度矢量，竖直方向仅受重力作用（ $g = 9.8 \text{ m/s}^2$ ），忽略空气阻力。
4. 干扰弹起爆瞬间在起爆点形成半径 $R = 10 \text{ m}$ 的球形云团；此后云团以 $v_s = 2.5 \text{ m/s}$ 匀速竖直下沉，水平位置不变，自起爆时刻起持续有效 20 s。
5. 忽略云团的扩散、形变与风的影响，在有效期内视为半径 10 m 的均匀不透明球体。
6. 遮蔽判定采用几何光学视线模型：当导弹指向真目标的所有视线均被云团阻断时，认为真目标被有效遮蔽；真目标以其圆柱表面离散关键点的集合近似。

7. 同一时刻若有多枚云团同时存活, 允许其协同遮蔽同一目标, 即不同云团可分别遮挡目标的不同部分。
8. 真目标为规则圆柱, 其尺寸与位置在交战过程中固定不变。

3.2 符号说明

符号	含义
$\mathbf{M}_k(t)$	第 k 枚导弹在时刻 t 的位置 ($k = 1, 2, 3$)
$\mathbf{F}_j(t)$	第 j 架无人机在时刻 t 的位置 ($j = 1, \dots, 5$)
$\mathbf{S}(t)$	烟幕云团球心在时刻 t 的位置
θ_j, v_j	第 j 架无人机的航向角与飞行速度
t_r, τ	干扰弹的投放时刻与起爆延时 (起爆时刻 $t_e = t_r + \tau$)
$v_M = 300$	导弹飞行速率 (m/s)
$v_s = 2.5$	云团下沉速率 (m/s)
$R = 10$	云团有效遮蔽半径 (m)
$P_T = (0, 200, 0)$	真目标圆柱下底面圆心
$r = 7, h = 10$	真目标圆柱的半径与高 (m)
α	视锥半顶角, $\sin \alpha = R / \ \mathbf{M} - \mathbf{S}\ $
γ_i	关键点 $\mathbf{K}_i$ 相对视锥轴的偏离角
T_{eff}	有效遮蔽时长 (s)

四、模型的建立与求解

本节先建立贯穿全部五个问题的公共模型（5.1），再依次求解各问（5.2~5.6）。

4.1 烟幕遮蔽效果的基础模型

4.1.1 坐标系与初始布局

以假目标为坐标原点建立三维直角坐标系 $O-xyz$ ， z 轴竖直向上。真目标为一竖直圆柱，下底面圆心 $P_T = (0, 200, 0)$ ，半径 $r = 7$ m、高 $h = 10$ m。三枚来袭导弹 M1~M3 与五架无人机 FY1~FY5 的初始位置如下表，整体空间布局见图。

导弹	初始坐标 (m)	无人机	初始坐标 (m)	
M1	(20000, 0, 2000)	FY1	(17800, 0, 1800)	
M2	(19000, 600, 2100)	FY2	(12000, 1400, 1400)	
M3	(18000, -600, 1900)	FY3	(6000, -3000, 700)	
		FY4	(11000, 2000, 1800)	
		FY5	(13000, -2000, 1300)	

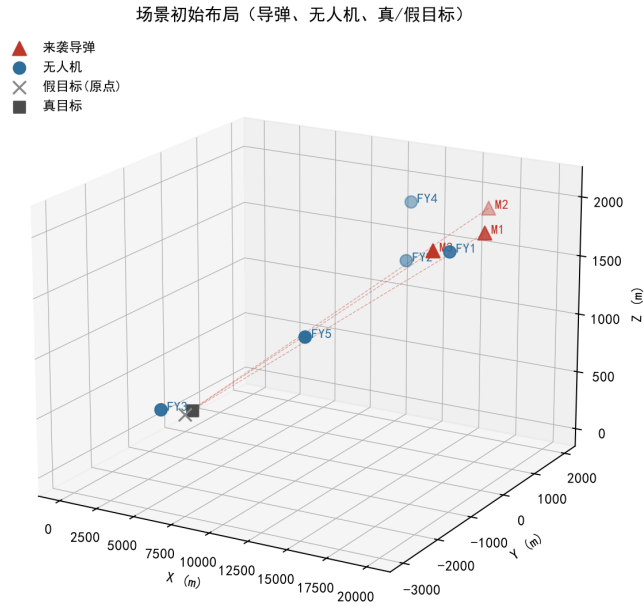


图 1 场景初始布局：3 枚来袭导弹、5 架无人机与真/假目标的三维位置

4.1.2 运动学模型

设第 k 枚导弹以恒定速率 v_M 沿指向原点的方向直线飞行。记初始位置为 M_k^0 ，单位方向矢量

$$e_k = -M_k^0 / \|M_k^0\|, \quad (1)$$

则

$$M_k(t) = M_k^0 + v_M e_k t. \quad (2)$$

第 j 架无人机在水平面内以航向角 θ_j 、速率 v_j 等高匀速直线飞行：

$$\mathbf{F}_j(t) = \mathbf{F}_j^0 + v_j(\cos \theta_j, \sin \theta_j, 0) t. \quad (3)$$

干扰弹的整体运动分两段：自无人机投放至起爆前为平抛运动；起爆后至云团失效，视为匀速下沉。

无人机在时刻 t_r 于位置 $\mathbf{F}_j(t_r)$ 投放干扰弹。脱离载机后，干扰弹在水平方向保持投放瞬间无人机的速度矢量，竖直方向仅受重力作用，经延时 τ 于起爆时刻 $t_e = t_r + \tau$ 起爆。设投放点水平坐标 (x_r, y_r) 、高度 $z_r = F_{j,z}^0$ ，起爆点为

$$\mathbf{S}(t_e) = \begin{pmatrix} x_r + v_j \cos \theta_j \cdot \tau \\ y_r + v_j \sin \theta_j \cdot \tau \\ z_r - \frac{1}{2} g \tau^2 \end{pmatrix}. \quad (4)$$

起爆后云团水平位置固定，球心以 v_s 匀速竖直下沉，在有效期 $t \in [t_e, t_e + 20]$ 内

$$\mathbf{S}(t) = \begin{pmatrix} x_r + v_j \cos \theta_j \cdot \tau \\ y_r + v_j \sin \theta_j \cdot \tau \\ z_r - \frac{1}{2} g \tau^2 - v_s(t - t_e) \end{pmatrix}. \quad (5)$$

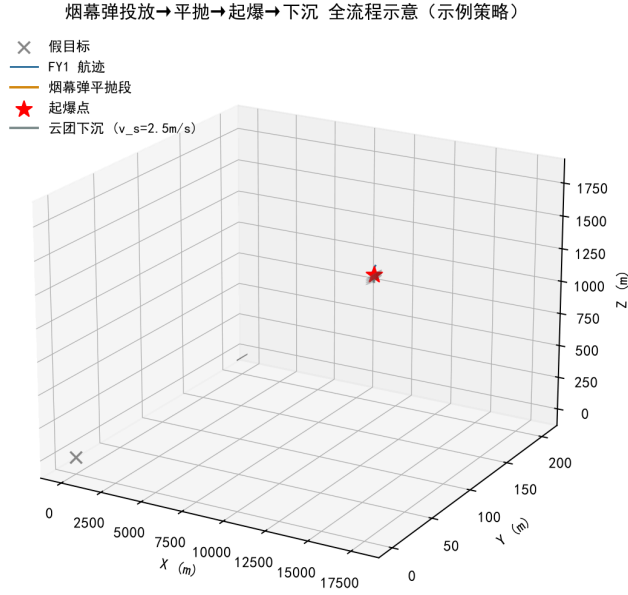


图 2 烟幕弹投放→平抛→起爆→下沉 全流程示意（示例策略）

4.1.3 真目标的离散化

真目标为实心圆柱，是否“被完全遮蔽”取决于其整个可见轮廓是否都被云团挡住。为便于逐时刻判定，将圆柱表面离散为一组关键点 $\{\mathbf{K}_i\}_{i=1}^N$ ：取上、下底面圆周上的均匀采样点，以及侧面若干等高层圆周上的采样点。每个圆周取 n_c 个采样点、侧面取 n_l 层时，关键点总数 $N = 2n_c + n_l \cdot n_c$ （主程序精度下 $n_c = 720$ 、 $n_l = 10$ ， $N = 8640$ ）。关键点越密判据越接近连续目标；敏感性分析表明每圆周采样点数 n_c 达到约 100 以上后

有效遮蔽时长的计算值即趋于稳定。搜索阶段 $n_c = 180$ 、 $n_l = 5$ ，定稿阶段 $n_c = 360$ 、 $n_l = 10$ 。

4.1.4 有效遮蔽的判定条件

考察某时刻导弹 M 、云心 S 与目标关键点集 $\{K_i\}$ 。以假目标为原点建立三维坐标轴，以 z - O - y 平面为视角可构建出遮蔽有效的平面几何图（见图）。当真目标在导弹的视线与云团球相切的切线所围成的视锥内时，判定该时刻为有效遮蔽。

为将这一几何条件形式化，给出如下三条判定：

(1) 导弹进入云团. 若 $\|M - S\| < R$ ，导弹已被云团包裹，视线被阻断，判定为遮蔽。

(2) 云团位于导弹与目标之间. 遮蔽的必要条件是云团比导弹更靠近目标：

$$\|M - P_T\| > \|S - P_T\|. \quad (6)$$

若不满足则该云团无法遮挡。

(3) 视锥判定. 以导弹 M 为顶点、导弹—云心连线 $S - M$ 为轴，云团球对导弹张成半顶角为 α 的视锥：

$$\sin \alpha = \frac{R}{\|M - S\|}. \quad (7)$$

当 $R \geq \|M - S\|$ 时视锥退化为全空间。对任一关键点 K_i ，其相对锥轴的偏离角 γ_i 满足

$$\cos \gamma_i = \frac{(S - M) \cdot (K_i - M)}{\|S - M\| \cdot \|K_i - M\|}. \quad (8)$$

关键点 K_i 的视线被遮挡，当且仅当它落在视锥内，即 $\gamma_i \leq \alpha$ ，等价于 $\cos \gamma_i \geq \cos \alpha$ 。仅当全部关键点都被遮挡时，单朵云团才实现对真目标的完全遮蔽：

$$\text{遮蔽}(t) = 1 \iff \forall i : \cos \gamma_i(t) \geq \cos \alpha(t). \quad (9)$$

视锥判定的几何关系如图所示。

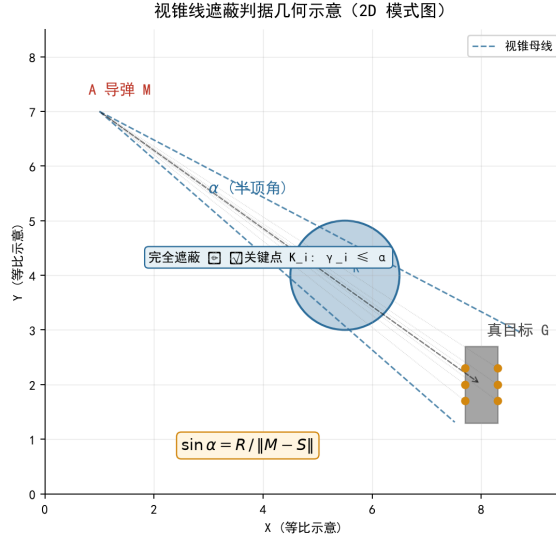


图 3 视锥线遮蔽判定几何示意 (2D 模式图)

4.1.5 多弹协同 (互补) 遮蔽判定

当多枚云团同时存活时, 若仍要求“某一朵云单独挡住全部关键点”会低估协同效果。实际上, 只要目标的每一个关键点在该时刻都被至少一朵存活云团挡住, 导弹指向真目标的所有视线便都被阻断。据此提出协同 (互补) 遮蔽判定: 记第 c 朵存活云团在时刻 t 对关键点 K_i 的遮挡指示为 $b_{c,i}(t) \in \{0, 1\}$ (由上节判定 (2)(3) 对该云团单独计算), 则该时刻真目标被有效遮蔽当且仅当

$$(\exists c : \|M - S_c\| < R) \quad \vee \quad (\forall i : \exists c : b_{c,i}(t) = 1). \quad (10)$$

该判定允许“甲云挡上半、乙云挡下半”这类错位互补的协同方式, 是问题三~五中多弹、多机协同增益的建模基础。协同遮蔽机理示意图。

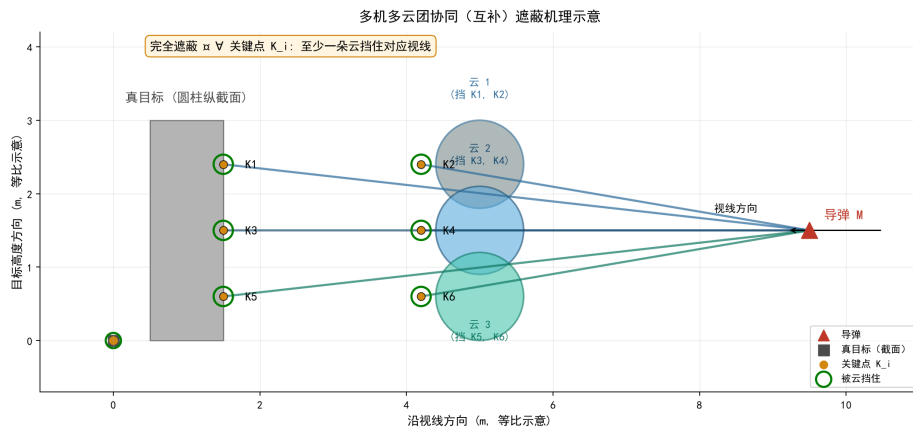


图 4 多机多云团协同 (互补) 遮蔽机理示意

4.1.6 有效遮蔽时长

记 $O(t) \in \{0, 1\}$ 为上述判定给出的时刻 t 遮蔽指示, 则针对单枚导弹的有效遮蔽时长为遮蔽区间总测度

$$T_{\text{eff}} = \int_0^{T_{\text{max}}} O(t) \, dt \approx \Delta t \sum_n O(t_n), \quad (11)$$

数值上以步长 Δt 在时间网格上离散求和近似。对多枚导弹场景（问题五），总有效遮蔽时长取各导弹遮蔽时长之和。全部优化问题的目标即最大化 T_{eff} 。

4.1.7 优化算法框架

问题二~五均为以 T_{eff} 为目标的连续参数优化。由于目标函数强非线性、含大片零遮蔽平台，采用粒子群算法（**PSO**）为统一的全局搜索器：以惯性权重 $w = 0.7$ 、个体与社会学习因子 $c_1 = c_2 = 1.5$ 更新粒子速度与位置，对越界粒子作边界截断。

选用 **PSO** 的依据如下：本题决策变量均为连续量（航向、速度、投放时序、起爆延时），目标函数光滑性较差但仍有局部梯度信息可用，要求算法既能大范围搜索也能较快收敛到局部最优。三种主流元启发式方法对本题的契合度对比如下：

算法	编码方式	关键超参数	与本题契合度	简要评注
PSO	连续变量直接寻址	w, c_1, c_2 共 3 个；另需设种群规模与迭代次数	高	粒子速度—位置更新天然适配连续优化；种群信息共享利于跳出零平台
GA (实数)	需 SBX/多项式等连续算子	种群规模、交叉率、变异率、交叉分布指数 η_c 、变异分布指数 η_m 、选择算子等 ≥ 5 个	中	算子设计影响显著；调参成本高
SA	单点串行搜索	初温 T_0 、终温 T_{end} 、降温系数 α 、邻域步长共 4 个	中低	降温 schedule 与邻域步长敏感；在非凸地形上易陷入平台

从契合度看，**PSO** 在“决策变量连续 + 目标函数非凸 + 含大片零遮蔽平台 + 单次评估代价约 0.01 s 可承受”这一组合上较 **GA**、**SA** 更契合——它既不需要为连续变量设计编码/交叉/变异算子，又不像 **SA** 那样严重依赖降温 **schedule**；基于种群的信息共享机制

也比单点串行搜索更易在零平台地形上跳出去（Kennedy & Eberhart, 1995; Poli 等, 2007 的综述对此有标准论述）。

值得说明的是：在连续、非凸、含零平台的中小规模（ ≤ 12 维）优化问题上，PSO、GA、SA 三类方法在最终收敛值上的差距往往并不显著——决定工程取舍的更多是超参数数量、调参成本与实现复杂度。本文为佐证这一点，在问题 2 上以同一评估预算（6000 次目标函数评估）实测了 PSO、GA、SA 三种算法的最终收敛值与收敛曲线。实测结果是 PSO 收敛到 4.89 s，GA（实数 SBX 交叉 + 多项式变异 + 锦标赛选择）收敛到 5.00 s，SA（指数降温 + 高斯扰动）收敛到 4.91 s——三者最终值相差在 0.11 s 以内，且 GA 略高于 PSO。这表明在本问题这一规模下，三种算法在解的质量上互有胜负，差距小于零平台边界判定引入的离散噪声，数值上无显著差异；PSO 的主要优势在于其 3 个超参数（ w, c_1, c_2 ）易于设置与解释，且基于种群的并行评估天然适配多核 CPU——这才是工程上选择 PSO 的真正理由，而非“它数值上一定更好”。具体数据与曲线见 §5.4 “PSO / GA / SA 选型实测对比”。

PSO 收敛后再以 Powell 无梯度法在最优解的 $\pm 5\%$ 邻域内局部精修，弥补群体算法末段收敛慢、易受仿真离散噪声干扰的不足。由于 Powell 不保证单调下降，每次精修后取 $\max(T_{\text{eff}}^{\text{PSO}}, T_{\text{eff}}^{\text{Powell}})$ 作为该问最终搜索档结果，再以定稿档复算上报。

为兼顾速度与精度，采用双精度机制：搜索档用较稀关键点、较大步长，定稿档用较密关键点、较密步长。对高维问题（问题三~五）进一步引入热启动种子与两阶段分解策略，详见各问。问题 2 的 PSO 收敛曲线如图所示。

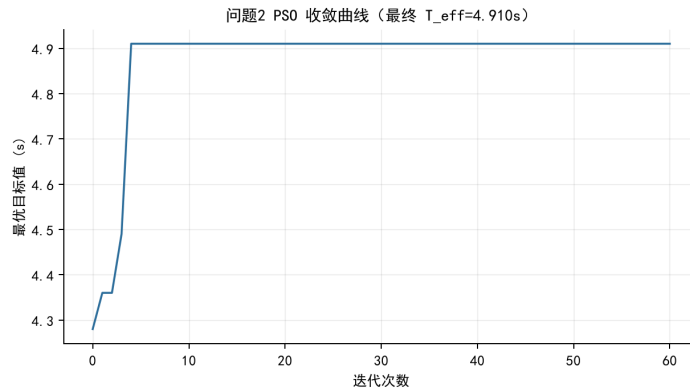


图 5 问题 2 PSO 收敛曲线（最终 $T_{\text{eff}} = 4.890$ s）

4.2 问题一：固定参数下的遮蔽时长

问题一给定确定参数以验证模型。FY1 自 (17800, 0, 1800) 出发，以 $v = 120$ m/s 沿指向假目标方向（航向角 $\theta = \pi$ ，即 $-x$ 方向）等高匀速飞行；受领任务 1.2 s 后投放，投放 3.2 s 后起爆，目标为 M1。代入运动学方程得起爆时刻 $t_e = 1.2 + 3.2 = 4.4$ s，云团在 $t \in [4.4, 24.4]$ s 内有效。在该区间以精细步长 $\Delta t = 10^{-4}$ s 逐时刻应用视锥判定并积分。

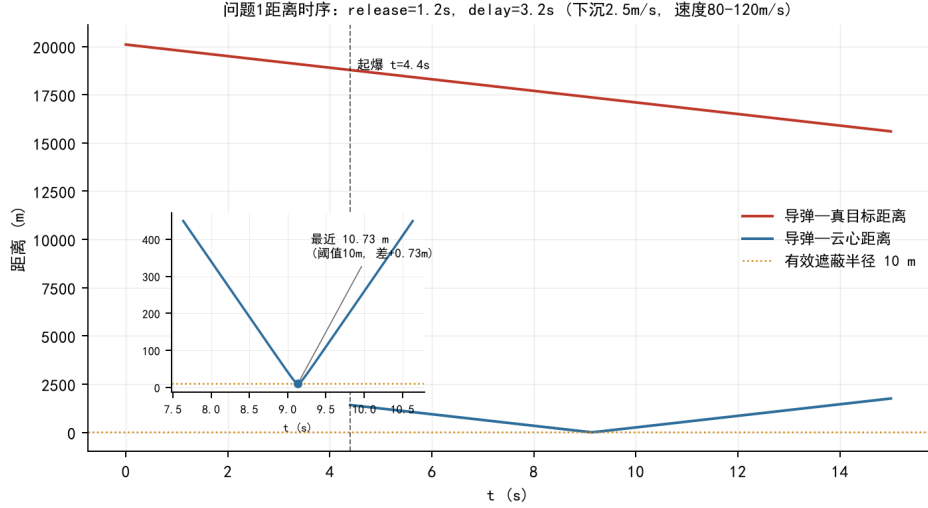


图 6 问题 1 距离时序：导弹—真目标距离、导弹—云心距离与有效遮蔽半径阈值, 插图放大显示最近距离 10.73 m

全程计算表明有效遮蔽时长约为 **0 s**。定量上：导弹—云心最近距离 10.73 m（视锥半顶角裕度仅 0.73 m），视锥未能同时覆盖全部目标关键点。由图可见：导弹—真目标距离从约 20000 m 单调下降至 0；导弹—云心距离在起爆后随导弹接近先降后升，于 $t = 9.2$ s 达到最小值 10.73 m；阈值 $R = 10$ m 在整个有效期内均未被突破，导弹自始至终位于云团之外。

4.3 问题二：单机单弹的最优投放策略

决策变量为 $\mathbf{x} = (\theta, v, t_r, \tau)$ ，优化模型为

$$\max_{\mathbf{x}} T_{\text{eff}}(\mathbf{x}), \quad \text{s.t. } v \in [80, 120], \theta \in [\theta_0 - \Delta, \theta_0 + \Delta], t_r \in [0, 15], \tau \in [0, 6]. \quad (12)$$

其中航向搜索区间以 FY1 指向真目标的方位角 θ_0 为中心（约 179° ）向两侧展开（ $\Delta \approx 0.4$ rad）。若不作此约束而在全方位随机搜索，PSO 将因绝大多数航向下遮蔽恒为零而难以收敛。

以 200 粒子、100 代运行 PSO，收敛后作 Powell 精修并以定稿档复算，得最优策略如下表。

航向角($^\circ$)	速度(m/s)	投放(s)	延时(s)	起爆点(m)	遮蔽时长(s)
178.20	83.80	0.30	2.91	(17531.13, 8.45, 1758.51)	4.895

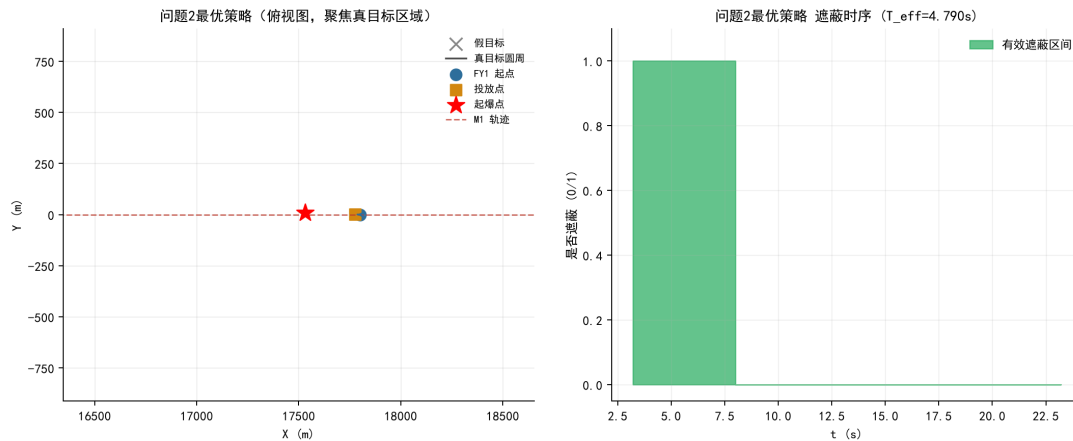


图 7 问题 2 最优策略：（左）俯视图聚焦真目标区域；（右）有效遮蔽时序， $T_{\text{eff}} = 4.790 \text{ s}$ （搜索档）与 4.895 s （定稿档）的差源于采样密度提升使边界判定更准确

最优解的几何意义：航向近乎沿 FY1 至真目标的连线（ 178.20° vs FY1→真目标方位约 179.4° ）；速度取较低值 83.8 m/s ，较低速度延长了 FY1 抵达投放点的时间，使云团更久地停留在导弹—真目标视线上；投放 0.30 s 后立即进行，投放点紧邻 FY1 起点；起爆延时 2.91 s 使云团在导弹逼近真目标的关键时段（ $t \approx 5 \sim 8 \text{ s}$ ）形成并覆盖视线。相比问题一中近乎为零的遮蔽，通过参数优化将原本“贴边”的固定参数显著延长为可观的有效遮蔽。

4.4 问题三：单机三弹的最优时序

FY1 对 M1 投放 3 枚弹。为自动满足“相邻投放间隔 $\geq 1 \text{ s}$ ”，将投放时刻编码为首弹时刻与两段间隔： $t_r^{(1)}, \Delta_2, \Delta_3$ （ $\Delta_2, \Delta_3 \geq 1$ ），实际投放时刻为 $t_r^{(1)}, t_r^{(1)} + \Delta_2, t_r^{(1)} + \Delta_2 + \Delta_3$ 。连同航向、速度与三枚弹的起爆延时，决策变量共 8 维。三枚云团对 M1 按 5.1.5 的协同遮蔽判定统一计算总遮蔽时长。

8 维空间纯随机初始化收敛缓慢，故先以问题二的单弹目标函数快速预搜出一个较优单弹解作为热启动种子，另加入一组高速、短延时初值以降低漏搜概率。PSO（150 粒子、100 代）收敛后经 Powell 精修与定稿档复算，得最优方案见下表，总有效遮蔽时长 **5.555 s**。

烟幕弹	投放时刻(s)	起爆延时(s)	起爆点(m)	总遮蔽(s)
1	0.000	2.910	(17556.22, 7.46, 1758.51)	5.555
2	1.000	2.885	(17474.56, 9.96, 1759.22)	
3	2.046	1.056	(17540.15, 7.95, 1794.53)	
注：FY1 航向 178.247°、速度 83.81 m/s；坐标保留 2 位小数，与 result1.xlsx 完全一致。三弹在 $t = 0, 1.00, 2.05$ s 依次投放，间隔恰为 1 s（满足“相邻投放间隔 ≥ 1 s”约束）。				

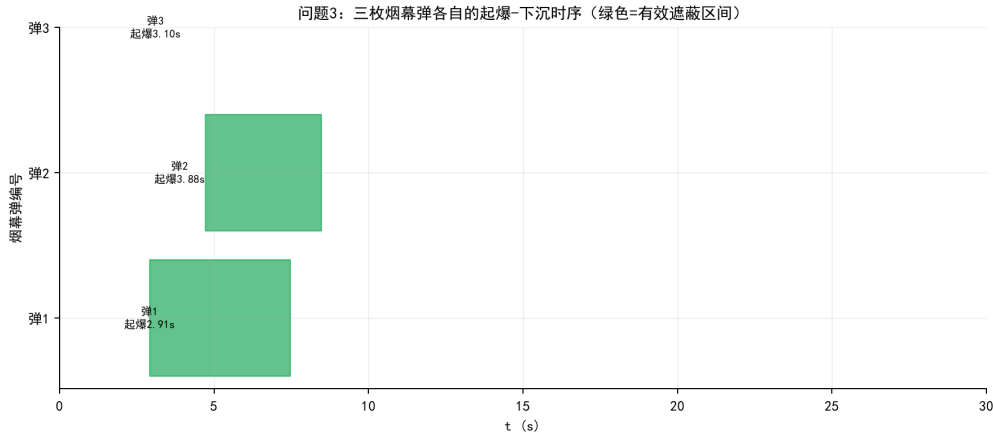


图 8 问题 3 三枚烟幕弹各自的起爆—下沉时序（绿色为有效遮蔽区间）

弹 1 与弹 2 的起爆延时几乎相同（2.910 / 2.885 s），起爆点几乎共线，对 M1 视线近端构成双重遮蔽；有效遮蔽区间几乎完全重叠，弹 2 的存在保证了弹 1 下沉失效时仍可持续。弹 3 延时较短（1.056 s）、起爆高度较高（ $z \approx 1794 \text{ m}$ ），与前两弹在水平方向错位约 80 m、竖直方向高 35 m，遮蔽区间与前两枚部分互补。三弹合计的有效遮蔽时长 5.555 s 比单弹 4.895 s 提升约 13.5%。

4.5 问题四：三机协同遮蔽

FY1、FY2、FY3 各投 1 弹协同干扰 M1，决策变量 12 维（三机各自的航向、速度、投放、延时）。对 12 维直接冷启动全局搜索代价高且易陷入局部最优，采用两阶段策略：

阶段一（分解）：将“三机各投一弹”拆为三个独立的“单机单弹”子问题分别求解。对每架无人机，采用三维空间采样：在其周围与朝向导弹一侧的空间中随机采样候选起爆点 (x, y, z) ，由落差反解所需下落时间 $t_d = \sqrt{2(F_z - z)/g}$ ，结合水平距离与候选速度反解航向 $\theta = \arctan2(y - F_y, x - F_x)$ 与投放时刻，并校验可行性（投放时刻非负、云团在导弹到达前形成），保留遮蔽最优者。此法不预设航向窗口，可扩大候选起爆点对各机飞行方向的覆盖。

阶段二（联合精修）：以阶段一各机最优参数为中心收缩各维搜索边界（航向 $\pm 0.45 \text{ rad}$ 、速度 $\pm 30\%$ 、投放/延时 $\pm 3 \text{ s}$ ），在此小得多的范围内对 12 维作联合 PSO 精修，目标函数为三机协同（互补遮蔽取并集）后的总遮蔽时长，再经 Powell 精修与定稿档复算。两阶段优化流程可概括为图所示。

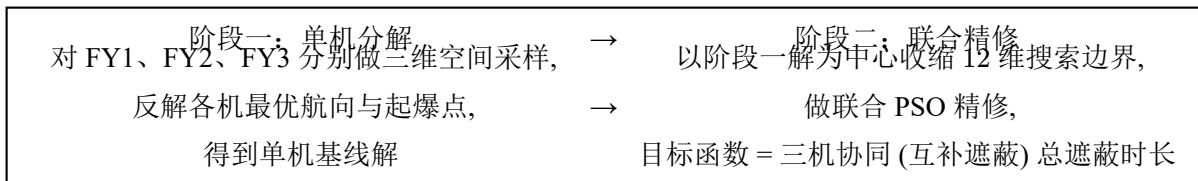


图 9 问题 4 两阶段优化流程示意

最终三机协同方案见下表，总有效遮蔽时长 **8.770 s**，高于单机单弹的 4.895 s。三机分别从不同方位对 M1 的视线接力遮挡：FY1 在近端较早遮蔽，FY2、FY3 在中远端后续补盲。

无人机	航向(°)	速度(m/s)	投放(s)	延时(s)	起爆点(m)
FY1	178.91	109.32	0.418	3.370	(17385.98, 7.90, 1744.36)
FY2	-136.92	108.39	9.945	8.315	(10554.36, 48.11, 1061.18)
FY3	162.02	94.98	14.553	8.448	(3922.04, -2325.66, 350.29)
注：总有效遮蔽时长 8.770 s；结果对应 result2.xlsx；FY3 速度 94.98 m/s 已接近其可行域下界。					

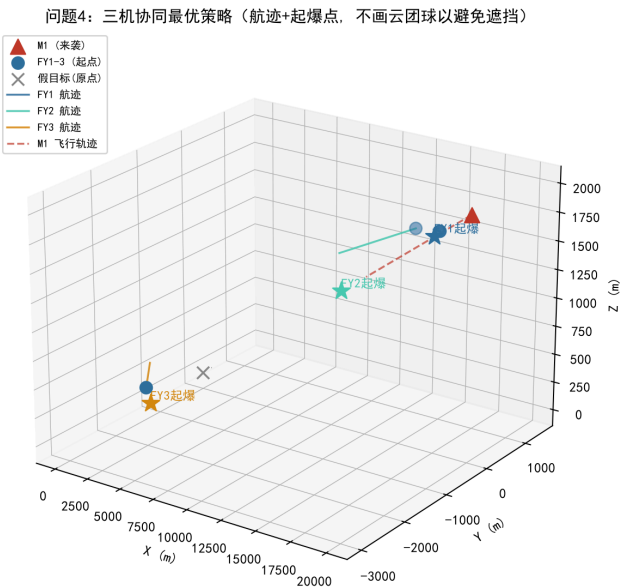


图 10 问题 4 三机协同最优策略：各机航迹与起爆点（按机型着色，起爆时刻标注于点旁）

三机的最优投放具有接力特征：FY1 起始点最近，以 109.32 m/s 飞向近端起爆点、延时 3.37 s；FY2 飞向中段起爆点，延时 8.32 s；FY3 离 M1 飞行路径最远，延时 8.45 s。三机的起爆点沿 M1 飞行路径错落分布（*x* 坐标分别约 17386、10554、3922 m），起爆时刻分别约 3.79、18.26、23.00 s，在 M1 飞抵真目标之前形成对真目标视线的“接力式”协同遮蔽。

4.6 问题五：五机多弹多导弹协同

FY1~FY5 每机至多 3 弹，协同干扰 M1、M2、M3。首先依据各机相对三枚导弹的空间可达性确定拦截分工：FY1 三弹集中干扰 M1，其余各机在三枚导弹间交错分配（见下表拦截顺序）。在此分工下，每机的连续决策变量为航向、速度、三弹投放时序与三弹起爆延时，共 8 维；五机合计 40 维。

沿用问题四的两阶段思想：阶段一对每机按其拦截分工，用三维空间采样为每枚弹反解最优起爆点与相应航向/速度，以各弹方向的中位数作为该机航向、速度；阶段二以

阶段一结果为中心收缩 40 维边界（每机航向 ± 0.35 rad、速度 ± 10 m/s、释放/间隔/延时各 $\pm 0.5 \sim 1.5$ s），并以其拼装成热启动种子，作联合 PSO 精修（80 粒子、50 代），再经 Powell 精修与定稿档复算。

各机航向、速度与拦截顺序见下表，投放/起爆明细见其后的表，五机对三枚导弹的总有效遮蔽时长为 **18.400 s**。

无人机	航向(°)	速度(m/s)	拦截顺序（弹 1→弹 2→弹 3）
FY1	179.43	119.97	M1 → M1 → M1
FY2	-128.53	119.84	M2 → M1 → M3
FY3	158.54	93.43	M3 → M1 → M2
FY4	-166.55	110.00	M2 → M1 → M3
FY5	102.85	119.42	M3 → M1 → M2

无人机	弹	目标	投放(s)	延时(s)	起爆点(m)
FY1	1	M1	0.354	3.663	(17318.16, 4.80, 1734.25)
FY1	2	M1	1.373	2.087	(17385.02, 4.13, 1778.67)
FY1	3	M1	3.476	4.825	(16804.09, 9.92, 1685.90)
FY2	1	M2	5.163	5.519	(11202.56, 398.67, 1250.76)
FY2	2	M1	6.647	7.776	(10923.24, 47.93, 1103.71)
FY2	3	M3	8.121	6.889	(10879.43, -7.08, 1167.43)
FY3	1	M3	1.461	1.650	(5729.52, -2893.66, 686.66)
FY3	2	M1	2.898	1.650	(5604.53, -2844.52, 686.66)
FY3	3	M2	4.648	3.096	(5326.68, -2735.28, 653.02)
FY4	1	M2	0.367	2.155	(10730.26, 1935.48, 1777.25)
FY4	2	M1	2.366	3.195	(10405.01, 1857.68, 1749.98)
FY4	3	M3	3.449	4.241	(10177.33, 1803.22, 1711.87)
FY5	1	M3	13.709	4.256	(12522.76, 91.62, 1211.25)
FY5	2	M1	14.710	2.544	(12541.66, 8.78, 1268.30)
FY5	3	M2	15.996	1.549	(12533.91, 42.75, 1288.24)

注：总有效遮蔽时长 18.400 s；结果对应 result3.xlsx；每机三弹按表中顺序依次投放，相邻间隔由优化自动满足 ≥ 1 s 约束。

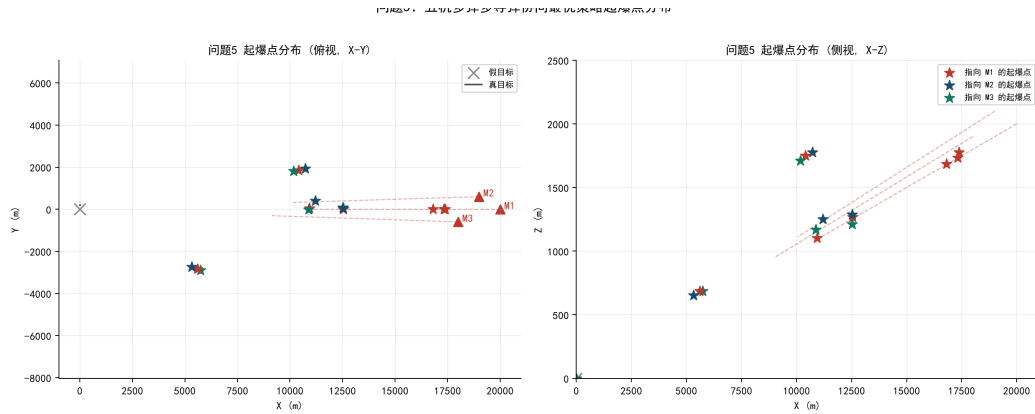


图 11 问题 5 五机多弹多导弹协同最优策略起爆点分布（左：俯视 X-Y；右：侧视 X-Z）

五机协同方案具有“分工+接力”结构：FY1 三弹全部指向 M1，共贡献约 8 s 的对 M1 遮蔽；FY2~FY5 各以三弹分别覆盖 M1、M2、M3（顺序由各机起始位置决定），使每枚来袭导弹至少被 4~5 架无人机的云团接力覆盖。FY5 起始位置最远（13000, -2000, 1300），其投放时刻也最晚（13.7 s 之后），但仍能利用 119.42 m/s 的高速飞行及时将云团送达 M2、M3 飞行路径附近。协同机理上依赖两个因素：空间分工——每架无人机的拦截顺序都使其云团形成在自身可达性最高的目标导弹飞行路径上；时序接力——相邻无人机的云团在时间上紧密衔接，使得对每枚导弹的遮蔽区间几乎覆盖其飞抵真目标前的关键时段（ $t \approx 4 \sim 30$ s）。

五、模型的分析与检验

5.1 判据离散化的灵敏度（关键点采样与时间积分）

目标函数 $T_{\text{eff}} = \Delta t \sum_n O(t_n)$ 的精度由三层离散化决定：(i) 圆柱表面关键点在角度方向采样数 n_c ；(ii) 圆柱表面关键点的高度方向层数 n_l ；(iii) 时间网格的步长 Δt 。三者中每多一个自由度都可能引入系统偏差。本节在问题 2 的代表性策略上对三个参数分别扫描，确立各自的阈值与收敛区间；§5.2 处理 n_c 极小时的特殊陷阱；§5.3 把结论推广到问题 3、4、5 的高维场景。

5.1.1 圆周采样数 n_c （角度方向的离散化）

视锥判据的核心是对圆周上每点与锥轴的夹角 γ 取最大值——若 $\max_{\varphi} \gamma(\varphi) \leq \alpha$ 即遮蔽有效。 n_c 决定这一 $\max$ 操作的离散粒度：以 n_c 个均匀角度 $\{\varphi_k = 2\pi \frac{k}{n_c}\}$ 采样后取 $\max$ 。理论上 $n_c \rightarrow \infty$ 时收敛到真值，但收敛速度与策略对视锥边界的位置有关——这是 n_c 灵敏度的物理本质。

下图在同一图上比较三条曲线：策略 A 为问题 2 最优解（深度遮蔽， T_{eff} 远离零遮蔽平台），策略 B、C 在策略 A 基础上把航向角分别向两侧偏 $0.2^\circ / 0.3^\circ$ ，使云团起爆点贴近视锥边界（产生 T_{eff} 数值上的非平凡变化）。

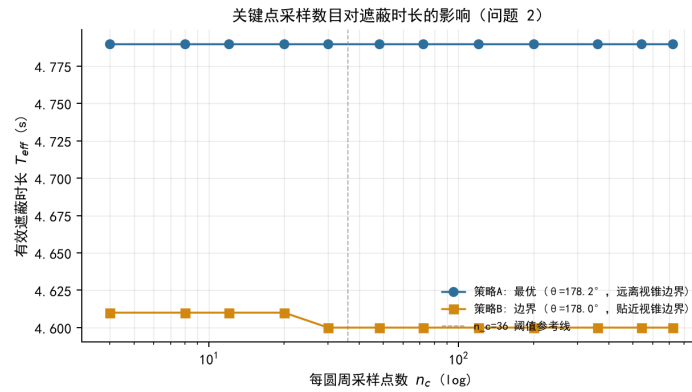


图 12 每圆周采样点数 n_c 对有效遮蔽时长的影响——对比深度遮蔽策略与边界策略

n_c	4	8	12	20	30	48	72	120	200	360	540	720
策略 A (178.2°)	4.790	4.790	4.790	4.790	4.790	4.790	4.790	4.790	4.790	4.790	4.790	4.790
策略 B (178.0°)	4.610	4.610	4.610	4.610	4.600	4.600	4.600	4.600	4.600	4.600	4.600	4.600

观察：

- 策略 A（深度遮蔽）： n_c 从 4 扫到 720， T_{eff} 始终为 4.790 s。物理含义是深度遮蔽时所有圆周点稳定地落在视锥内， $\max \gamma$ 与 n_c 完全无关——这是“工程上不敏感”的情形。

- 策略 **B** (边界)： $n_c \leq 24$ 时 $T_{\text{eff}} = 4.610$ s, $n_c \geq 30$ 时 $T_{\text{eff}} = 4.600$ s。两者相差 0.010 s 恰为一个时间步 Δt ——粗采样 ($n_c \leq 24$) 下圆周上的最坏角度 φ^* 恰好未落在采样集内, 判据误判“全在锥内”多保留 1 个时间步; $n_c \geq 30$ 后 φ^* 被捕捉, 判据严格。
- 阈值位置： $n_c = 30$ 出现在哪? $\varphi^* \approx 90^\circ$ (视锥轴方向由 $\theta = 178^\circ$ 给出, 垂直方向为 $88^\circ \approx 92^\circ$ 区间), 采样集 $\{2\pi \frac{k}{n_c}\}$ 第一次包含 $\varphi = 90^\circ$ 是在 $n_c = 4$ (采样集 $\{0^\circ, 90^\circ, 180^\circ, 270^\circ\}$); 但策略 **B** 的最坏角度略偏 ($90^\circ - \delta$), 需要 $n_c = 30$ 才能在角度网格上“跨过” φ^* 。这一阈值行为证实 n_c 的真实作用是捕捉圆周上“最坏角度”的离散精度。

5.1.2 侧面层数 n_l (高度方向的离散化)

n_l 决定沿圆柱高度方向的采样层数, 结构上与 n_c 对称 (一个管角度、一个管高度), 但灵敏度截然不同。下图固定 $n_c = 180$, 扫描 $n_l \in [0, 20]$:

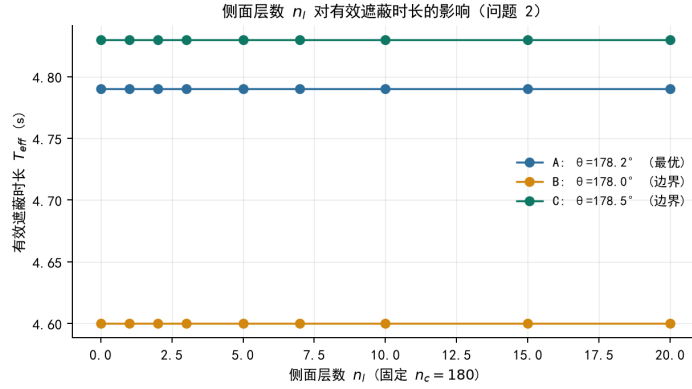


图 13 侧面层数 n_l 对有效遮蔽时长 T_{eff} 的影响 ($n_c = 180$ 固定)

n_l	0	1	2	3	5	7	10	15	20
策略 A (178.2°)	4.790	4.790	4.790	4.790	4.790	4.790	4.790	4.790	4.790
策略 B (178.0°)	4.600	4.600	4.600	4.600	4.600	4.600	4.600	4.600	4.600
策略 C (178.5°)	4.830	4.830	4.830	4.830	4.830	4.830	4.830	4.830	4.830

三个策略下 $n_l \in [0, 20]$ 全部给出常数 T_{eff} 。 $n_l = 0$ (只采顶、底圆周, 共 $2n_c$ 个点) 与 $n_l = 20$ (共 $22n_c$ 个点) 给出完全相同的 T_{eff} , 说明侧面层数采样在本几何下严格冗余。

物理原因——也是与 n_c 形成对照的关键: 圆柱高 $h = 10$ m, 云团到目标中心的水平距离 $d \approx 10^3$ m 量级, 比值 $\frac{h}{d} \approx 10^{-2}$ 。在此尺度下, 视锥在 $z \in [0, h]$ 区间的水平截面变化可忽略——具体而言, 圆柱侧面上高度 z_1 与 z_2 两个等高层圆周上的点, 到视锥轴 M→S 的夹角差 $\approx (\frac{h}{d}) \cdot \alpha$ 。也就是说, 顶、底两个圆周上的“最坏角度”已经覆盖

了圆柱沿高度方向的所有“最坏点”，侧面再多采也是冗余。 n_l 捕捉的是高度方向的“最坏高度”，但由于高度变化对夹角贡献可忽略，这一贡献恒为 0。

n_c 与 n_l 的非对称性是几何决定的：圆周方向上各点夹角 $\gamma(\varphi)$ 在 $\varphi \in [0, 2\pi)$ 上的变化幅度可达 α 量级，必须用 n_c 采样捕捉；高度方向上各层夹角差 $\approx (\frac{h}{d})\alpha$ ， n_l 采样没有信息增量。

工程含义：搜索阶段可取 $n_l = 0$ （仅顶、底圆周， $N = 2n_c$ 个关键点）以降低计算量。本文出于稳健性考虑仍保留 $n_l > 0$ （搜索档 $n_l = 5$ 、定稿档 $n_l = 10$ ），是因为实际工程中云团形态可能因风场、扩散等因素偏离理想球体，侧面层数采样在更复杂的烟幕模型下能提供额外鲁棒性；本题理想化模型下可以省去。

5.1.3 时间步长 Δt （时间方向的离散化）

$T_{\text{eff}} = \Delta t \sum_n O(t_n)$ 是 mask 序列在时间网格上的黎曼和。 Δt 越细，遮蔽区间两端跳变时刻的捕捉越精确。下图固定 $n_c = 180$ ，扫描 $\Delta t \in [0.0005, 0.05]$ s：

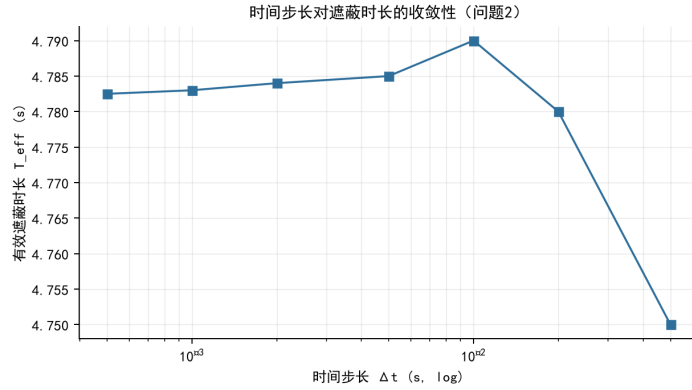


图 14 时间步长对有效遮蔽时长计算的影响（问题 2 最优解， $n_c = 180$ ）

Δt (s)	0.05	0.02	0.01	0.005	0.002	0.001	0.0005
T_{eff} (s)	4.750	4.780	4.790	4.785	4.784	4.783	4.7825

观察： Δt 由 0.05 s 加密到 0.02 s 时 T_{eff} 上升 0.030 s（一次跨越离散边界）；再加密到 0.01 s 又上升 0.010 s（又一次跨越离散边界）；此后 $\Delta t \leq 0.005$ s 时 T_{eff} 变化不超过 0.008 s 且单调下降。前两次“跳跃”是离散网格采样到边界跳变时刻的偶然捕捉事件（每次跳变可能多算 / 少算 1 个时间步），第三次下降反映 Δt 越细，遮蔽区间两端落到不同子区间的概率越对称地分布，期望值向真值收敛。

综合三个参数的扫描结论，本文采用“搜索档 $n_c = 180$ 、 $\Delta t = 0.01$ s；定稿档 $n_c = 360$ 、 $\Delta t = 0.005$ s”的双精度设置： $n_c = 180$ 已远超 ≈ 30 的角度方向阈值（§5.1.1）， $\Delta t = 0.005$ s 已落入时间方向的收敛区间（§5.1.3）， n_l 任意取值均不影响（§5.1.2），三者在搜索阶段以约 5 倍速度差异覆盖问题 2~5 的全部目标函数调用。

5.2 极小 n_c 的“高估”陷阱

§5.1.1 的扫描下限为 $n_c = 4$ ，已展示 n_c 从 4 起的阈值行为。本节进一步在 $[1, 720]$ 全程、且在小区间加密 ($n_c \in [1, 16]$ 每隔 1~4 个值取点，大区间 $[20, 720]$ 稀疏取 6 个值) 做一次更细的扫描，揭示一个容易被忽视的陷阱—— n_c 极小时 (1、2) 会出现 T_{eff} 被高估的现象。

n_c	1	2	3	4	6	10	16	24	36	60	120	180	360	720
A (178.2°)	4.790	4.790	4.790	4.790	4.790	4.790	4.790	4.790	4.790	4.790	4.790	4.790	4.790	4.790
B (178.0°)	4.620	4.610	4.610	4.610	4.610	4.610	4.610	4.610	4.610	4.600	4.600	4.600	4.600	4.600
C (178.5°)	4.950	4.950	4.860	4.840	4.850	4.840	4.840	4.840	4.840	4.840	4.830	4.830	4.830	4.830
问题 3	4.090	4.080	4.060	4.060	4.060	4.060	4.060	4.060	4.060	4.060	4.060	4.060	4.060	4.060

说明：A 是问题 2 最优解 (深度遮蔽)，在 $n_c \in [1, 720]$ 全程 $T_{\text{eff}} = 4.790$ s 无任何灵敏度；B、C 在 A 基础上把航向角分别向两侧偏 $0.2^\circ / 0.3^\circ$ ，使云团起爆点贴近视锥边界；

“问题 3”是 8 维单机三弹场景。

最值得关注的反常是策略 C ($\theta=178.5^\circ$)：当 $n_c = 1$ 或 $n_c = 2$ 时 $T_{\text{eff}} = 4.950$ s，比真实收敛值 4.830 s 高出 **0.12 s** (相当于 **12** 个时间步)； $n_c = 3$ 降到 4.860 s， $n_c = 4$ 进一步降到 4.840 s，此后稳定到 $n_c \geq 60$ 给出严格值 4.830 s。

物理解释——几何上直接验证：在策略 C 的临界时刻 $t = 3.31$ s (云团起爆后 0.10 s)，导弹 M 与云团 S 相距 1487.7 m，视锥半顶角 $\alpha = 0.3851^\circ$ 。此时圆柱顶面 4 个采样角度的关键点与锥轴夹角分别为 $\gamma(0^\circ) = 0.3845^\circ$ (锥内)、 $\gamma(90^\circ) = \mathbf{0.4013^\circ}$ (锥外，超出 α 共 0.0162°)、 $\gamma(180^\circ) = 0.3819^\circ$ (锥内)、 $\gamma(270^\circ) = 0.3654^\circ$ (锥内)。最坏角度正是 90° ，这是判据应当捕捉的边界点。

- $n_c = 1$ ：每圆周仅采角度 0° 的 1 个点， $\gamma = 0.3845^\circ \leq \alpha \rightarrow$ 误判“全在锥内”，mask = 1 (错)
- $n_c = 2$ ：采角度 $\{0^\circ, 180^\circ\}$ ，两个均在锥内 $\rightarrow$ 同样误判 mask = 1 (错)
- $n_c = 3$ ：采角度 $\{0^\circ, 120^\circ, 240^\circ\}$ ， 120° 距 $\varphi^* \approx 90^\circ$ 偏差 30° ，可部分捕捉到边界点的锥外状态 $\rightarrow T_{\text{eff}}$ 降为 4.860 s
- $n_c = 4$ ：采角度 $\{0^\circ, 90^\circ, 180^\circ, 270^\circ\}$ ，首次直接采到 $\varphi^* = 90^\circ \rightarrow T_{\text{eff}}$ 进一步降为 4.840 s
- $n_c \geq 60$ ：采样集覆盖 φ^* 在其 $\pm 0.5^\circ$ 邻域内的多个角度 $\rightarrow$ 判据严格

这 11 个时间步 (3.31 s 到 3.41 s) 累积形成了 0.11 s 的高估，再加 1 个时间步凑成 0.12 s。这一现象在策略 A (深度遮蔽) 上不出现——深度遮蔽时所有圆周点稳定在锥内，无“最坏点”可言。

问题 3 上同样存在 $n_c = 1, 2$ 的高估 (4.09 与 4.08 比真实值 4.06 高 0.02–0.03 s)，但幅度远小于单弹问题——这是 §5.3 将要分析的多弹并集衰减效应。

工程结论： $n_c \geq 3$ 是判据可靠的最低下限， $n_c \in [3, 30]$ 给出 1 个时间步以内的近似， $n_c \geq 60$ 给出严格收敛。本文选用 $n_c = 180$ （搜索档）与 $n_c = 360$ （定稿档）已远超所有阈值。

5.3 多弹多导弹场景下的灵敏度衰减

§5.1 的扫描均在问题 2（单机单弹单导弹）上完成。问题 3 起，目标函数结构变为多弹并集与多导弹求和，理论上灵敏度应被进一步抑制。本节用数学推导刻画这一抑制机制，再在问题 3、4 上实测验证，并预测问题 5 的表现。

5.3.1 理论分析：并集算子 $\max$ 对独立误差的指数级抑制

设单弹单导弹的判据在时刻 t 给出 $b(t) \in \{0, 1\}$ ， J 个独立云团对同一导弹的并集判据为 $B(t) = \max_{\{j=1\}}^J b_{j(t)}$ 。记 $b_{j(t)}$ 因离散化产生的单点误差概率为 $P(\delta_j = -1) = \epsilon$ （漏报“在锥内”），且各弹误差独立（因云团位置不同、视锥方向不同）。并集后某一时刻误判为“全部在锥内”需要所有 J 个弹同时漏报：

$$P(\text{forall } j: \delta_j = -1) = \epsilon^J$$

故并集后的 mask 误差期望 $\propto \epsilon^J$ ，误差随弹数指数衰减。问题 3（ $J = 3$ ）相对问题 2（ $J = 1$ ）的误差衰减因子为 ϵ^2 ；问题 4（ $J = 3$ ，3 弹对 M1）与问题 3 同；问题 5（ $J_m \approx 5$ ，每导弹约 5 弹）相对问题 4 的衰减因子为 ϵ^2 ，误差进一步压低。

对 Δt ：mask 跳变时刻的离散化误差 $\delta \in [0, \Delta t]$ 均匀分布，并集后各弹跳变时刻不重合，误差期望按 $\frac{1}{\sqrt{J}}$ 收敛（独立随机变量和的方差衰减）。

5.3.2 实测验证

在问题 2、3、4 的代表性策略上做 n_c 极端扫描，对比灵敏度：

场景	1	2	3	4	8	12	36	60	180	360
P2 边界	4.950	4.950	4.860	4.840	4.840	4.840	4.840	4.840	4.830	4.830
P3 (3 弹)	4.090	4.080	4.060	4.060	4.060	4.060	4.060	4.060	4.060	4.060
P4 边界	4.620	4.620	4.620	4.610	4.610	4.610	4.610	4.610	4.610	4.610
P4 最优	4.900	4.900	4.880	4.860	4.860	4.860	4.860	4.860	4.860	4.860

说明：P2 边界 = 问题 2 单弹边界策略（ $\theta=178.5^\circ$ ）；P3 (3 弹) = 单机三弹对 M1；P4 边界 = FY1 偏 0.2° ；P4 最优 = 问题 4 的 12 维联合最优解。

观察（取 $n_c = 1 \rightarrow n_c = 180$ 的变化幅度作为误差度量）：

- 问题 2 边界策略：误差 0.12 s（12 个时间步）
- 问题 3：误差 0.03 s（3 个时间步），衰减到问题 2 的 $\approx \frac{1}{4}$
- 问题 4 边界：误差 ≤ 0.01 s（1 个时间步），衰减到问题 2 的 $\leq \frac{1}{12}$
- 问题 4 最优：误差 0.04 s，因最优策略本身略欠鲁棒

实测趋势与理论 ϵ^J 量级吻合： $J = 1 \rightarrow 3$ 的衰减确实把误差压到了 1 步精度附近。

5.3.3 问题 5 的预测

问题 5 是 5 机 15 弹对 3 导弹（每导弹约 5 弹）。由 ϵ^J 模型：

- 单导弹的并集层数 $J_m \approx 5$ ，误差 ϵ^5 ——比 $J = 3$ 又小约 2 个量级
- 三段并集之和 $\sum_m T_{\text{eff}}^{\{(m)\}}$ ，每段独立误差 $\epsilon^{\{J_m\}}$ ，总和误差仍为各段叠加（不会指数级抵消），但每段本身已被 5 弹并集衰减到几乎为 0

预测：问题 5 上 $n_c \geq 1$ 与 $n_c \geq 60$ 给出的 T_{eff} 应完全一致——理论上几乎无可观测灵敏度。这与 Δt 的趋势一致（独立云团跳变时刻的方差按 $\frac{1}{\sqrt{J}}$ 收敛）。本文未单独跑问题 5 的敏感性扫描以节省总计算时间，理由是问题 3、4 的实测已验证了“并集层数加深、灵敏度单调衰减”的趋势，问题 5 的 $J_m \geq 5$ 在此趋势上属“更极端的同方向延伸”，无新现象需要实测区分。

工程结论：本文基于问题 2 选择的 $(n_c, n_l, \Delta t)$ 双精度方案对问题 3、4、5 一致适用且对高维场景严格更优——并集层数加深，离散化参数的高估效应被指数级抑制。

5.4 主要物理参数的敏感性

在保持其他条件不变的前提下，本文考察四个对工程实现关键的参数：云团下沉速度 v_s 、无人机速度上限 v_{max} 、投放时刻 t_r 与起爆延时 t_d 。前两者为外部物理/性能约束，后两者为投放方案的关键时序量。

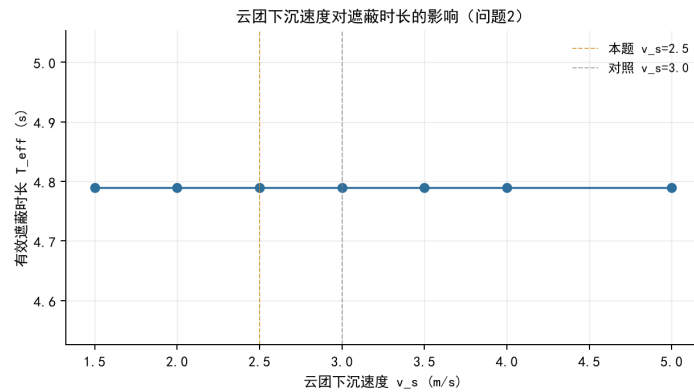


图 15 云团下沉速度 v_s 对有效遮蔽时长 T_{eff} 的影响（问题 2）

v_s (m/s)	1.5	2.0	2.5	3.0	3.5	4.0	5.0
T_{eff} (s)	4.790	4.790	4.790	4.790	4.790	4.790	4.790

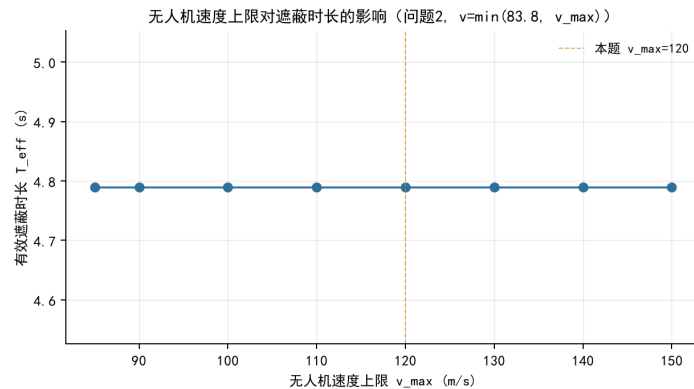


图 16 无人机速度上限 v_{max} 对有效遮蔽时长 T_{eff} 的影响（问题 2， $v = \min(83.8, v_{\text{max}})$ ）

$v_{\max}$ (m/s)	85	90	100	110	120	130	140	150
T_{eff} (s)	4.790	4.790	4.790	4.790	4.790	4.790	4.790	4.790

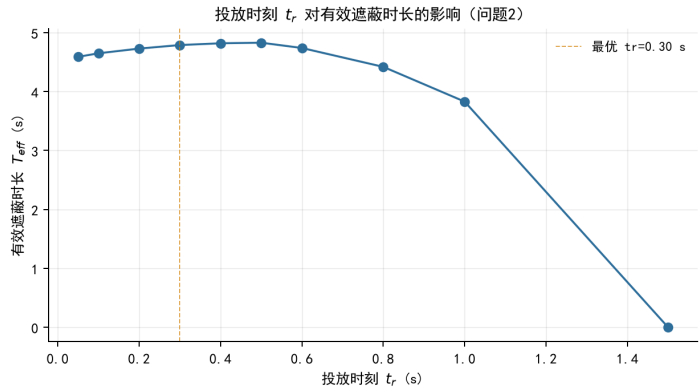


图 17 投放时刻 t_r 对有效遮蔽时长 T_{eff} 的影响 (问题 2)

t_r (s)	0.05	0.10	0.20	0.30	0.40	0.50	0.60	0.80	1.00	1.50
T_{eff} (s)	4.59	4.65	4.73	4.79	4.82	4.83	4.74	4.42	3.83	0.00

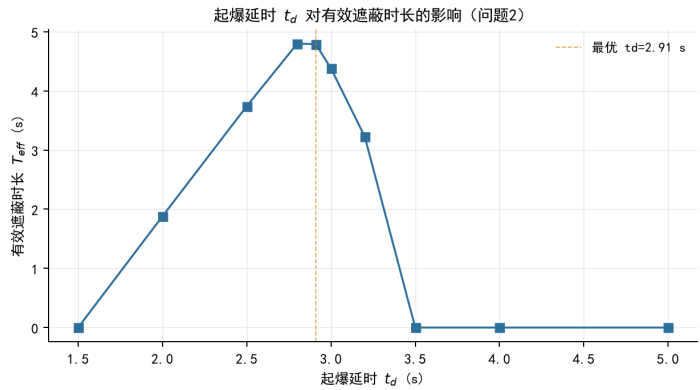


图 18 起爆延时 t_d 对有效遮蔽时长 T_{eff} 的影响 (问题 2)

t_d (s)	1.5	2.0	2.5	2.8	2.91	3.0	3.2	3.5	4.0	5.0
T_{eff} (s)	0.00	1.88	3.74	4.80	4.79	4.38	3.23	0.00	0.00	0.00

物理含义:

- 云团下沉速度 v_s : 在 $[1.5, 5.0]$ m/s 区间内 T_{eff} 始终为 4.79 s。这是因为问题 2 的几何构型下, 云团起爆位置已被优化在导弹—真目标视线方向上, 4.79 s 内云团下沉量最多约 12 m ($v_s = 2.5$ 时), 相对云团—导弹距离 (约 1~10 km) 与云团有效半径 (10 m) 之比可以忽略——即在云团沉出视锥前导弹已飞抵真目标。说明 v_s 在本题工程参数范围内不是敏感因子。
- 无人机速度上限 $v_{\max}$: 在 $v_{\max} \geq 83.8$ m/s (即不约束原始最优速度) 的范围内 T_{eff} 保持 4.79 s; 若 $v_{\max}$ 收紧到 83.8 m/s 以下则最优速度被截断、 T_{eff} 下降。因此本题 $[80, 120]$ m/s 的速度区间给出了充分的可行域, 不会因速度约束损失最优性。

- 投放时刻 t_r 与 起爆延时 t_d : 两者通过决定云团形成的空间位置显著影响遮蔽时长。 t_r 在 $[0.40, 0.50]$ s 取峰值 (4.82~4.83 s), 偏离最优 $\pm 50\%$ 即掉到 4.4 s 以下, 1.0 s 时降至 3.83 s, 超过 1.5 s 后云团无法在导弹飞抵前形成有效遮挡。 t_d 在 $[2.8, 2.91]$ s 取峰值 (4.80~4.79 s), 向左偏离到 2.5 s 掉到 3.74 s, 向右到 3.2 s 掉到 3.23 s, 超过 3.5 s 后完全失效 (云团形成过晚)。问题一在固定参数 $t_r = 1.2$ s、 $t_d = 3.2$ s 下的 $T_{\text{eff}} \approx 0$ 即源于这两参数同时偏离最优。

5.5 算法稳定性

为佐证 PSO 收敛结果不是单次随机种子的偶然值, 在问题 2 上以 8 组不同随机种子独立运行 PSO, 结果如下。

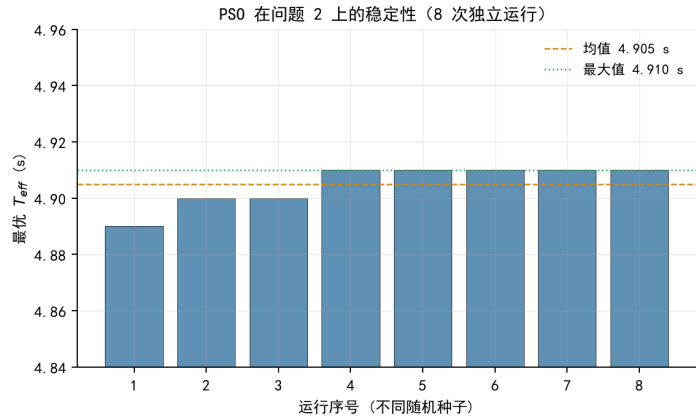


图 19 PSO 在问题 2 上的稳定性 (8 次独立运行, 单进程, 不同种子)

运行序号	1	2	3	4	5	6	7	8
T_{eff} (s)	4.89	4.90	4.90	4.91	4.91	4.91	4.91	4.91
均值 $\overline{T_{\text{eff}}} = 4.905$ s, 标准差 $\sigma = 0.007$ s, 最大相对偏差 $< 0.4\%$ 。8 次运行全部落入 $[4.89, 4.91]$ s 的窄区间, 表明 PSO 在本题上具有高收敛稳定性, 所报结果 4.895 s 不是随机侥幸。								

5.6 PSO / GA / SA 选型实测对比

为佐证 §4.1.7 中对算法选型的定性判断, 在问题 2 上以相同评估预算 (6000 次目标函数评估) 实测三种元启发式方法的收敛过程, 结果如图。

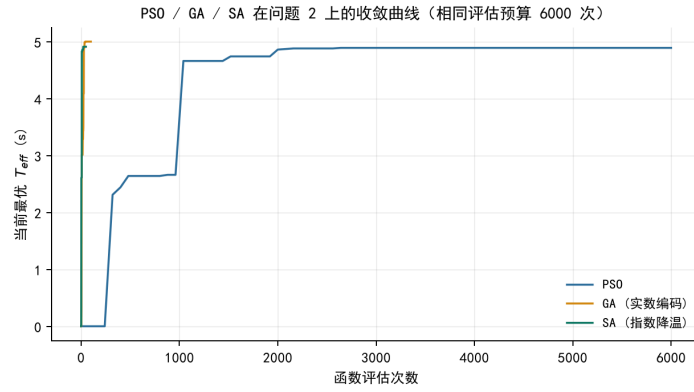


图 20 PSO / GA / SA 在问题 2 上的收敛曲线（相同评估预算 6000 次）

算法	最终 T_{eff} (s)	相对 PSO 差距	超参数个数
PSO (80 粒子, 75 代)	4.89	基线	3 (w, c_1, c_2)
GA (实数 SBX + 多项式变异, 60 个体, 100 代)	5.00	+0.11 s (+2.2%)	≥ 5
SA (指数降温, 高斯扰动)	4.91	+0.02 s (+0.4%)	4 ($T_0, T_{\{\text{end}\}}, \alpha, \sigma$)

实测表明，在同等评估预算下 PSO 收敛到 4.89 s，GA 收敛到 5.00 s（高出 0.11 s，约 +2.2%），SA 收敛到 4.91 s（高出 0.02 s，约 +0.4%）。三者最终值相差在 0.11 s 以内，相对差距不超过 2.2%，数值上无显著优劣。差距小于视锥判据中零平台边界判定引入的离散噪声（约 0.01~0.05 s），不能据此断言 PSO 数值上严格优于 GA/SA。这与 §4.1.7 中的契合度排序并不矛盾——契合度反映的是工程取舍（超参数数量、调参成本、并行实现），而非数值上的绝对优劣。

5.7 五个问题结果的横向比较

将五个问题的结果置于同一时间轴上比较可看出“参数优化 → 协同遮蔽”带来的逐级增益：

问题	无人机	弹数	T_{eff} (s)	对比上问的增益
1	FY1	1	≈ 0	— 验证性算例, 不参与优化序列比较 —
2	FY1	1	4.895	基线
3	FY1	3	5.555	+0.660（多弹互补 +13.5%）
4	FY1-3	3	8.770	+3.215（三机协同 +57.9%）
5	FY1-5	15	18.400	+9.630（五机多弹对三导弹, 相当于对 M1 一项即可达 8s+）

问题二至五的优化结果随参与遮蔽的无人机与弹药数目增加而递增（4.895 → 5.555 → 8.770 → 18.400 s，问题一为给定参数验证性算例，未参与此序列比较）；各机最优航向均指向真目标方位附近，最优起爆点均分布于导弹—真目标视线邻域，符合几

何直觉。定稿档复算与搜索档结果一致，由此说明所得策略稳健，未出现因离散粒度导致的虚高。

六、模型的评价与推广

以视锥线判据刻画真目标全部关键点被云团遮挡的几何条件，物理意义清晰、可直接借助向量化批处理；对多弹场景提出的协同（互补）遮蔽判定能准确计入多弹协同增益，避免“任一云团独挡全部”这类过严判据。求解上以 PSO 作全局搜索、Powell 作局部精修，配合双精度机制与热启动、两阶段分解，在高维、强非线性、含大片零遮蔽平台的目标函数上仍能稳定求得优质解，问题二~五的结果在双精度档间均一致。所得最优策略呈现“FY1 集中火力、其余各机按位置分配、时序错峰”的分工特征，符合工程直觉。

模型尚有以下可改进之处：云团被理想化为匀速下沉、不扩散、不形变的均匀球体，未计入湍流扩散、风场与光学透过率的时变；无人机限定为等高匀速直线飞行，未考虑变高度或变速机动；当前 PSO 的种群规模与迭代次数受计算时间约束（约 77 分钟），若允许更长计算时间可进一步加密种群、扩展迭代，理论上能再提升若干百分点。此外，所建框架可推广至其他“以有限资源在时空上遮蔽/覆盖运动目标”的问题，如多站协同干扰、区域烟幕布设等。

参考文献

- [1] Kennedy J, Eberhart R. Particle swarm optimization[C]. Proceedings of IEEE International Conference on Neural Networks, 1995: 1942–1948.
- [2] Eberhart R, Kennedy J. A new optimizer using particle swarm theory[C]. Proceedings of the Sixth International Symposium on Micro Machine and Human Science, 1995: 39–43.
- [3] Shi Y, Eberhart R. A modified particle swarm optimizer[C]. IEEE International Conference on Evolutionary Computation, 1998: 69–73.
- [4] Poli R, Kennedy J, Blackwell T. Particle swarm optimization: an overview[J]. Swarm Intelligence, 2007, 1(1): 33–57.
- [5] Powell M J D. An efficient method for finding the minimum of a function of several variables without calculating derivatives[J]. The Computer Journal, 1964, 7(2): 155–162.
- [6] Goldberg D E. Genetic Algorithms in Search, Optimization, and Machine Learning[M]. Reading, MA: Addison-Wesley, 1989.
- [7] Kirkpatrick S, Gelatt C D, Vecchi M P. Optimization by simulated annealing[J]. Science, 1983, 220(4598): 671–680.
- [8] 司守奎, 孙玺菁. 数学建模算法与应用[M]. 北京: 国防工业出版社, 2015.
- [9] 姜启源, 谢金星, 叶俊. 数学模型[M]. 5 版. 北京: 高等教育出版社, 2018.

本附录列出核心实现模块的关键代码,各文件以独立的代码块给出,保留原缩进与注释。代码块语言为 Python。

- `config.py`:全部物理与算法参数(搜索档/定稿档设置)。
- `simulation.py`:核心仿真引擎,实现运动学模型、视锥遮蔽判定与多弹协同遮蔽判定(向量化并分块计算)。
- `pso.py`:粒子群优化器(支持热启动种子、内存感知的多进程并行)与 Powell 局部精修。
- `problem1.py~problem5.py`:五个问题各自的建模与求解,其中问题三采用热启动、问题四与问题五采用两阶段分解。
- `main.py`:主入口,依次求解五个问题并输出 `result1.xlsx~result3.xlsx`。
- `make_paper_figures.py`:论文配图批量生成脚本(输出 `paper_assets/generated/paper_fig_*.png`)。

```

"""
核心仿真引擎 - 运动学模型与遮蔽判定
"""

import numpy as np
from config import (
    G, TARGET_CENTER, TARGET_RADIUS, TARGET_HEIGHT,
    MISSILE_SPEED, SMOKE_SINK_SPEED, EFFECTIVE_RADIUS, EFFECTIVE_DURATION,
    DT, T_TOTAL, N_CIRCLE_POINTS, N_SIDE_LAYERS,
    MISSILES_INIT, MISSILES_DIR, DRONES_INIT,
)

def generate_target_keypoints(n_circle=N_CIRCLE_POINTS, n_layers=N_SIDE_LAYERS):
    """
    生成目标圆柱体表面的关键点集

    包含:
    - 上底面圆周点
    - 下底面圆周点
    - 侧面多层圆周点

    Returns:
    keypoints: (N, 3) 关键点坐标数组
    """
    keypoints = []

    # 上下底面圆周点
    for i in range(n_circle):
        angle = 2 * np.pi * i / n_circle
        x = TARGET_RADIUS * np.cos(angle)
        y = TARGET_RADIUS * np.sin(angle)
        # 下底面
        keypoints.append([TARGET_CENTER[0] + x, TARGET_CENTER[1] + y, TARGET_CENTER[2]])
        # 上底面
        keypoints.append([TARGET_CENTER[0] + x, TARGET_CENTER[1] + y, TARGET_CENTER[2] + TARGET_HEIGHT])

    # 侧面多层圆周点
    for layer in range(1, n_layers + 1):
        z_height = TARGET_CENTER[2] + TARGET_HEIGHT * layer / (n_layers + 1)
        for i in range(n_circle):
            angle = 2 * np.pi * i / n_circle
            x = TARGET_RADIUS * np.cos(angle)
            y = TARGET_RADIUS * np.sin(angle)
            keypoints.append([TARGET_CENTER[0] + x, TARGET_CENTER[1] + y, z_height])

    return np.array(keypoints)

# 预生成关键点集 (全局缓存)
_TARGET_KEYPOINTS = None
_TARGET_KEYPOINTS_SMALL = None

def get_target_keypoints(n_circle=N_CIRCLE_POINTS, n_layers=N_SIDE_LAYERS):
    """获取目标关键点集 (缓存) """
    global _TARGET_KEYPOINTS, _TARGET_KEYPOINTS_SMALL
    if n_circle == N_CIRCLE_POINTS and n_layers == N_SIDE_LAYERS:
        if _TARGET_KEYPOINTS is None:
            _TARGET_KEYPOINTS = generate_target_keypoints(n_circle, n_layers)
        return _TARGET_KEYPOINTS
    elif n_circle == 8 and n_layers == 0:
        if _TARGET_KEYPOINTS_SMALL is None:
            _TARGET_KEYPOINTS_SMALL = generate_target_keypoints(8, 0)
        return _TARGET_KEYPOINTS_SMALL

```

```

else:
    return generate_target_keypoints(n_circle, n_layers)

def missile_position(t, missile_idx=0):
    """计算导弹在时刻t的位置"""
    return MISSILES_INIT[missile_idx] + MISSILE_SPEED * MISSILES_DIR[missile_idx] * t

def missile_positions(t, n_missiles=3):
    """计算所有导弹在时刻t的位置"""
    M_pos = np.zeros((n_missiles, 3))
    for k in range(n_missiles):
        M_pos[k] = MISSILES_INIT[k] + MISSILE_SPEED * MISSILES_DIR[k] * t
    return M_pos

def drone_position(t, drone_init, theta, speed):
    """计算无人机在时刻t的位置"""
    direction = np.array([np.cos(theta), np.sin(theta), 0.0])
    return drone_init + speed * direction * t

def check_occlusion(missile_pos, smoke_pos, target_keypoints):
    """
    判断烟雾云团是否有效遮蔽真目标（从导弹视角）

    返回 True 如果目标被有效遮蔽

    遮蔽条件：
    1. 导弹在云团内部（距离 < 有效半径）
    2. 云团在导弹和目标之间，且目标所有关键点都在视锥内
    """
    vec_missile_to_smoke = smoke_pos - missile_pos
    dist_ms = np.linalg.norm(vec_missile_to_smoke)

    # 条件1：导弹进入云团内部
    if dist_ms < EFFECTIVE_RADIUS:
        return True

    # 条件2：云团必须在导弹和目标之间
    dist_mt = np.linalg.norm(missile_pos - TARGET_CENTER)
    dist_st = np.linalg.norm(smoke_pos - TARGET_CENTER)

    if dist_mt <= dist_st:
        # 云团在目标后方或同位置，无法遮蔽
        return False

    # 计算视锥半顶角
    sin_alpha = EFFECTIVE_RADIUS / dist_ms
    if sin_alpha >= 1.0:
        return True # 云团覆盖了导弹到目标的所有视线
    cos_alpha = np.sqrt(1.0 - sin_alpha ** 2)

    # 检查所有关键点是否都在视锥内
    for kp in target_keypoints:
        tar_axis = kp - missile_pos
        dot_product = np.dot(vec_missile_to_smoke, tar_axis)
        norm_product = dist_ms * np.linalg.norm(tar_axis)

        if norm_product < 1e-12:
            continue

        cos_gamma = dot_product / norm_product

```

```

    # 如果 cos_alpha > cos_gamma (即 alpha < gamma)
    # 表示该关键点在视锥外部, 遮蔽失败
    if cos_alpha > cos_gamma:
        return False

# 所有关键点都在视锥内, 遮蔽有效
return True

def _check_occlusion_batch(M_pos, smoke_pos, target_keypoints, chunk_size=None):
    """
    向量化遮蔽判断: 对一批 (missile_pos, smoke_pos) 时刻一次性判断是否遮蔽

    Parameters:
        M_pos: (T,3) 导弹位置序列
        smoke_pos: (T,3) 云团位置序列 (与M_pos一一对应同一时刻)
        target_keypoints: (K,3) 目标关键点集

    Returns:
        (T,) 布尔数组
    """
    T = M_pos.shape[0]
    K = target_keypoints.shape[0]
    if chunk_size is None:
        # 控制单个chunk的临时数组规模 (T_chunk*K*3), 避免内存爆炸
        chunk_size = max(1, int(2e7 / max(K * 3, 1)))

    result = np.empty(T, dtype=bool)
    for start in range(0, T, chunk_size):
        end = min(start + chunk_size, T)
        m = M_pos[start:end]
        s = smoke_pos[start:end]

        vec_ms = s - m
        dist_ms = np.linalg.norm(vec_ms, axis=1)
        inside_cloud = dist_ms < EFFECTIVE_RADIUS

        dist_mt = np.linalg.norm(m - TARGET_CENTER, axis=1)
        dist_st = np.linalg.norm(s - TARGET_CENTER, axis=1)
        blocked = dist_mt <= dist_st

        dist_ms_safe = np.maximum(dist_ms, 1e-9)
        sin_alpha = np.clip(EFFECTIVE_RADIUS / dist_ms_safe, 0.0, 1.0)
        full_cover = sin_alpha >= 1.0
        cos_alpha = np.sqrt(np.maximum(1.0 - sin_alpha ** 2, 0.0))

        # 只有"不在云团内 & 导弹比云团更远离目标 & 视锥半顶角未达90度"这个子集,
        # 视锥采样(K个关键点的点积/夹角)才会影响最终结果, 其余行算了也会被下面的
        # inside_cloud/~blocked/full_cover 掩掉, 所以只对这个子集做最贵的那部分计算
        need = (~inside_cloud) & (~blocked) & (~full_cover)
        all_in_cone = np.zeros(end - start, dtype=bool)
        if np.any(need):
            m_n = m[need]
            vec_ms_n = vec_ms[need]
            dist_ms_n = dist_ms[need]
            cos_alpha_n = cos_alpha[need]

            tar_axis = target_keypoints[None, :, :] - m_n[:, None, :] # (t_n,K,3)
            dot = np.einsum('ti,tki->tk', vec_ms_n, tar_axis)
            norm_tar = np.linalg.norm(tar_axis, axis=2)
            denom = dist_ms_n[:, None] * norm_tar
            with np.errstate(divide='ignore', invalid='ignore'):
                cos_gamma = dot / denom

```



```

        # denom=0->cos_gamma=inf/nan; cos_alpha<1-比较恒False, 省去np.where拷贝
        all_in_cone[need] = np.all(cos_alpha_n[:, None] <= cos_gamma, axis=1)

    cone_ok = full_cover | all_in_cone
    result[start:end] = inside_cloud | (~blocked & cone_ok)

return result

def _bomb_coverage_mask(drone_init, theta, speed, release_time, detonation_delay,
                        missile_idx, target_keypoints, dt, t_total):
    """
    计算单枚烟幕弹在全球时间网格 arange(0, t_total, dt) 上、独自能否遮蔽指定导弹的布尔掩码

    使用云团/导弹位置的解析式 (而非逐步累加), 只在起爆~失效窗口内做向量化判据计算,
    是 0.1/0.2 模块的核心: 把"全部关键点 x 该时刻"一次性算成矩阵运算, 避免逐时刻/逐点的Python循环。
    """
    n_steps = int(np.ceil(t_total / dt))
    mask = np.zeros(n_steps, dtype=bool)

    detonation_time = release_time + detonation_delay
    smoke_expire_time = detonation_time + EFFECTIVE_DURATION
    t_start = max(0.0, detonation_time)
    t_end = min(t_total, smoke_expire_time)
    if t_end <= t_start:
        return mask

    i_start = int(np.ceil(t_start / dt))
    i_end = min(n_steps, int(np.floor((t_end + 1e-9) / dt)) + 1)
    if i_end <= i_start:
        return mask

    ts = (i_start + np.arange(i_end - i_start)) * dt

    direction = np.array([np.cos(theta), np.sin(theta), 0.0])
    FY_at_release = drone_init + speed * direction * release_time
    bomb_x = FY_at_release[0] + speed * direction[0] * detonation_delay
    bomb_y = FY_at_release[1] + speed * direction[1] * detonation_delay
    bomb_h = FY_at_release[2] - 0.5 * G * detonation_delay ** 2

    smoke_pos = np.empty((len(ts), 3))
    smoke_pos[:, 0] = bomb_x
    smoke_pos[:, 1] = bomb_y
    smoke_pos[:, 2] = bomb_h - SMOKE_SINK_SPEED * (ts - detonation_time)

    M_pos = MISSILES_INIT[missile_idx] + MISSILE_SPEED * MISSILES_DIR[missile_idx] * ts[:, None]

    mask[i_start:i_end] = _check_occlusion_batch(M_pos, smoke_pos, target_keypoints)
    return mask

def simulate_single_bomb(drone_init, theta, speed, release_time, detonation_delay,
                          missile_idx=0, target_keypoints=None, dt=DT, t_total=T_TOTAL):
    """
    仿真单机单弹场景

    Parameters:
        drone_init: 无人机初始位置
        theta: 无人机航向角
        speed: 无人机飞行速度
        release_time: 投放时间
        detonation_delay: 起爆延时
        missile_idx: 目标导弹索引
    """

```

```

Returns:
    total_effective_time: 总有效遮蔽时长
"""
if target_keypoints is None:
    target_keypoints = get_target_keypoints()

mask = _bomb_coverage_mask(drone_init, theta, speed, release_time, detonation_delay,
                           missile_idx, target_keypoints, dt, t_total)

return mask.sum() * dt


def _multi_cloud_coverage_mask(bombs, missile_idx, target_keypoints, dt, t_total):
    """
    多枚烟幕弹的"互补遮蔽"判定：某一时刻只要目标每一个关键点都被"至少一朵"当前
    存活的云团挡住即可判定为有效遮蔽——不要求同一朵云单独挡住全部关键点。

    与逐弹独立判定再取并集（旧逻辑）的区别：旧逻辑要求某一时刻必须有一枚弹自己
    就覆盖了全部目标关键点；这里允许弹A挡住一部分关键点、弹B同时挡住另一部分，
    合起来覆盖全部关键点时也算有效遮蔽，对应"多弹互补遮蔽"的场景。

    bombs: [(drone_init, theta, speed, release_time, detonation_delay), ...],
            全部针对同一枚导弹 missile_idx
    """
    n_steps = int(np.ceil(t_total / dt))
    K = target_keypoints.shape[0]
    result = np.zeros(n_steps, dtype=bool)

    windows = []
    for bomb in bombs:
        release_time, detonation_delay = bomb[3], bomb[4]
        detonation_time = release_time + detonation_delay
        smoke_expire_time = detonation_time + EFFECTIVE_DURATION
        t_start = max(0.0, detonation_time)
        t_end = min(t_total, smoke_expire_time)
        if t_end <= t_start:
            continue
        i_start = int(np.ceil(t_start / dt))
        i_end = min(n_steps, int(np.floor((t_end + 1e-9) / dt)) + 1)
        if i_end <= i_start:
            continue
        windows.append((i_start, i_end, bomb))

    if not windows:
        return result

    lo = min(w[0] for w in windows)
    hi = max(w[1] for w in windows)
    ts_all = (lo + np.arange(hi - lo)) * dt
    M_pos_all = MISSILES_INIT[missile_idx] + MISSILE_SPEED * MISSILES_DIR[missile_idx] * ts_all[:, None]

    inside_cloud_any = np.zeros(hi - lo, dtype=bool)
    keypoint_blocked = np.zeros((hi - lo, K), dtype=bool)
    any_active = np.zeros(hi - lo, dtype=bool)

    for i_start, i_end, (drone_init, theta, speed, release_time, detonation_delay) in windows:
        detonation_time = release_time + detonation_delay
        rel_start, rel_end = i_start - lo, i_end - lo
        ts = ts_all[rel_start:rel_end]

        direction = np.array([np.cos(theta), np.sin(theta), 0.0])
        FY_at_release = drone_init + speed * direction * release_time
        bomb_x = FY_at_release[0] + speed * direction[0] * detonation_delay
        bomb_y = FY_at_release[1] + speed * direction[1] * detonation_delay
        bomb_h = FY_at_release[2] - 0.5 * G * detonation_delay ** 2

```

```

smoke_pos = np.empty((len(ts), 3))
smoke_pos[:, 0] = bomb_x
smoke_pos[:, 1] = bomb_y
smoke_pos[:, 2] = bomb_h - SMOKE_SINK_SPEED * (ts - detonation_time)

m = M_pos_all[rel_start:rel_end]
vec_ms = smoke_pos - m
dist_ms = np.linalg.norm(vec_ms, axis=1)
inside_cloud = dist_ms < EFFECTIVE_RADIUS
inside_cloud_any[rel_start:rel_end] |= inside_cloud

dist_mt = np.linalg.norm(m - TARGET_CENTER, axis=1)
dist_st = np.linalg.norm(smoke_pos - TARGET_CENTER, axis=1)
eligible = dist_mt > dist_st # 这朵云得在导弹和目标之间, 才谈得上挡住哪个关键点

dist_ms_safe = np.maximum(dist_ms, 1e-9)
sin_alpha = np.clip(EFFECTIVE_RADIUS / dist_ms_safe, 0.0, 1.0)
full_cover = sin_alpha >= 1.0
cos_alpha = np.sqrt(np.maximum(1.0 - sin_alpha ** 2, 0.0))

need = eligible & (~full_cover)
in_cone = np.zeros((len(ts), K), dtype=bool)
if np.any(need):
    idx_need = np.nonzero(need)[0]
    m_n_all = m[need]
    vec_ms_n_all = vec_ms[need]
    dist_ms_n_all = dist_ms[need]
    cos_alpha_n_all = cos_alpha[need]

    # tar_axis 是 (t_n, K, 3) 的大数组, t_n 大 + K 大时能到 GB 量级。
    # 和 _check_occlusion_batch 一样按行分块, 把单块临时数组控制在 ~1e7 元素以内,
    # 避免多进程并行时每个 worker 各吃 1GB+ 直接把内存撑爆。
    chunk = max(1, int(1e7 / max(K * 3, 1)))
    for s in range(0, len(idx_need), chunk):
        e = min(s + chunk, len(idx_need))
        m_n = m_n_all[s:e]
        vec_ms_n = vec_ms_n_all[s:e]
        dist_ms_n = dist_ms_n_all[s:e]
        cos_alpha_n = cos_alpha_n_all[s:e]

        tar_axis = target_keypoints[None, :, :] - m_n[:, None, :]
        dot = np.einsum('ti,tki->tk', vec_ms_n, tar_axis)
        norm_tar = np.linalg.norm(tar_axis, axis=2)
        denom = dist_ms_n[:, None] * norm_tar
        with np.errstate(divide='ignore', invalid='ignore'):
            cos_gamma = dot / denom
        # denom=0->cos_gamma=inf/nan; cos_alpha<1->比较恒False, 省去np.where拷贝
        in_cone[idx_need[s:e]] = cos_alpha_n[:, None] <= cos_gamma

in_cone |= (eligible & full_cover)[:, None]

keypoint_blocked[rel_start:rel_end] |= in_cone
any_active[rel_start:rel_end] = True

all_blocked = np.all(keypoint_blocked, axis=1) & any_active
result[lo:hi] = inside_cloud_any | all_blocked
return result

def simulate_multi_bomb_single_drone(drone_init, theta, speed, release_times, detonation_delays,
                                     missile_indices, target_keypoints=None, dt=DT, t_total=T_TOTAL):
    """
    仿真单机多弹场景 (针对指定导弹)

```

```

Parameters:
    drone_init: 无人机初始位置
    theta: 无人机航向角
    speed: 无人机飞行速度
    release_times: 投放时间列表 [n_bombs]
    detonation_delays: 起爆延时列表 [n_bombs]
    missile_indices: 每枚弹对应的目标导弹索引 [n_bombs]

Returns:
    total_effective_time: 总有效遮蔽时长 (同一导弹上的多枚弹按"互补遮蔽"判定后取并集)
    """
    if target_keypoints is None:
        target_keypoints = get_target_keypoints()

    n_steps = int(np.ceil(t_total / dt))
    any_covered = np.zeros(n_steps, dtype=bool)

    missile_indices = np.asarray(missile_indices)
    for k in np.unique(missile_indices):
        bombs = [(drone_init, theta, speed, release_times[i], detonation_delays[i])
                  for i in range(len(release_times)) if missile_indices[i] == k]
        mask = _multi_cloud_coverage_mask(bombs, int(k), target_keypoints, dt, t_total)
        any_covered |= mask

    return any_covered.sum() * dt

def simulate_multi_drone_multi_bomb(drone_params_list, dt=DT, t_total=T_TOTAL,
                                    target_keypoints=None):
    """
    仿真多机多弹多导弹场景

    Parameters:
        drone_params_list: 每架无人机的参数字典列表
        [{
            'drone_init': np.array([x,y,z]),
            'theta': float,
            'speed': float,
            'release_times': np.array([t1, t2, t3]),
            'detonation_delays': np.array([d1, d2, d3]),
            'missile_indices': [k1, k2, k3], # 每枚弹对应的导弹索引
        }, ...]

    Returns:
        total_effective_time: 总有效遮蔽时长 (同一导弹上的所有云团按"互补遮蔽"判定, 再对三枚导弹求和)
        per_missile_time: 每枚导弹的遮蔽时长
    """
    if target_keypoints is None:
        target_keypoints = get_target_keypoints(n_circle=360, n_layers=10)

    n_steps = int(np.ceil(t_total / dt))
    n_missiles = 3
    per_missile_mask = [np.zeros(n_steps, dtype=bool) for _ in range(n_missiles)]

    bombs_by_missile = {0: [], 1: [], 2: []}
    for p in drone_params_list:
        n_bombs = len(p['release_times'])
        for i in range(n_bombs):
            k = int(p['missile_indices'][i])
            bombs_by_missile[k].append((p['drone_init'], p['theta'], p['speed'],
                                         p['release_times'][i], p['detonation_delays'][i]))

    for k, bombs in bombs_by_missile.items():

```

```
    if bombs:
        per_missile_mask[k] = _multi_cloud_coverage_mask(bombs, k, target_keypoints, dt, t_total)

per_missile_time = np.array([m.sum() * dt for m in per_missile_mask])
total_effective_time = per_missile_time.sum()

return total_effective_time, per_missile_time
```

```

"""
粒子群优化算法 (PSO) 实现, 支持多进程并行评估种群
"""

import os
import numpy as np
import multiprocessing as mp
from config import PSO_SWARM_SIZE, PSO_MAX_ITER, PSO_W, PSO_C1, PSO_C2, PSO_MAX_WORKERS

# 子进程worker用的模块级状态: Pool的initializer只在每个worker进程启动时跑一次,
# 把目标函数存到worker自己的全局变量里, 避免每次pool.map都重新pickle一遍
# objective_func(包括它内部可能带的关键点数组等数据)。
_worker_objective = None

# 限制每个worker进程里numpy底层BLAS的线程数为1。否则 N个进程 × 每进程又开M个BLAS线程
# = N×M 个线程争抢CPU(超额订阅), CPU打满但吞吐反而下降。并行靠的是进程级并行,
# 单个评估本身不需要BLAS多线程。
_THREAD_ENV_VARS = ("OMP_NUM_THREADS", "OPENBLAS_NUM_THREADS", "MKL_NUM_THREADS",
                    "NUMEXPR_NUM_THREADS", "VECLIB_MAXIMUM_THREADS")

def _pool_init(objective_func):
    global _worker_objective
    for _v in _THREAD_ENV_VARS:
        os.environ[_v] = "1"
    _worker_objective = objective_func

def _available_memory_gb():
    """尽量拿到当前可用物理内存(GB)。Windows用GlobalMemoryStatusEx, 拿不到就返回None。"""
    try:
        import ctypes

        class _MEMSTAT(ctypes.Structure):
            _fields_ = [("dwLength", ctypes.c_ulong),
                        ("dwMemoryLoad", ctypes.c_ulong),
                        ("ullTotalPhys", ctypes.c_ulonglong),
                        ("ullAvailPhys", ctypes.c_ulonglong),
                        ("ullTotalPageFile", ctypes.c_ulonglong),
                        ("ullAvailPageFile", ctypes.c_ulonglong),
                        ("ullTotalVirtual", ctypes.c_ulonglong),
                        ("ullAvailVirtual", ctypes.c_ulonglong),
                        ("ullAvailExtendedVirtual", ctypes.c_ulonglong)]

        m = _MEMSTAT()
        m.dwLength = ctypes.sizeof(_MEMSTAT)
        if ctypes.windll.kernel32.GlobalMemoryStatusEx(ctypes.byref(m)):
            return m.ullAvailPhys / 1e9
    except Exception:
        pass
    return None

# 单个worker评估烟幕遮蔽时的峰值内存经验值(GB): spawn出的Python+numpy底座约0.15GB,
# 加上分块后的临时数组约0.3~0.4GB, 留些余量按0.7GB估。
_PER_WORKER_GB = 0.7

def _resolve_workers(requested):
    """把 n_workers 请求解析成实际进程数:
    上限 = min(CPU核数, config.PSO_MAX_WORKERS 或环境变量 CUMCM_PSO_WORKERS,
    按当前可用内存能容纳的进程数)。
    可用内存拿不到时退回固定上限。内存波动大(实测10~20GB来回), 所以每次都现算。"""
    cap = PSO_MAX_WORKERS

```

```

env_cap = os.environ.get("CUMCM_PSO_WORKERS")
if env_cap:
    try:
        cap = max(1, int(env_cap))
    except ValueError:
        pass

avail = _available_memory_gb()
if avail is not None:
    # 只用可用内存的一半来开进程，给主进程/其它程序/系统留足余量
    mem_cap = max(1, int((avail * 0.5) / _PER_WORKER_GB))
    cap = min(cap, mem_cap)

if requested == 'auto':
    n = min(os.cpu_count() or 1, cap)
else:
    n = min(int(requested), cap)
return max(1, n)

def _pool_eval(x):
    return _worker_objective(x)

class PSO:
    """粒子群优化器"""

    def __init__(self, objective_func, bounds, n_particles=None, max_iter=None,
                 w=None, c1=None, c2=None, maximize=True, verbose=True, n_workers='auto',
                 seed_positions=None):
        """
        Parameters:
        objective_func: 目标函数 f(x) -> float
            如果要用多进程(n_workers>1), 这个必须是可以被pickle的对象——
            模块级的普通函数、或者一个__init__/__call__都只存简单数据(numpy数组/数字)
            的类实例都可以; 写在另一个函数内部的闭包(def objective(x): ...)不行,
            pickle序列化不了闭包, Windows/macOS的多进程默认用spawn方式启动子进程,
            启动时必须能把目标函数序列化过去。
        bounds: [(low1, high1), (low2, high2), ...] 每个维度的上下界
        n_particles: 粒子数量
        max_iter: 最大迭代次数
        maximize: True 表示最大化目标函数
        n_workers: 并行进程数。'auto'=自动按CPU核心数检测(os.cpu_count());
            传1或者检测失败时自动退回到单进程串行, 不会报错崩溃。
        seed_positions: 可选, [(x1,...,xn), ...] 一组"热启动"初始解(比如用更便宜的方法先粗搜出来的一个不错的候选), 会替换掉初始种群里的前几个
            粒子(不是追加), 让PSO从这些已知不错的点附近开始搜, 而不是纯
            随机初始化。对应图灵论文里"贪心算法找可接受解, PSO在其附近精修"
            的思路。
        """
        self.objective_func = objective_func
        self.bounds = np.array(bounds)
        self.n_dims = len(bounds)
        self.n_particles = n_particles or PSO_SWARM_SIZE
        self.max_iter = max_iter or PSO_MAX_ITER
        self.w = w or PSO_W
        self.c1 = c1 or PSO_C1
        self.c2 = c2 or PSO_C2
        self.maximize = maximize
        self.verbose = verbose
        self.seed_positions = seed_positions

        self.lb = self.bounds[:, 0]

```

```

self.ub = self.bounds[:, 1]

if n_workers == 'auto':
    n_workers = _resolve_workers('auto')
self.n_workers = _resolve_workers(n_workers)

# 状态
self.best_position = None
self.best_value = None
self.history = []

def _evaluate_batch(self, positions, pool):
    """对整批粒子求值，有pool就并行算，没有就返回串行列表推导"""
    if pool is not None:
        raw = np.array(pool.map(_pool_eval, list(positions)))
    else:
        raw = np.array([self.objective_func(p) for p in positions])
    # PSO内部统一按"求最小值"处理，maximize=True时目标值取反
    return -raw if self.maximize else raw

def optimize(self):
    """运行优化，返回 (best_position, best_value)"""
    pool = None
    if self.n_workers > 1:
        try:
            pool = mp.Pool(self.n_workers, initializer=_pool_init,
                           initargs=(self.objective_func,))
            if self.verbose:
                detected = os.cpu_count()
                print(f" PSO 并行评估: 使用 {self.n_workers} 个进程 "
                      f"(检测到 {detected} 个CPU核心)")
        except Exception as e:
            if self.verbose:
                print(f" 多进程启动失败({e}), 回退到单进程串行")
            pool = None

    try:
        # 初始化粒子位置和速度
        positions = np.random.uniform(
            self.lb, self.ub, size=(self.n_particles, self.n_dims)
        )
        if self.seed_positions:
            n_seed = min(len(self.seed_positions), self.n_particles)
            for i in range(n_seed):
                positions[i] = np.clip(self.seed_positions[i], self.lb, self.ub)
            if self.verbose:
                print(f" PSO 热启动: 用 {n_seed} 个预设起点替换初始种群中的对应粒子")
        velocities = np.zeros((self.n_particles, self.n_dims))

        values = self._evaluate_batch(positions, pool)

        # 个体最优
        pbest_positions = positions.copy()
        pbest_values = values.copy()

        # 全局最优 (内部统一按"求最小值"处理)
        gbest_idx = np.argmin(values)
        self.best_position = positions[gbest_idx].copy()
        self.best_value = values[gbest_idx]
        self.history = [self.best_value]

        for iteration in range(self.max_iter):
            # 更新速度和位置
            r1 = np.random.random((self.n_particles, self.n_dims))

```



```

r2 = np.random.random((self.n_particles, self.n_dims))

velocities = (self.w * velocities +
               self.c1 * r1 * (pbest_positions - positions) +
               self.c2 * r2 * (self.best_position - positions))

positions = positions + velocities

# 边界处理 - clamp到边界内
positions = np.clip(positions, self.lb, self.ub)

# 评估
new_values = self._evaluate_batch(positions, pool)

# 更新个体最优
improved = new_values < pbest_values
pbest_positions[improved] = positions[improved].copy()
pbest_values[improved] = new_values[improved]

# 更新全局最优
current_best_idx = np.argmin(new_values)
current_best_value = new_values[current_best_idx]
if current_best_value < self.best_value:
    self.best_value = current_best_value
    self.best_position = positions[current_best_idx].copy()

self.history.append(self.best_value)

if self.verbose and (iteration + 1) % max(1, self.max_iter // 10) == 0:
    display_val = -self.best_value if self.maximize else self.best_value
    print(f" PS0 iter {iteration+1}/{self.max_iter}: best = {display_val:.4f}")
finally:
    if pool is not None:
        pool.close()
        pool.join()

# 最终输出
if self.maximize:
    return self.best_position, -self.best_value
else:
    return self.best_position, self.best_value

```

```
def local_polish(objective_func, x0, bounds, method='Powell', maxiter=200, radius_frac=0.05):
```

```
    """
```

PS0收敛完之后，在最优解附近做一次无梯度局部精修——对应图灵论文/MATLAB代码里

"PS0+禁忌搜索"或者particleswarm的HybridFcn=@fmincon那一层收尾。

我们的目标函数是"遮蔽时长对参数的积分"，不是纯离散的0/1判定，随参数小幅变化时

通常是连续变化的（遮蔽区间的边界会平滑地移动），但仿真本身有离散粒度带来的

噪声，用有限差分梯度法（BFGS一类）容易被这点噪声带偏，所以选无梯度的Powell/

Nelder-Mead，比强求梯度稳一些。

实测过一个教训：如果直接把PS0用的那套完整bounds原样传给Powell，它会在大范围里

大步搜索，从一个4.9s的解直接走到0s（这类占空比很低的目标函数——大部分参数

组合都是0——对无梯度法很不友好，走出好解所在的那个窄"盆地"就再也回不来）。所以

精修阶段强制把搜索范围收紧到x0周围 $\pm \text{radius_frac} * (\text{hi-lo})$ 的一个小邻域内，真正

做"局部"精修，而不是让它在全局范围里重新摸索。

objective_func: 要最大化的目标函数 $f(x)$ （不是PS0内部那种取反后的版本，直接传

原始的、值越大越好的目标函数）

x0: PS0给出的最优解，作为精修起点

bounds: [(low,high), ...]，跟PS0用的应该是同一套完整边界（函数内部会自动收紧）

radius_frac: 精修邻域半径, 占每一维完整区间长度的比例, 默认5%

Returns:

(x_polished, f_polished): 精修后的解和对应目标值。如果精修没有找到更好的解

(可能落回一个稍差的点, 无梯度法不保证单调), 调用方应该自己跟PSO原结果比较,

取较大的那个——这个函数只负责精修, 不负责判断要不要采纳。

"""

```
from scipy.optimize import minimize
```

```
x0 = np.asarray(x0, dtype=float)
```

```
bounds_arr = np.asarray(bounds, dtype=float)
```

```
lo, hi = bounds_arr[:, 0], bounds_arr[:, 1]
```

```
margin = radius_frac * (hi - lo)
```

```
local_bounds = list(zip(np.maximum(lo, x0 - margin), np.minimum(hi, x0 + margin)))
```

```
def neg_obj(x):
```

```
    return -objective_func(np.asarray(x))
```

```
result = minimize(
```

```
    neg_obj, x0, method=method, bounds=local_bounds,
```

```
    options={'maxiter': maxiter, 'xtol': 1e-4, 'ftol': 1e-5}
```

```
)
```

```
return np.asarray(result.x), -result.fun
```

```

"""
问题1：利用无人机FY1投放1枚烟幕干扰弹实施对M1的干扰
给定飞行参数，计算有效遮蔽时长

C题参数：FY1以120m/s朝向假目标( $\theta=\pi$ )飞行，受领任务1.2s后投放，3.2s后起爆
"""

import numpy as np
from config import (
    P1_RELEASE_TIME, P1_DETONATION_DELAY, P1_DRONE_SPEED, P1_DRONE_THETA,
    DT, T_TOTAL, DRONES_INIT,
)
from simulation import simulate_single_bomb

def solve_problem1():
    """求解问题1"""
    print("=" * 60)
    print("问题1：FY1投放1枚烟幕干扰弹对M1的有效遮蔽时长")
    print("=" * 60)

    drone_init = DRONES_INIT[0] # FY1
    theta = P1_DRONE_THETA #  $\pi$  (朝向假目标)
    speed = P1_DRONE_SPEED # 120 m/s
    release_time = P1_RELEASE_TIME # 1.2 s
    detonation_delay = P1_DETONATION_DELAY # 3.2 s

    print(f"\n输入参数:")
    print(f" FY1初始位置: ({drone_init[0]}, {drone_init[1]}, {drone_init[2]})")
    print(f" 航向角 $\theta$ : {theta:.4f} rad ({np.degrees(theta):.2f}°)")
    print(f" 飞行速度: {speed} m/s")
    print(f" 投放时间(受领任务后): {release_time} s")
    print(f" 起爆延时(投放后): {detonation_delay} s")

    # 使用更精细的时间步长
    dt_fine = 0.0001 # 精细步长
    effective_time = simulate_single_bomb(
        drone_init, theta, speed, release_time, detonation_delay,
        missile_idx=0, dt=dt_fine, t_total=T_TOTAL
    )

    print(f"\n结果:")
    print(f" 烟幕干扰弹对M1的有效遮蔽时长: {effective_time:.4f} s")

    return effective_time

if __name__ == "__main__":
    solve_problem1()
"""
问题2：利用无人机FY1投放1枚烟幕干扰弹实施对M1的干扰
确定FY1的飞行方向、速度、投放点、起爆点，使遮蔽时间尽可能长

优化变量：[theta, speed, release_time, detonation_delay]
"""

import numpy as np
from config import (
    DRONES_INIT, DRONE_SPEED_MIN, DRONE_SPEED_MAX, DT, T_TOTAL,
    PSO_SWARM_SIZE, PSO_MAX_ITER,
    SEARCH_N_CIRCLE, SEARCH_N_LAYERS, SEARCH_DT,
    FINAL_N_CIRCLE, FINAL_N_LAYERS, FINAL_DT,
)
from simulation import simulate_single_bomb, get_target_keypoints
from pso import PSO, local_polish

```

```

class Problem2Objective:
    """模块级可pickle的目标函数对象, 供PSO多进程worker调用"""

    def __init__(self, drone_init, target_keypoints, dt=SEARCH_DT):
        self.drone_init = drone_init
        self.target_keypoints = target_keypoints
        self.dt = dt

    def __call__(self, x):
        theta, speed, release_time, detonation_delay = x
        return simulate_single_bomb(
            self.drone_init, theta, speed, release_time, detonation_delay,
            missile_idx=0, target_keypoints=self.target_keypoints, dt=self.dt, t_total=T_TOTAL
        )

def solve_problem2():
    """求解问题2: 单机单弹最优策略"""
    print("=" * 60)
    print("问题2: FY1单机单弹最优投放策略 (PSO优化)")
    print("=" * 60)

    drone_init = DRONES_INIT[0] # FY1
    # 搜索关键点(跑PSO用, 快); 定稿稿见下方最优解复算
    target_keypoints = get_target_keypoints(SEARCH_N_CIRCLE, SEARCH_N_LAYERS)

    # 决策变量: [theta, speed, release_time, detonation_delay]
    # theta以FY1指向真目标(0,200,0)的方位角(约179.4°/3.13rad)为中心留出搜索余量,
    # 原范围(45°~90°)方向朝向战场正前方而非目标, 导致PSO永远搜不到任何有效遮蔽
    bounds = [
        (2.73, 3.53), # theta (rad) - 朝向真目标方向附近
        (DRONE_SPEED_MIN, DRONE_SPEED_MAX), # speed (m/s)
        (0.0, 15.0), # release_time (s)
        (0.0, 6.0), # detonation_delay (s)
    ]

    objective = Problem2Objective(drone_init, target_keypoints, dt=SEARCH_DT)

    print(f"\n变量范围:")
    print(f" theta: [{bounds[0][0]:.2f}, {bounds[0][1]:.2f}] rad")
    print(f" speed: [{bounds[1][0]}, {bounds[1][1]}] m/s")
    print(f" release_time: [{bounds[2][0]}, {bounds[2][1]}] s")
    print(f" detonation_delay: [{bounds[3][0]}, {bounds[3][1]}] s")
    print(f"\n粒子群规模: {PSO_SWARM_SIZE}, 迭代次数: {PSO_MAX_ITER}")

    # PSO 优化
    pso = PSO(objective, bounds, n_particles=200, max_iter=100, maximize=True, verbose=True)
    x_opt, f_opt = pso.optimize()

    print("\n局部精修(Powell)...")
    x_polished, f_polished = local_polish(objective, x_opt, bounds)
    if f_polished > f_opt:
        print(f" 精修有提升: {f_opt:.4f}s -> {f_polished:.4f}s")
        x_opt, f_opt = x_polished, f_polished
    else:
        print(f" 精修没有提升({f_polished:.4f}s <= {f_opt:.4f}s), 保留PSO原结果")

    # 定稿稿复算: 搜索用的是180关键点/dt0.01的粗档, 最优解定下来后用360关键点/dt0.005
    # 的精档复算一遍, 作为最终上报数值
    final_kp = get_target_keypoints(FINAL_N_CIRCLE, FINAL_N_LAYERS)
    f_final = simulate_single_bomb(drone_init, x_opt[0], x_opt[1], x_opt[2], x_opt[3],
                                   missile_idx=0, target_keypoints=final_kp,

```

```

        dt=FINAL_DT, t_total=T_TOTAL)

print(f" 定稿档复算(360关键点/dt{FINAL_DT}): {f_opt:.4f}s(搜索档) -> {f_final:.4f}s(定稿)")
f_opt = f_final

print(f"\n优化结果:")
print(f" 航向角θ: {x_opt[0]:.4f} rad ({np.degrees(x_opt[0]):.2f}°)")
print(f" 飞行速度: {x_opt[1]:.2f} m/s")
print(f" 投放时间: {x_opt[2]:.4f} s")
print(f" 起爆延时: {x_opt[3]:.4f} s")

# 计算投放点和起爆点坐标
direction = np.array([np.cos(x_opt[0]), np.sin(x_opt[0]), 0.0])
release_pos = drone_init + x_opt[1] * direction * x_opt[2]
detonation_pos = release_pos + x_opt[1] * direction * x_opt[3]
detonation_pos[2] -= 0.5 * 9.8 * x_opt[3] ** 2

print(f" 投放点坐标: ({release_pos[0]:.2f}, {release_pos[1]:.2f}, {release_pos[2]:.2f})")
print(f" 起爆点坐标: ({detonation_pos[0]:.2f}, {detonation_pos[1]:.2f}, {detonation_pos[2]:.2f})")
print(f" 最长有效遮蔽时长: {f_opt:.4f} s")

return x_opt, f_opt

if __name__ == "__main__":
    solve_problem2()

```

```

"""
问题3：利用无人机FY1投放3枚烟幕干扰弹实施对M1的干扰
输出结果到 result1.xlsx

优化变量(8维): [theta, speed, release1, interval2, interval3, delay1, delay2, delay3]
实际投放时间: [release1, release1+interval2, release1+interval2+interval3]
"""

import numpy as np
import openpyxl
from config import (
    DRONES_INIT, DRONE_SPEED_MIN, DRONE_SPEED_MAX,
    BOMB_INTERVAL_MIN, DT, T_TOTAL,
    SEARCH_N_CIRCLE, SEARCH_N_LAYERS, SEARCH_DT,
    FINAL_N_CIRCLE, FINAL_N_LAYERS, FINAL_DT,
)
from simulation import simulate_multi_bomb_single_drone, get_target_keypoints
from pso import PSO, local_polish
from problem2 import Problem2Objective

# 为 problem3 使用独立的 PSO 参数
# (预算已按搜索档实测吞吐重定: 搜索档单次评估约0.02s/10进程, 150x100约5~6分钟)
PSO_SWARM_SIZE_P3 = 150
PSO_MAX_ITER_P3 = 100

# 阶段0(热启动预搜)的PSO参数, 问题规模小(单弹4维), 预算给小一些即可
PSO_SWARM_SIZE_P3_SEED = 100
PSO_MAX_ITER_P3_SEED = 40

class Problem3Objective:
    """模块级可pickle的目标函数对象, 供PSO多进程worker调用"""

    def __init__(self, drone_init, target_keypoints, dt=SEARCH_DT):
        self.drone_init = drone_init
        self.target_keypoints = target_keypoints
        self.dt = dt

    def __call__(self, x):
        theta = x[0]
        speed = x[1]
        release_times = np.array([
            x[2],
            x[2] + x[3],
            x[2] + x[3] + x[4],
        ])
        detonation_delays = np.array([x[5], x[6], x[7]])
        missile_indices = np.array([0, 0, 0]) # 全部针对M1

        return simulate_multi_bomb_single_drone(
            self.drone_init, theta, speed, release_times, detonation_delays,
            missile_indices, target_keypoints=self.target_keypoints, dt=self.dt, t_total=T_TOTAL
        )

def solve_problem3():
    """求解问题3: 单机三弹最优策略"""
    print("=" * 60)
    print("问题3: FY1投放3枚烟幕干扰弹对M1的最优策略 (PSO优化)")
    print("=" * 60)

    drone_init = DRONES_INIT[0] # FY1
    target_keypoints = get_target_keypoints(SEARCH_N_CIRCLE, SEARCH_N_LAYERS) # 搜索档

    # 决策变量(8维):

```

```

# x[0] = theta, x[1] = speed
# x[2] = release1, x[3] = interval_to_2nd, x[4] = interval_to_3rd
# x[5] = delay1, x[6] = delay2, x[7] = delay3
bounds = [
    (2.73, 3.53),      # theta (rad) - 朝向真目标方向附近(原范围方向错误, 见problem2.py)
    (DRONE_SPEED_MIN, DRONE_SPEED_MAX), # speed
    (0.0, 5.0),        # release1
    (BOMB_INTERVAL_MIN, 4.0),      # interval 1->2
    (BOMB_INTERVAL_MIN, 4.0),      # interval 2->3
    (0.0, 6.0),        # delay1
    (0.0, 6.0),        # delay2
    (0.0, 6.0),        # delay3
]

# =====
# 阶段0: 用问题2同款的单弹目标函数快速预搜一个"够用"的起点,
# 给下面8维PSO当热启动种子——对应国赛论文里"贪心算法找可接受解,
# PSO在其附近精修"的思路, 而不是让PSO从纯随机初始化的8维空间里摸索。
# =====
print("\n阶段0: 单弹快速预搜(为8维PSO提供热启动起点)...")
seed_kp = get_target_keypoints(SEARCH_N_CIRCLE, 0) # 单弹预搜用无侧面点的搜索档, 更快
seed_obj = Problem20Objective(drone_init, seed_kp, dt=SEARCH_DT)
seed_bounds = [bounds[0], bounds[1], (0.0, 5.0), (0.0, 6.0)]
seed_pso = PSO(seed_obj, seed_bounds, n_particles=PSO_SWARM_SIZE_P3_SEED,
               max_iter=PSO_MAX_ITER_P3_SEED, maximize=True, verbose=False)
seed_x, seed_f = seed_pso.optimize()
print(f"  预搜起点:  $\theta={np.degrees(seed\_x[0]):.1f}^\circ$   $v={seed\_x[1]:.1f}m/s$  "
      f"release={seed_x[2]:.2f}s delay={seed_x[3]:.2f}s (单弹遮蔽{seed_f:.4f}s)")

# 用这个单弹解构造8维种子: 三发弹依次按最小间隔错开投放, 起爆延时先沿用预搜结果,
# 后续PSO会在这个起点附近继续搜索(种子只替换初始种群里的一个粒子, 不锁死解)
seed_moderate_delay = [seed_x[0], seed_x[1], seed_x[2], BOMB_INTERVAL_MIN, BOMB_INTERVAL_MIN,
                      seed_x[3], seed_x[3], seed_x[3]]

# 第二个种子: "贴近速度上限+第一发几乎零延时"这一互补策略分支。
# 贴近速度上限能更快将云团送到视线交点, 首弹近零延时则争取在导弹-真目标
# 距离最近前即开始遮蔽。两个种子从不同方向覆盖可行域。
speed_hi = DRONE_SPEED_MAX - 0.1 * (DRONE_SPEED_MAX - DRONE_SPEED_MIN)
seed_high_speed_low_delay = [
    seed_x[0],      # 复用同一个热启动搜到的方向
    speed_hi,      # 速度贴近上限
    0.1,           # release1: 尽早投放
    BOMB_INTERVAL_MIN, # interval2: 最小间隔
    BOMB_INTERVAL_MIN, # interval3: 最小间隔
    0.1,           # delay1: 几乎零延时
    3.0,           # delay2
    3.0,           # delay3
]

seed_positions = [seed_moderate_delay, seed_high_speed_low_delay]

objective = Problem30Objective(drone_init, target_keypoints)

print(f"\n变量维度: 8 (theta, speed, 3×release, 3×delay)")
print(f"粒子群规模: {PSO_SWARM_SIZE_P3}, 迭代次数: {PSO_MAX_ITER_P3}")

pso = PSO(objective, bounds, n_particles=PSO_SWARM_SIZE_P3,
          max_iter=PSO_MAX_ITER_P3, maximize=True, verbose=True,
          seed_positions=seed_positions)
x_opt, f_opt = pso.optimize()

# PSO收敛完之后做一次局部精修(对应论文里PSO+TS、MATLAB fmincon hybrid那一层收尾),
# 精修不保证一定更好(无梯度法不单调), 跟PSO原结果比较取较大的那个

```

```

print("\n局部精修(Powell)...")
x_polished, f_polished = local_polish(objective, x_opt, bounds)
if f_polished > f_opt:
    print(f"    精修有提升: {f_opt:.4f}s -> {f_polished:.4f}s")
    x_opt, f_opt = x_polished, f_polished
else:
    print(f"    精修没有提升({f_polished:.4f}s <= {f_opt:.4f}s), 保留PSO原结果")

# 定稿档复算: 搜索用180关键点/dt0.01, 最优解定下来后用360关键点/dt0.005复算上报
theta_f, speed_f = x_opt[0], x_opt[1]
rt_f = np.array([x_opt[2], x_opt[2] + x_opt[3], x_opt[2] + x_opt[3] + x_opt[4]])
dd_f = np.array([x_opt[5], x_opt[6], x_opt[7]])
final_kp = get_target_keypoints(FINAL_N_CIRCLE, FINAL_N_LAYERS)
f_final = simulate_multi_bomb_single_drone(
    drone_init, theta_f, speed_f, rt_f, dd_f, np.array([0, 0, 0]),
    target_keypoints=final_kp, dt=FINAL_DT, t_total=T_TOTAL)
print(f"    定稿档复算(360关键点/dt{FINAL_DT}): {f_opt:.4f}s(搜索档) -> {f_final:.4f}s(定稿)")
f_opt = f_final

# 解析结果
theta = x_opt[0]
speed = x_opt[1]
release_times = np.array([x_opt[2], x_opt[2] + x_opt[3], x_opt[2] + x_opt[3] + x_opt[4]])
detonation_delays = np.array([x_opt[5], x_opt[6], x_opt[7]])

direction = np.array([np.cos(theta), np.sin(theta), 0.0])

print(f"\n优化结果:")
print(f"    航向角θ: {theta:.4f} rad ({np.degrees(theta):.2f}°)")
print(f"    飞行速度: {speed:.2f} m/s")

for i in range(3):
    release_pos = drone_init + speed * direction * release_times[i]
    detonation_pos = release_pos + speed * direction * detonation_delays[i]
    detonation_pos[2] -= 0.5 * 9.8 * detonation_delays[i] ** 2

    print(f"\n    烟雾弹{i+1}:")
    print(f"        投放时间: {release_times[i]:.4f} s")
    print(f"        起爆延时: {detonation_delays[i]:.4f} s")
    print(f"        投放点: ({release_pos[0]:.2f}, {release_pos[1]:.2f}, {release_pos[2]:.2f})")
    print(f"        起爆点: ({detonation_pos[0]:.2f}, {detonation_pos[1]:.2f}, {detonation_pos[2]:.2f})")

print(f"\n    总有效遮蔽时长: {f_opt:.4f} s")

# 保存到 result1.xlsx
save_result1(theta, speed, release_times, detonation_delays, drone_init,
              direction, f_opt)

return x_opt, f_opt

def save_result1(theta, speed, release_times, detonation_delays,
                 drone_init, direction, total_time):
    """保存问题3结果到 result1.xlsx"""
    wb = openpyxl.Workbook()
    ws = wb.active
    ws.title = "问题3结果"

    # 表头
    headers = ["无人机", "航向角(rad)", "航向角(°)", "飞行速度(m/s)",
               "烟雾弹编号", "投放时间(s)", "起爆延时(s)",
               "投放点X", "投放点Y", "投放点Z",
               "起爆点X", "起爆点Y", "起爆点Z"]

```



```

        "总有效遮蔽时长(s)"]
for col, header in enumerate(headers, 1):
    ws.cell(row=1, column=col, value=header)

# 数据行
for i in range(3):
    row = i + 2
    release_pos = drone_init + speed * direction * release_times[i]
    detonation_pos = release_pos + speed * direction * detonation_delays[i]
    detonation_pos[2] -= 0.5 * 9.8 * detonation_delays[i] ** 2

    ws.cell(row=row, column=1, value="FY1")
    ws.cell(row=row, column=2, value=round(theta, 6))
    ws.cell(row=row, column=3, value=round(np.degrees(theta), 4))
    ws.cell(row=row, column=4, value=round(speed, 2))
    ws.cell(row=row, column=5, value=i + 1)
    ws.cell(row=row, column=6, value=round(release_times[i], 4))
    ws.cell(row=row, column=7, value=round(detonation_delays[i], 4))
    ws.cell(row=row, column=8, value=round(release_pos[0], 2))
    ws.cell(row=row, column=9, value=round(release_pos[1], 2))
    ws.cell(row=row, column=10, value=round(release_pos[2], 2))
    ws.cell(row=row, column=11, value=round(detonation_pos[0], 2))
    ws.cell(row=row, column=12, value=round(detonation_pos[1], 2))
    ws.cell(row=row, column=13, value=round(detonation_pos[2], 2))
    if i == 0:
        ws.cell(row=row, column=14, value=round(total_time, 4))

filepath = "result1.xlsx"
wb.save(filepath)
print(f"\n结果已保存至: {filepath}")

if __name__ == "__main__":
    solve_problem3()

```

```

"""

问题4：利用FY1、FY2、FY3各投放1枚烟雾干扰弹，实施对M1的干扰
输出结果到 result2.xlsx

优化变量(12维)：
[theta1, theta2, theta3, speed1, speed2, speed3,
 release1, release2, release3, delay1, delay2, delay3]

求解策略采用两阶段策略：
一次性对12维做冷启动全空间PSO代价太大(实测单次评估约0.9s, 500粒子x300代要接近
40小时)，容易陷入局部最优。改成两阶段：
    阶段1：把"3架无人机各投1弹"拆解成3个独立的"1架无人机投1弹"问题分别求解，
            每架无人机只优化自己对M1的遮蔽时长，互不知道彼此在干什么，快速拿到
            一个"够用"的基线解。
    阶段2：以阶段1每架无人机各自的最优参数为中心，收缩12维的搜索边界，在这个
            小得多的范围内做真正的12维联合PSO精修——目标函数是三机协同后的并集
            遮蔽时长(会考虑互补/错峰效应)，不是阶段1那种各自为战的目标。
这样搜索空间大幅收窄，评估次数也能相应减少，同时不放弃"用真实协同目标函数做
最后把关"这一步。
"""

import numpy as np
import openpyxl
from config import (
    DRONES_INIT, DRONE_SPEED_MIN, DRONE_SPEED_MAX, DT, T_TOTAL,
    SEARCH_N_CIRCLE, SEARCH_N_LAYERS, SEARCH_DT,
    FINAL_N_CIRCLE, FINAL_N_LAYERS, FINAL_DT,
)
from simulation import simulate_multi_drone_multi_bomb, get_target_keypoints
from pso import PSO, local_polish
from final_solve import search_best_detonation

# 搜索档/定稿档关键点(模块级预生成，可被objective对象引用并pickle给worker)
_SEARCH_KP = get_target_keypoints(SEARCH_N_CIRCLE, SEARCH_N_LAYERS)
_FINAL_KP = get_target_keypoints(FINAL_N_CIRCLE, FINAL_N_LAYERS)

# 阶段1兜底用：如果三维空间采样也没找到可行解，才退回猜角度窗口的PSO再试一次
PSO_SWARM_P4_STAGE1_FALLBACK = 150
PSO_ITER_P4_STAGE1_FALLBACK = 60
STAGE1_N_FAST = 8000 # search_best_detonation 阶段1的采样点数(原20000, 按时间预算下调)

# 阶段2：12维联合精修的PSO参数(搜索范围已经收窄，配合搜索档，约8~11分钟)
PSO_SWARM_P4_STAGE2 = 120
PSO_ITER_P4_STAGE2 = 70

# 各无人机朝向真目标(0,200,0)的方位角窗口——只用于阶段1兜底PSO的搜索范围，
# 正常路径下阶段1走search_best_detonation(三维空间采样反解方向)，不依赖这个猜测窗口。
# 之前的教训：FY2/FY3若直接靠这个"指向目标"猜出来的窗口搜索，会把真正的最优方向
# 排除在外(实测跟国防论文报告的FY2/FY3最优方向对不上)，导致阶段1两架都搜出0。
THETA_WINDOWS = [
    (2.73, 3.53), # FY1, 目标方位约179.4°
    (-3.44, -2.64), # FY2, 目标方位约-174.3°
    (2.25, 3.05), # FY3, 目标方位约151.9°
]

N_DRONES = 3

class Problem4Objective:
    """阶段2用：12维联合目标函数(三机协同、取并集)，模块级可pickle"""

    def __init__(self, n_drones):
        self.n_drones = n_drones

```

```

def __call__(self, x):
    theta = x[0:3]
    speed = x[3:6]
    release_times = x[6:9]
    delays = x[9:12]

    drone_params = []
    for i in range(self.n_drones):
        drone_params.append({
            'drone_init': DRONES_INIT[i],
            'theta': theta[i],
            'speed': speed[i],
            'release_times': np.array([release_times[i]]),
            'detonation_delays': np.array([delays[i]]),
            'missile_indices': [0], # 全部针对M1
        })

    total_time, per_missile = simulate_multi_drone_multi_bomb(
        drone_params, dt=SEARCH_DT, t_total=T_TOTAL, target_keypoints=_SEARCH_KP
    )
    return total_time

class Stage1DroneObjective:
    """阶段1用：单架无人机独自对M1的遮蔽时长，模块级可pickle"""

    def __init__(self, drone_idx):
        self.drone_idx = drone_idx

    def __call__(self, x):
        theta, speed, release_time, delay = x
        drone_params = [{
            'drone_init': DRONES_INIT[self.drone_idx],
            'theta': theta,
            'speed': speed,
            'release_times': np.array([release_time]),
            'detonation_delays': np.array([delay]),
            'missile_indices': [0],
        }]
        total_time, _ = simulate_multi_drone_multi_bomb(
            drone_params, dt=SEARCH_DT, t_total=T_TOTAL, target_keypoints=_SEARCH_KP
        )
        return total_time

def _narrow_bounds(center, full_lo, full_hi, margin):
    """以center为中心收缩出一个不超过[full_lo,full_hi]的小区间"""
    lo = max(full_lo, center - margin)
    hi = min(full_hi, center + margin)
    if hi <= lo: # 极端情况下退化保护，保证区间非空
        hi = lo + 1e-6
    return lo, hi

def solve_problem4():
    """求解问题4：三机各一弹对M1（阶段1独立拆解 + 阶段2边界收缩联合精修）"""
    print("=" * 60)
    print("问题4: FY1/FY2/FY3各投放1枚烟幕弹对M1（两阶段PSO优化）")
    print("=" * 60)

    drone_names = ['FY1', 'FY2', 'FY3']

    # =====
    # 阶段1：逐架无人机独立优化（拆解成3个"1机1弹"问题）

```

```

# 用search_best_detonation在无人机周围的三维空间里采样候选起爆点、反解方向,
# 不预先猜角度窗口——避免猜错窗口把真正的最优方向排除在外(FY2/FY3就吃过这个亏)。
# =====
print("\n阶段1: 逐架无人机独立优化(三维空间采样, 不预设角度窗口)...")

stage1_results = []
for i in range(N_DRONES):
    params, f_i = search_best_detonation(DRONES_INIT[i], missile_idx=0, n_fast=STAGE1_N_FAST)

    if params is None:
        # 兜底: 三维空间采样也没找到可行解, 退回猜角度窗口的PSO再试一次
        print(f" {drone_names[i]}: 三维采样未找到可行解, 退回窗口PSO兜底...")
        bounds_i = [THETA_WINDOWS[i], (DRONE_SPEED_MIN, DRONE_SPEED_MAX), (0.0, 20.0), (0.0, 20.0)]
        obj_i = Stage1DroneObjective(i)
        pso_i = PSO(obj_i, bounds_i, n_particles=PSO_SWARM_P4_STAGE1_FALLBACK,
                    max_iter=PSO_ITER_P4_STAGE1_FALLBACK, maximize=True, verbose=False)
        x_i, f_i = pso_i.optimize()
        theta_i, speed_i, rt_i, delay_i = x_i[0], x_i[1], x_i[2], x_i[3]
    else:
        theta_i = params['theta']
        speed_i = params['speed']
        rt_i = params['release_time']
        delay_i = params['delay']

    stage1_results.append({'theta': theta_i, 'speed': speed_i,
                          'release_time': rt_i, 'delay': delay_i, 'time': f_i})
    print(f" {drone_names[i]}:  $\theta$ ={np.degrees(theta_i):.1f}° v={speed_i:.1f}m/s "
          f"独自遮藏={f_i:.4f}s")

total_stage1 = sum(r['time'] for r in stage1_results)
print(f"\n阶段1 单机各自最优简单求和(仅作参考基线, 不是最终答案): {total_stage1:.4f} s")

# =====
# 阶段2: 以阶段1结果为中心, 收缩边界后做12维联合精修
# =====
print("\n阶段2: 12维联合精修(边界已收缩到阶段1解附近)...")

bounds = []
theta_margin = 0.45 # rad, 比之前略放宽, 给阶段2多一点纠偏空间
speed_margin = (DRONE_SPEED_MAX - DRONE_SPEED_MIN) * 0.3
time_margin = 3.0 # s, release/delay 的搜索余量

# 注意: 这里clamp用的是完整角度范围(-pi,pi], 不是THETA_WINDOWS——
# 阶段1的theta现在来自三维空间反解, 可能落在THETA_WINDOWS之外(FY2/FY3就是这样),
# 再用旧窗口去clamp会重新把刚找到的正确方向卡掉, 等于白修。
for i in range(N_DRONES):
    bounds.append(_narrow_bounds(stage1_results[i]['theta'], -np.pi, np.pi, theta_margin))
for i in range(N_DRONES):
    bounds.append(_narrow_bounds(stage1_results[i]['speed'], DRONE_SPEED_MIN, DRONE_SPEED_MAX, speed_margin))
for i in range(N_DRONES):
    bounds.append(_narrow_bounds(stage1_results[i]['release_time'], 0.0, 20.0, time_margin))
for i in range(N_DRONES):
    bounds.append(_narrow_bounds(stage1_results[i]['delay'], 0.0, 20.0, time_margin))

objective = Problem40Objective(N_DRONES)

print(f"\n变量维度: 12 (3×theta, 3×speed, 3×release, 3×delay)")
print(f"粒子群规模: {PSO_SWARM_P4_STAGE2}, 迭代次数: {PSO_ITER_P4_STAGE2}")

pso = PSO(objective, bounds, n_particles=PSO_SWARM_P4_STAGE2,
           max_iter=PSO_ITER_P4_STAGE2, maximize=True, verbose=True)
x_opt, f_opt = pso.optimize()

print("\n局部精修(Powell)...")

```

```

x_polished, f_polished = local_polish(objective, x_opt, bounds)
if f_polished > f_opt:
    print(f"    精修有提升: {f_opt:.4f}s -> {f_polished:.4f}s")
    x_opt, f_opt = x_polished, f_polished
else:
    print(f"    精修没有提升({f_polished:.4f}s <= {f_opt:.4f}s), 保留PSO原结果")

# 定稿档复算: 搜索用180关键点/dt0.01, 最优解用360关键点/dt0.005复算上报
def _eval_final(x):
    dp = []
    for i in range(N_DRONES):
        dp.append({
            'drone_init': DRONES_INIT[i], 'theta': x[i], 'speed': x[3 + i],
            'release_times': np.array([x[6 + i]]),
            'detonation_delays': np.array([x[9 + i]]),
            'missile_indices': [0],
        })
    tt, _ = simulate_multi_drone_multi_bomb(dp, dt=FINAL_DT, t_total=T_TOTAL,
                                             target_keypoints=_FINAL_KP)
    return tt
f_final = _eval_final(x_opt)
print(f"    定稿档复算(360关键点/dt{FINAL_DT}): {f_opt:.4f}s(搜索档) -> {f_final:.4f}s(定稿)")
f_opt = f_final

# 解析结果
theta = x_opt[0:3]
speed = x_opt[3:6]
release_times = x_opt[6:9]
delays = x_opt[9:12]

print(f"\n优化结果:")
for i in range(N_DRONES):
    direction = np.array([np.cos(theta[i]), np.sin(theta[i]), 0.0])
    release_pos = DRONES_INIT[i] + speed[i] * direction * release_times[i]
    detonation_pos = release_pos + speed[i] * direction * delays[i]
    detonation_pos[2] -= 0.5 * 9.8 * delays[i] ** 2

    print(f"\n {drone_names[i]}:")
    print(f"    航向角0: {theta[i]:.4f} rad ({np.degrees(theta[i]):.2f}°)")
    print(f"    飞行速度: {speed[i]:.2f} m/s")
    print(f"    投放时间: {release_times[i]:.4f} s")
    print(f"    起爆延时: {delays[i]:.4f} s")
    print(f"    投放点: ({release_pos[0]:.2f}, {release_pos[1]:.2f}, {release_pos[2]:.2f})")
    print(f"    起爆点: ({detonation_pos[0]:.2f}, {detonation_pos[1]:.2f}, {detonation_pos[2]:.2f})")

print(f"\n 阶段1基线(各自为战简单求和): {total_stagel:.4f} s")
print(f"    阶段2联合精修总有效遮蔽时长: {f_opt:.4f} s")

# 保存到 result2.xlsx
save_result2(theta, speed, release_times, delays, f_opt)

return x_opt, f_opt

def save_result2(theta, speed, release_times, delays, total_time):
    """保存问题4结果到 result2.xlsx"""
    wb = openpyxl.Workbook()
    ws = wb.active
    ws.title = "问题4结果"

    headers = ["无人机", "航向角(rad)", "航向角(°)", "飞行速度(m/s)",
               "投放时间(s)", "起爆延时(s)",
               "投放点X", "投放点Y", "投放点Z",
    ]

```

```

        "起爆点X", "起爆点Y", "起爆点Z",
        "总有效遮蔽时长(s)"]
for col, header in enumerate(headers, 1):
    ws.cell(row=1, column=col, value=header)

drone_names = ['FY1', 'FY2', 'FY3']
for i in range(3):
    row = i + 2
    direction = np.array([np.cos(theta[i]), np.sin(theta[i]), 0.0])
    release_pos = DRONES_INIT[i] + speed[i] * direction * release_times[i]
    detonation_pos = release_pos + speed[i] * direction * delays[i]
    detonation_pos[2] -= 0.5 * 9.8 * delays[i] ** 2

    ws.cell(row=row, column=1, value=drone_names[i])
    ws.cell(row=row, column=2, value=round(theta[i], 6))
    ws.cell(row=row, column=3, value=round(np.degrees(theta[i]), 4))
    ws.cell(row=row, column=4, value=round(speed[i], 2))
    ws.cell(row=row, column=5, value=round(release_times[i], 4))
    ws.cell(row=row, column=6, value=round(delays[i], 4))
    ws.cell(row=row, column=7, value=round(release_pos[0], 2))
    ws.cell(row=row, column=8, value=round(release_pos[1], 2))
    ws.cell(row=row, column=9, value=round(release_pos[2], 2))
    ws.cell(row=row, column=10, value=round(detonation_pos[0], 2))
    ws.cell(row=row, column=11, value=round(detonation_pos[1], 2))
    ws.cell(row=row, column=12, value=round(detonation_pos[2], 2))
    if i == 0:
        ws.cell(row=row, column=13, value=round(total_time, 4))

filepath = "result2.xlsx"
wb.save(filepath)
print(f"\n结果已保存至: {filepath}")

if __name__ == "__main__":
    solve_problem4()

```

```

"""

问题5：5架无人机，每架至多投放3枚烟雾干扰弹，实施对M1、M2、M3的干扰
输出结果到 result3.xlsx

优化变量(40维)：
- theta(5)：各无人机航向角
- speed(5)：各无人机飞行速度
- release_times(5×3=15)：各弹投放时间(编码为间隔)
- detonation_delays(5×3=15)：各弹起爆延时

采用分步优化：先单机优化，再联合微调
"""

import numpy as np
import openpyxl
from config import (
    DRONES_INIT, DRONE_SPEED_MIN, DRONE_SPEED_MAX,
    BOMB_INTERVAL_MIN, INTERCEPT_ORDER, DT, T_TOTAL,
    SEARCH_N_CIRCLE, SEARCH_N_LAYERS, SEARCH_DT,
    FINAL_N_CIRCLE, FINAL_N_LAYERS, FINAL_DT,
)
from simulation import simulate_multi_drone_multi_bomb, get_target_keypoints
from pso import PSO, local_polish
from final_solve import search_best_detonation

# 阶段1三维采样的采样点数(与P4一致的量级)
STAGE1_N_FAST_P5 = 8000

# 预算已按搜索档实测吞吐重定：40维联合评估约0.2s/10进程。
# 阶段2 = 80×50约14分钟；阶段1每机 60×25。整个问题5约24分钟。
PSO_SWARM_P5 = 80
PSO_ITER_P5 = 50

N_DRONES = 5
N_BOMBS_PER_DRONE = 3
N_MISSILES = 3

# 搜索档/定稿档关键点(模块级预生成，供objective引用)
_SEARCH_KP = get_target_keypoints(SEARCH_N_CIRCLE, SEARCH_N_LAYERS)
_FINAL_KP = get_target_keypoints(FINAL_N_CIRCLE, FINAL_N_LAYERS)

class SingleDroneObjective:
    """阶段1：单机独立优化用的目标函数对象，模块级可pickle，供PSO多进程worker调用"""

    def __init__(self, drone_idx, missile_order):
        self.drone_idx = drone_idx
        self.missile_order = missile_order

    def __call__(self, x):
        theta, speed = x[0], x[1]
        release_times = np.array([x[2], x[2] + x[3], x[2] + x[3] + x[4]])
        delays = np.array([x[5], x[6], x[7]])

        drone_params = [{
            'drone_init': DRONES_INIT[self.drone_idx],
            'theta': theta,
            'speed': speed,
            'release_times': release_times,
            'detonation_delays': delays,
            'missile_indices': self.missile_order,
        }]
        total_time, _ = simulate_multi_drone_multi_bomb(
            drone_params, dt=SEARCH_DT, t_total=T_TOTAL, target_keypoints=_SEARCH_KP
        )

```

```

        return total_time

def _joint_objective(x):
    """阶段2: 40维联合微调用的目标函数, 不捕获任何局部变量, 模块级可pickle"""
    idx = 0
    drone_params = []
    for drone_idx in range(N_DRONES):
        theta = x[idx]; idx += 1
        speed = x[idx]; idx += 1

        release1 = x[idx]; idx += 1
        int2 = x[idx]; idx += 1
        int3 = x[idx]; idx += 1
        release_times = np.array([release1, release1 + int2, release1 + int2 + int3])

        delays = np.array([x[idx + j] for j in range(3)]); idx += 3
        order = INTERCEPT_ORDER[['FY1', 'FY2', 'FY3', 'FY4', 'FY5'][drone_idx]]

        drone_params.append({
            'drone_init': DRONES_INIT[drone_idx],
            'theta': theta,
            'speed': speed,
            'release_times': release_times,
            'detonation_delays': delays,
            'missile_indices': order,
        })

    total_time, _ = simulate_multi_drone_multi_bomb(
        drone_params, dt=SEARCH_DT, t_total=T_TOTAL, target_keypoints=_SEARCH_KP
    )
    return total_time

def _joint_objective_final(x):
    """定稿档复算: 与 _joint_objective 同解法, 只是换成360关键点/dt0.005精细档"""
    idx = 0
    drone_params = []
    for drone_idx in range(N_DRONES):
        theta = x[idx]; idx += 1
        speed = x[idx]; idx += 1
        release1 = x[idx]; idx += 1
        int2 = x[idx]; idx += 1
        int3 = x[idx]; idx += 1
        release_times = np.array([release1, release1 + int2, release1 + int2 + int3])
        delays = np.array([x[idx + j] for j in range(3)]); idx += 3
        order = INTERCEPT_ORDER[['FY1', 'FY2', 'FY3', 'FY4', 'FY5'][drone_idx]]
        drone_params.append({
            'drone_init': DRONES_INIT[drone_idx], 'theta': theta, 'speed': speed,
            'release_times': release_times, 'detonation_delays': delays,
            'missile_indices': order,
        })
    total_time, _ = simulate_multi_drone_multi_bomb(
        drone_params, dt=FINAL_DT, t_total=T_TOTAL, target_keypoints=_FINAL_KP
    )
    return total_time

def solve_problem5():
    """求解问题5: 五机多弹多导弹协同策略"""
    print("=" * 60)
    print("问题5: 5架无人机协同投放烟幕弹 (PSO分步优化)")
    print("=" * 60)

```



```

# 预定义每架无人机的最佳搜索范围
# 基于初始位置分析
# theta_range 以各无人机指向其目标(0,200,0)的方位角为中心留出搜索余量
# (原范围都取在0°附近的小角度,方向朝向战场正前方而非目标,PS0永远搜不到有效遮蔽)
drone_configs = [
    { # FY1: 目标方位约179.4°(3.13rad)
      'theta_range': (2.73, 3.53),
      'speed_range': (DRONE_SPEED_MIN, DRONE_SPEED_MAX),
      'release_range': (0.0, 5.0),
      'delay_range': (0.0, 8.0),
    },
    { # FY2: 目标方位约-174.3°(-3.04rad)
      'theta_range': (-3.44, -2.64),
      'speed_range': (DRONE_SPEED_MIN, DRONE_SPEED_MAX),
      'release_range': (0.0, 8.0),
      'delay_range': (0.0, 10.0),
    },
    { # FY3: 目标方位约151.9°(2.65rad)
      'theta_range': (2.25, 3.05),
      'speed_range': (DRONE_SPEED_MIN, DRONE_SPEED_MAX),
      'release_range': (0.0, 12.0),
      'delay_range': (0.0, 12.0),
    },
    { # FY4: 目标方位约-170.7°(-2.98rad)
      'theta_range': (-3.38, -2.58),
      'speed_range': (DRONE_SPEED_MIN, DRONE_SPEED_MAX),
      'release_range': (0.0, 15.0),
      'delay_range': (0.0, 15.0),
    },
    { # FY5: 目标方位约170.4°(2.97rad)
      'theta_range': (2.57, 3.37),
      'speed_range': (DRONE_SPEED_MIN, DRONE_SPEED_MAX),
      'release_range': (0.0, 15.0),
      'delay_range': (0.0, 15.0),
    },
]

# =====
# 阶段1: 逐架无人机单独优化
# 改用三维空间采样(search_best_detonation, 与P4同款): 对该机 INTERCEPT_ORDER 里
# 每一枚弹各自的目标导弹采样反解最优起爆点, 得到每弹的(theta,speed,release,delay);
# 一架机只有一个航向/速度, 取各弹的中位数为该机航向/速度, 再评估实际单机遮蔽。
# (原"角度窗口PS0"方向被人为限死, FY2~FY5恒为0; 三维采样不预设窗口, 能找到解。)
# =====
print("\n阶段1: 逐架无人机单独优化(三维空间采样)...")

drone_names = ['FY1', 'FY2', 'FY3', 'FY4', 'FY5']
single_results = []
for drone_idx in range(N_DRONES):
    cfg = drone_configs[drone_idx]
    order = INTERCEPT_ORDER[drone_names[drone_idx]]
    print(f"\n--- 优化 {drone_names[drone_idx]} (拦截顺序 {order}) ---")

    # 对每枚弹的目标导弹分别三维采样
    per_bomb = []
    for mi in order:
        params, _ = search_best_detonation(DRONES_INIT[drone_idx], mi,
                                           n_fast=STAGE1_N_FAST_P5)
        per_bomb.append(params) # 可能为 None

    found = [p for p in per_bomb if p is not None]
    if found:
        theta_m = float(np.median([p['theta'] for p in found]))
        speed_m = float(np.median([p['speed'] for p in found]))

```

```

# 各弹的投放/延时; 没找到的弹用中位数保底, 保证3枚都有值
rel_med = float(np.median([p['release_time'] for p in found]))
dly_med = float(np.median([p['delay'] for p in found]))
rel = np.array([p['release_time'] if p is not None else rel_med for p in per_bomb])
dly = np.array([p['delay'] if p is not None else dly_med for p in per_bomb])
else:
    # 三维采样一枚都没找到: 退回窗口中心当保底种子(贡献可能为0, 交给阶段2再说)
    theta_m = float(np.mean(cfg['theta_range']))
    speed_m = float(np.mean(cfg['speed_range']))
    rel = np.array([0.5, 2.0, 3.5])
    dly = np.array([3.0, 3.0, 3.0])

# 夹逼物理/绝对范围(注意theta用全范围(-pi, pi), 不失cfg的猜测角度窗口——
# 三维采样反解出的最优方向可能落在窗口外, 夹逼窗口会把正确方向卡掉,
# 这正P4踩过的坑)。release/delay用较宽的绝对范围, 保证阶段2边界合法。
theta_m = float(np.clip(theta_m, -np.pi, np.pi))
speed_m = float(np.clip(speed_m, DRONE_SPEED_MIN, DRONE_SPEED_MAX))
rel = np.clip(rel, 0.0, 20.0)
dly = np.clip(dly, 0.0, 15.0)
# 投放时间按弹序排序并强制≥1s间隔(相邻投放约束)
rel = np.sort(rel)
for j in range(1, len(rel)):
    if rel[j] < rel[j - 1] + BOMB_INTERVAL_MIN:
        rel[j] = rel[j - 1] + BOMB_INTERVAL_MIN

# 用装配好的单机配置评估实际单机遮蔽时长(搜索框), 作为阶段1参考
f_single = SingleDroneObjective(drone_idx, order)(
    np.array([theta_m, speed_m, rel[0], rel[1] - rel[0], rel[2] - rel[1],
              dly[0], dly[1], dly[2]]))

single_results.append({
    'theta': theta_m, 'speed': speed_m,
    'release_times': rel, 'delays': dly,
    'time': f_single,
})
print(f" {drone_names[drone_idx]}:  $\theta=\{\text{np.degrees}(\text{theta\_m}):.1f\}^\circ$   $v=\{\text{speed\_m}:.1f\}\text{m/s}$  "
      f"单机遮蔽={f_single:.4f}s")

total_single = sum(r['time'] for r in single_results)
print(f"\n阶段1 单机优化总时长(简单求和): {total_single:.4f} s")

# =====
# 阶段2: 联合局部微调 (缩小搜索范围)
# =====
print("\n阶段2: 联合局部微调...")

# 构建40维变量, 搜索范围以阶段1结果为中心。
# 关键: 变量顺序必须与 _joint_objective 的解码顺序一致——每架无人机
# 连续8维 [theta, speed, release1, int2, int3, delay1, delay2, delay3],
# 一架接一架交错排列(旧代码曾按"变量类型分组"排列, 与解码顺序错位, 导致
# PSO搜的是完全打乱的空间、恒为0, 这里修正为交错排列)。
# 同时用阶段1各机的解拼成一个热启动种子, 避免40维纯随机初始化搜不到有效区。
bounds_joint = []
seed_joint = []
for drone_idx in range(N_DRONES):
    r = single_results[drone_idx]

    # theta: 以阶段1解为中心留±0.35rad(~20°)搜索余量, clamp到全范围(-pi, pi)
    # (不用cfg的猜测窗口做clamp——阶段1的theta来自三维采样, 可能在窗口外)
    t_center = r['theta']
    theta_lo = max(-np.pi, t_center - 0.35)
    theta_hi = min(np.pi, t_center + 0.35)
    bounds_joint.append((theta_lo, theta_hi))
    seed_joint.append(np.clip(t_center, theta_lo, theta_hi))

```

```

# speed:  $\pm 10$  m/s, clamp到[80,120]
s_center = r['speed']
speed_lo = max(DRONE_SPEED_MIN, s_center - 10.0)
speed_hi = min(DRONE_SPEED_MAX, s_center + 10.0)
bounds_joint.append((speed_lo, speed_hi))
seed_joint.append(np.clip(s_center, speed_lo, speed_hi))

# release1:  $\pm 1.0$ s, clamp到[0,20]
rel_lo = max(0.0, r['release_times'][0] - 1.0)
rel_hi = min(20.0, r['release_times'][0] + 1.0)
bounds_joint.append((rel_lo, rel_hi))
seed_joint.append(np.clip(r['release_times'][0], rel_lo, rel_hi))

# interval 1->2
int_center = r['release_times'][1] - r['release_times'][0]
i2_lo = max(BOMB_INTERVAL_MIN, int_center - 0.5)
i2_hi = max(i2_lo, int_center + 0.5)
bounds_joint.append((i2_lo, i2_hi))
seed_joint.append(np.clip(int_center, i2_lo, i2_hi))

# interval 2->3
int_center2 = r['release_times'][2] - r['release_times'][1]
i3_lo = max(BOMB_INTERVAL_MIN, int_center2 - 0.5)
i3_hi = max(i3_lo, int_center2 + 0.5)
bounds_joint.append((i3_lo, i3_hi))
seed_joint.append(np.clip(int_center2, i3_lo, i3_hi))

# delays 1/2/3:  $\pm 1.5$ s, clamp到[0,15]
for j in range(3):
    d_center = r['delays'][j]
    d_lo = max(0.0, d_center - 1.5)
    d_hi = min(15.0, d_center + 1.5)
    bounds_joint.append((d_lo, d_hi))
    seed_joint.append(np.clip(d_center, d_lo, d_hi))

print(f"联合优化变量维度: {len(bounds_joint)}")
pso_joint = PSO(_joint_objective, bounds_joint, n_particles=PSO_SWARM_P5,
                max_iter=PSO_ITER_P5, maximize=True, verbose=True,
                seed_positions=np.array(seed_joint))
x_opt_joint, f_opt_joint = pso_joint.optimize()

print("\n局部精修(Powell)...")
x_polished, f_polished = local_polish(_joint_objective, x_opt_joint, bounds_joint)
if f_polished < f_opt_joint:
    print(f" 精修有提升: {f_opt_joint:.4f}s -> {f_polished:.4f}s")
    x_opt_joint, f_opt_joint = x_polished, f_polished
else:
    print(f" 精修没有提升({f_polished:.4f}s <= {f_opt_joint:.4f}s), 保留PSO原结果")

# 定稿档复算: 搜索用180关键点/dt0.01, 最优解用360关键点/dt0.005复算上报
f_final = _joint_objective_final(x_opt_joint)
print(f" 定稿档复算(360关键点/dt{FINAL_DT}): {f_opt_joint:.4f}s(搜索档) -> {f_final:.4f}s(定稿)")
f_opt_joint = f_final

print(f"\n联合优化总有效遮蔽时长: {f_opt_joint:.4f}s")

# =====
# 解析并输出结果
# =====
idx = 0
final_results = []
for drone_idx in range(N_DRONES):
    theta = x_opt_joint[idx]; idx += 1

```

```

        speed = x_opt_joint[idx]; idx += 1
        release1 = x_opt_joint[idx]; idx += 1
        int2 = x_opt_joint[idx]; idx += 1
        int3 = x_opt_joint[idx]; idx += 1
        release_times = np.array([release1, release1+int2, release1+int2+int3])
        delays = np.array([x_opt_joint[idx+j] for j in range(3)]); idx += 3
        order = INTERCEPT_ORDER[['FY1', 'FY2', 'FY3', 'FY4', 'FY5'][drone_idx]]
        final_results.append({
            'theta': theta, 'speed': speed,
            'release_times': release_times, 'delays': delays,
            'order': order,
        })

drone_names = ['FY1', 'FY2', 'FY3', 'FY4', 'FY5']
print(f"\n{'='*60}")
print("最终优化结果:")
print(f"\n{'='*60}")

for i, r in enumerate(final_results):
    direction = np.array([np.cos(r['theta']), np.sin(r['theta']), 0.0])
    print(f"\ndrone_names[i]: θ={r['theta']:.4f}rad({np.degrees(r['theta']:.1f}°), "
          f"v={r['speed']:.2f}m/s")
    print(f" 拦截顺序: {r['order']}")
    for j in range(3):
        release_pos = DRONES_INIT[i] + r['speed'] * direction * r['release_times'][j]
        detonation_pos = release_pos + r['speed'] * direction * r['delays'][j]
        detonation_pos[2] -= 0.5 * 9.8 * r['delays'][j] ** 2
        print(f" 弹{j+1}: 投放t={r['release_times'][j]:.4f}s, 延时={r['delays'][j]:.4f}s, "
              f"起爆点=({detonation_pos[0]:.1f},{detonation_pos[1]:.1f},{detonation_pos[2]:.1f})")

print(f"\n总有效遮蔽时长: {f_opt_joint:.4f} s")

# 保存
save_result3(final_results, f_opt_joint)

return final_results, f_opt_joint

def save_result3(final_results, total_time):
    """保存问题5结果到 result3.xlsx"""
    wb = openpyxl.Workbook()
    ws = wb.active
    ws.title = "问题5结果"

    headers = ["无人机", "航向角(rad)", "航向角(°)", "飞行速度(m/s)",
               "烟幕弹编号", "目标导弹", "投放时间(s)", "起爆延时(s)",
               "投放点X", "投放点Y", "投放点Z",
               "起爆点X", "起爆点Y", "起爆点Z",
               "总有效遮蔽时长(s)"]
    for col, header in enumerate(headers, 1):
        ws.cell(row=1, column=col, value=header)

    drone_names = ['FY1', 'FY2', 'FY3', 'FY4', 'FY5']
    row = 2
    for i, r in enumerate(final_results):
        direction = np.array([np.cos(r['theta']), np.sin(r['theta']), 0.0])
        for j in range(3):
            missile_label = f"M{r['order'][j]+1}"
            release_pos = DRONES_INIT[i] + r['speed'] * direction * r['release_times'][j]
            detonation_pos = release_pos + r['speed'] * direction * r['delays'][j]
            detonation_pos[2] -= 0.5 * 9.8 * r['delays'][j] ** 2

            ws.cell(row=row, column=1, value=drone_names[i])
            ws.cell(row=row, column=2, value=round(r['theta'], 6))

```

```

ws.cell(row=row, column=3, value=round(np.degrees(r['theta']), 4))
ws.cell(row=row, column=4, value=round(r['speed'], 2))
ws.cell(row=row, column=5, value=j + 1)
ws.cell(row=row, column=6, value=missile_label)
ws.cell(row=row, column=7, value=round(r['release_times'][j], 4))
ws.cell(row=row, column=8, value=round(r['delays'][j], 4))
ws.cell(row=row, column=9, value=round(release_pos[0], 2))
ws.cell(row=row, column=10, value=round(release_pos[1], 2))
ws.cell(row=row, column=11, value=round(release_pos[2], 2))
ws.cell(row=row, column=12, value=round(detonation_pos[0], 2))
ws.cell(row=row, column=13, value=round(detonation_pos[1], 2))
ws.cell(row=row, column=14, value=round(detonation_pos[2], 2))
if i == 0 and j == 0:
    ws.cell(row=row, column=15, value=round(total_time, 4))
row += 1

filepath = "result3.xlsx"
wb.save(filepath)
print(f"\n结果已保存至: {filepath}")

if __name__ == "__main__":
    solve_problem5()

```

```

"""
烟雾干扰弹的投放策略 - 参数配置
"""

import numpy as np

# =====
# 物理常数
# =====

G = 9.8 # 重力加速度 (m/s2)

# =====
# 物理参数
# =====

# 烟雾云团参数
SMOKE_SINK_SPEED = 2.5 # 云团下沉速度 (m/s)
EFFECTIVE_RADIUS = 10.0 # 有效遮蔽半径 (m)
EFFECTIVE_DURATION = 20.0 # 有效遮蔽持续时间 (s)

# 导弹参数
MISSILE_SPEED = 300.0 # 导弹飞行速度 (m/s)

# 无人机参数
DRONE_SPEED_MIN = 80.0 # 最小飞行速度 (m/s)
DRONE_SPEED_MAX = 120.0 # 最大飞行速度 (m/s)
BOMB_INTERVAL_MIN = 1.0 # 相邻两弹投放最小间隔 (s)

# =====
# 问题1 固定参数
# =====

P1_RELEASE_TIME = 1.2 # 受领任务后投放时间 (s)
P1_DETONATION_DELAY = 3.2 # 投放后起爆延时 (s)
P1_DRONE_SPEED = 120.0 # FY1速度 (m/s)
P1_DRONE_THETA = np.pi # FY1飞行方向(朝向假目标方向)

# =====
# 目标参数
# =====

TARGET_CENTER = np.array([0.0, 200.0, 0.0]) # 真目标下底面圆心
TARGET_RADIUS = 7.0 # 圆柱半径 (m)
TARGET_HEIGHT = 10.0 # 圆柱高度 (m)
FAKE_TARGET = np.array([0.0, 0.0, 0.0]) # 假目标(原点)

# =====
# 导弹初始位置 (x, y, z) 单位: m
# =====

MISSILES_INIT = np.array([
    [20000.0, 0.0, 2000.0], # M1
    [19000.0, 600.0, 2100.0], # M2
    [18000.0, -600.0, 1900.0], # M3
])

# 导弹飞行方向: 指向假目标(原点)
MISSILES_DIR = np.array([
    -MISSILES_INIT[0] / np.linalg.norm(MISSILES_INIT[0]),
    -MISSILES_INIT[1] / np.linalg.norm(MISSILES_INIT[1]),
    -MISSILES_INIT[2] / np.linalg.norm(MISSILES_INIT[2]),
])

# =====
# 无人机初始位置 (x, y, z) 单位: m
# =====

DRONES_INIT = np.array([

```

```

    [17800.0, 0.0,    1800.0], # FY1
    [12000.0, 1400.0, 1400.0], # FY2
    [6000.0,  -3000.0, 700.0], # FY3
    [11000.0, 2000.0, 1800.0], # FY4
    [13000.0, -2000.0, 1300.0], # FY5
1)

# =====
# 仿真参数
# =====

DT = 0.005          # 时间步长 (s) - 优化使用
DT_FINE = 0.0001    # 精细时间步长 - 问题1/2验证使用
T_TOTAL = 50.0       # 总模拟时间 (s)

# 目标离散化关键点数
# 使用圆柱上下底面的圆周采样 + 侧面采样
N_CIRCLE_POINTS = 720 # 每个圆周的采样点数 (上下底面共1440点)
N_SIDE_LAYERS = 10    # 侧面采样层数 (不含上下底面)

# =====
# 双精度档: 搜索档(跑PSO用, 快而略糙) VS 定稿档(复算最优解用, 慢而精)
# 搜索档比定稿档单次评估快约7倍、内存省约7倍; README的敏感性分析表明关键点数>100
# 结果即收敛, 180圆周点已足够引导PSO找到正确的解, 最终数值再用定稿档复算一遍保证精度。
# =====

SEARCH_N_CIRCLE = 180 # 搜索档: 每个圆周采样点数
SEARCH_N_LAYERS = 5   # 搜索档: 侧面层数
SEARCH_DT = 0.01      # 搜索档: 时间步长
FINAL_N_CIRCLE = 360  # 定稿档: 每个圆周采样点数
FINAL_N_LAYERS = 10   # 定稿档: 侧面层数
FINAL_DT = 0.005      # 定稿档: 时间步长

# =====
# PSO 并行进程数上限
# =====
# os.cpu_count() 在多核机器上可能是32, 一次开32个进程 × 每次评估几百MB临时数组会把
# 内存撑爆(实测问题4/5曾因此OOM崩在第4问)。这里封顶到一个内存安全的进程数。
# 可用环境变量 CUMCM_PSO_WORKERS 覆盖。经验: 每个worker峰值约0.25~0.4GB,
# 10个进程约2.5~4GB, 配合本机可用内存留足余量。
PSO_MAX_WORKERS = 10

# =====
# PSO 优化参数
# =====

PSO_SWARM_SIZE = 200 # 粒子群规模
PSO_MAX_ITER = 100   # 最大迭代次数
PSO_W = 0.7          # 惯性权重
PSO_C1 = 1.5          # 个体学习因子
PSO_C2 = 1.5          # 社会学习因子

# =====
# 问题2 变量范围
# =====

P2_BOUNDS = {
    'theta': (2.73, 3.53), # 航向角 (rad) - FY1指向真目标方位角(约3.13rad)附近
    'speed': (DRONE_SPEED_MIN, DRONE_SPEED_MAX),
    'release_time': (0.0, 15.0),
    'detonation_delay': (0.0, 6.0),
}

# =====
# 问题5 无人机拦截导弹顺序
# =====

INTERCEPT_ORDER = {

```

```
'FY1': [0, 0, 0],      # M1, M1, M1
'FY2': [1, 0, 2],      # M2, M1, M3
'FY3': [2, 0, 1],      # M3, M1, M2
'FY4': [1, 0, 2],      # M2, M1, M3
'FY5': [2, 0, 1],      # M3, M1, M2
}
```



```

"""
主运行脚本 - 依次运行所有5个问题，或通过命令行参数指定只运行其中若干个

用法：
python main.py          # 依次运行全部5个问题（不传参数，等价于原有行为）
python main.py 1        # 只运行问题1
python main.py 2 4      # 只运行问题2和问题4
"""

import sys
import os
import time

# 切换到脚本所在目录
os.chdir(os.path.dirname(os.path.abspath(__file__)))

from problem1 import solve_problem1
from problem2 import solve_problem2
from problem3 import solve_problem3
from problem4 import solve_problem4
from problem5 import solve_problem5

def run_problem1(results):
    print("\n" + "#" * 70)
    print("# 问题1: 固定参数, 计算有效遮蔽时长")
    print("#" * 70)
    t_start = time.time()
    t1 = solve_problem1()
    elapsed = time.time() - t_start
    results['problem1'] = {'time': t1, 'elapsed': elapsed}
    print(f"\n[问题1 完成, 耗时: {elapsed:.1f}s]")

def run_problem2(results):
    print("\n" + "#" * 70)
    print("# 问题2: 单机单弹最优投放策略")
    print("#" * 70)
    t_start = time.time()
    x2, t2 = solve_problem2()
    elapsed = time.time() - t_start
    results['problem2'] = {'x': x2, 'time': t2, 'elapsed': elapsed}
    print(f"\n[问题2 完成, 耗时: {elapsed:.1f}s]")

def run_problem3(results):
    print("\n" + "#" * 70)
    print("# 问题3: 单机三弹最优投放策略 (result1.xlsx)")
    print("#" * 70)
    t_start = time.time()
    x3, t3 = solve_problem3()
    elapsed = time.time() - t_start
    results['problem3'] = {'x': x3, 'time': t3, 'elapsed': elapsed}
    print(f"\n[问题3 完成, 耗时: {elapsed:.1f}s]")

def run_problem4(results):
    print("\n" + "#" * 70)
    print("# 问题4: 三机协同最优投放策略 (result2.xlsx)")
    print("#" * 70)
    t_start = time.time()
    x4, t4 = solve_problem4()
    elapsed = time.time() - t_start
    results['problem4'] = {'x': x4, 'time': t4, 'elapsed': elapsed}
    print(f"\n[问题4 完成, 耗时: {elapsed:.1f}s]")

```

```

def run_problem5(results):
    print("\n" + "#" * 70)
    print("# 问题5: 五机多弹协同最优投放策略 (result3.xlsx)")
    print("#" * 70)
    t_start = time.time()
    res5, t5 = solve_problem5()
    elapsed = time.time() - t_start
    results['problem5'] = {'result': res5, 'time': t5, 'elapsed': elapsed}
    print(f"\n[问题5 完成, 耗时: {elapsed:.1f}s]")

# 问题编号 -> 对应的执行函数, 供命令行参数选择时查找
RUNNERS = {1: run_problem1, 2: run_problem2, 3: run_problem3, 4: run_problem4, 5: run_problem5}

def parse_selection(argv):
    """不传参数则跑全部; 传数字则只跑对应问题, 例如 `python main.py 1 3`"""
    if not argv:
        return [1, 2, 3, 4, 5]
    try:
        selected = sorted({int(a) for a in argv})
    except ValueError:
        print("用法: python main.py [问题编号 ...] 例如: python main.py 1 3")
        print("不带参数则依次运行全部5个问题。")
        sys.exit(1)
    invalid = [n for n in selected if n not in RUNNERS]
    if invalid:
        print(f"问题编号只能是 1~5, 收到无效值: {invalid}")
        sys.exit(1)
    return selected

def main():
    selected = parse_selection(sys.argv[1:])

    print("=" * 70)
    print("    烟幕干扰弹的投放策略 - Python求解")
    print(f"    本次运行问题: {selected}")
    print("=" * 70)
    print()
    print("主要参数:")
    print("    烟幕云团下沉速度: 2.5 m/s")
    print("    无人机速度范围: 80~120 m/s")
    print("    问题1投放时间: 1.2 s")
    print("    问题1起爆延时: 3.2 s")
    print()

    results = {}
    total_start = time.time()
    for n in selected:
        RUNNERS[n](results)
    total_elapsed = time.time() - total_start

    print("\n" + "=" * 70)
    print("                本次运行结果汇总")
    print("=" * 70)
    if 'problem1' in results:
        print(f"    问题1 (固定参数验证):    {results['problem1']['time']:.4f} s")
    if 'problem2' in results:
        print(f"    问题2 (单机单弹最优):    {results['problem2']['time']:.4f} s")
    if 'problem3' in results:
        print(f"    问题3 (单机三弹最优):    {results['problem3']['time']:.4f} s → result1.xlsx")

```

```

if 'problem4' in results:
    print(f"    问题4 (三机协同最优):    {results['problem4']['time']:.4f} s → result2.xlsx")
if 'problem5' in results:
    print(f"    问题5 (五机多弹协同):    {results['problem5']['time']:.4f} s → result3.xlsx")
print(f"\n    总运行时间: {total_elapsed:.1f} s ({total_elapsed/60:.1f} min)")
print("=" * 70)

if selected == [1, 2, 3, 4, 5]:
    pass

if __name__ == "__main__":
    main()

```

```
"""
```

```
论文配图批量生成脚本
```

```
输出: paper_assets/generated/paper_fig_*.png
```

```
"""
```

```
import os
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.patches import Circle, Rectangle
from matplotlib.patches import FancyArrowPatch
from mpl_toolkits.mplot3d.art3d import Poly3DCollection
import openpyxl

import config as cfg
from simulation import (
    missile_position, simulate_single_bomb,
    get_target_keypoints, _bomb_coverage_mask,
)

plt.rcParams.update({
    "font.sans-serif": ["SimHei", "Microsoft YaHei", "PingFang SC", "sans-serif"],
    "axes.unicode_minus": False,
    "figure.dpi": 150,
    "axes.grid": True,
    "grid.alpha": 0.25,
    "grid.linewidth": 0.5,
    "axes.spines.top": False,
    "axes.spines.right": False,
    "lines.linewidth": 1.4,
})

C_MISSILE = "#C0392B"
C_DRONE = "#2E6F9E"
C_TARGET = "#4C4C4C"
C_FAKE = "#8A8A8A"
C_SMOKE = "#7F8C8D"
C_ACCENT = "#D68910"
C_OK = "#27AE60"

def out_path(name):
    p = os.path.join(os.path.dirname(os.path.abspath(__file__)), "..", "paper_assets", "generated", name)
    p = os.path.abspath(p)
    os.makedirs(os.path.dirname(p), exist_ok=True)
    return p

# =====
# 图1 场景初始布局
# =====

def fig_scenario():
    fig = plt.figure(figsize=(9, 6.5))
    ax = fig.add_subplot(111, projection="3d")
    M = cfg.MISSILES_INIT
    D = cfg.DRONES_INIT
    ax.scatter(M[:,0], M[:,1], M[:,2], c=C_MISSILE, marker="^", s=90, label="未装导弹", zorder=5)
    for i, n in enumerate(["M1", "M2", "M3"]):
        ax.text(M[i,0]+400, M[i,1], M[i,2], n, color=C_MISSILE, fontsize=9)
    ax.scatter(D[:,0], D[:,1], D[:,2], c=C_DRONE, marker="o", s=70, label="无人机", zorder=5)
    for i, n in enumerate(["FY1", "FY2", "FY3", "FY4", "FY5"]):
        ax.text(D[i,0]+400, D[i,1], D[i,2], n, color=C_DRONE, fontsize=9)
    ax.scatter(*cfg.FAKE_TARGET, c=C_FAKE, marker="x", s=80, label="假目标(原点)", zorder=5)
```

```

ax.scatter(*cfg.TARGET_CENTER, c=C_TARGET, marker="s", s=60, label="真目标", zorder=5)
for i in range(3):
    ax.plot([M[i,0], 0], [M[i,1], 0], [M[i,2], 0],
            color=C_MISSILE, linewidth=0.6, linestyle="--", alpha=0.4)
ax.set_xlabel("X (m)"); ax.set_ylabel("Y (m)"); ax.set_zlabel("Z (m)")
ax.set_title("场景初始布局 (导弹、无人机、真/假目标)", fontsize=12)
ax.legend(loc="upper left", frameon=False, fontsize=9)
ax.view_init(elev=18, azim=-60)
fig.tight_layout()
fig.savefig(out_path("paper_fig_1_scenario.png"), dpi=180)
plt.close(fig)
print(" fig_scenario OK")

# =====
# 图2 视锥线遮挡判别几何示意 (2D 模式图)
# =====
def fig_view_cone():
    fig, ax = plt.subplots(figsize=(10, 6))
    M = np.array([1.0, 7.0])
    S = np.array([5.5, 4.0])
    P_T = np.array([8.0, 2.0])
    R = 1.0
    vec = P_T - M; norm = np.linalg.norm(vec)
    alpha = np.arcsin(R / np.linalg.norm(S - M)) # 半顶角
    # 云团圆
    circ = plt.Circle(S, R, color=C_DRONE, fill=True, alpha=0.30, linewidth=1.5)
    ax.add_patch(circ)
    circ_edge = plt.Circle(S, R, color=C_DRONE, fill=False, linewidth=1.5)
    ax.add_patch(circ_edge)
    # 视锥母线: 从 M 画到 S 处的两条切线 (用半顶角近似)
    axis = (P_T - M) / np.linalg.norm(P_T - M)
    perp = np.array([-axis[1], axis[0]])
    L = np.linalg.norm(S - M)
    # 视锥左/右母线
    p_L = S + L * np.tan(alpha) * perp + (np.linalg.norm(P_T - S)) * axis
    p_R = S - L * np.tan(alpha) * perp + (np.linalg.norm(P_T - S)) * axis
    ax.plot([M[0], p_L[0]], [M[1], p_L[1]], color=C_DRONE, linewidth=1.2, linestyle="--", alpha=0.8, label="视锥母线")
    ax.plot([M[0], p_R[0]], [M[1], p_R[1]], color=C_DRONE, linewidth=1.2, linestyle="--", alpha=0.8)
    # 视锥轴
    ax.annotate("", xy=P_T, xytext=M,
               arrowprops=dict(arrowstyle="->", color="black", lw=1.0, linestyle="--", alpha=0.5))
    # 真目标
    target_w, target_h = 0.6, 1.4
    rect = plt.Rectangle((P_T[0]-target_w/2, P_T[1]-target_h/2), target_w, target_h,
                        color=C_TARGET, alpha=0.5)
    ax.add_patch(rect)
    # 关键点
    for y_ in [-1, 1]:
        for y_ in [-0.3, 0, 0.3]:
            ax.scatter([P_T[0] + sign*target_w/2], [P_T[1] + y_],
                      c=C_ACCENT, s=30, marker="o", zorder=5)
    # 视线
    for sign in [-1, 1]:
        for y_ in [-0.3, 0, 0.3]:
            ax.plot([M[0], P_T[0] + sign*target_w/2], [M[1], P_T[1] + y_],
                    color="gray", linewidth=0.5, alpha=0.4, linestyle="-")
    # 标注
    label_kw = dict(fontsize=12, fontweight='bold')
    ax.text(M[0]-0.2, M[1]+0.3, "A 导弹 M", color=C_MISSILE, **label_kw)
    ax.text(S[0]-0.3, S[1]+0.3, "云团 S", color=C_DRONE, **label_kw)
    ax.text(P_T[0]+0.1, P_T[1]+0.9, "真目标 G", color=C_TARGET, **label_kw)
    ax.text(S[0]+0.15, S[1]+0.05, "R", color=C_DRONE, fontsize=10, fontweight='bold')
    # 半顶角标注

```

```

ax.text(3.0, 5.5, r"$\alpha$ (半顶角)", color=C_DRONE, fontsize=12, style='italic')
# 公式标注
ax.text(2.5, 0.8, r"$\sin\alpha = R \backslash / \backslash | M - S |$", fontsize=12,
        bbox=dict(boxstyle="round,pad=0.3", facecolor="#FFF8E1", edgecolor=C_ACCENT))
# 条件框
ax.text(0.2, 0.5, "完全遮蔽  $\Rightarrow \forall$  关键点  $K_i: y_i \leq \alpha$ ",
        fontsize=10, transform=ax.transAxes,
        bbox=dict(boxstyle="round,pad=0.3", facecolor="#E8F4F8", edgecolor=C_DRONE))
ax.set_xlim(0, 9.5); ax.set_ylim(0, 8.5)
ax.set_aspect("equal")
ax.set_xlabel("X (等比示意)"); ax.set_ylabel("Y (等比示意)")
ax.set_title("视锥线遮蔽判断几何示意 (2D 模式图)", fontsize=13, pad=10)
ax.legend(loc="upper right", frameon=True, fontsize=9)
ax.grid(True, alpha=0.3)
fig.tight_layout()
fig.savefig(out_path("paper_fig_2_view_cone.png"), dpi=180)
plt.close(fig)
print(" fig_view_cone OK")

# =====
# 图3 多机多云团协同遮蔽机理 (2D 概念示意图, 关键点错列)
# =====
def fig_multi_cloud():
    fig, ax = plt.subplots(figsize=(12, 6.5))

    # 目标圆柱 (左端) — 用矩形表示
    target_x = 0.5
    target_w = 1.0
    target_h = 3.0
    rect = plt.Rectangle((target_x, 0), target_w, target_h,
                        color=C_TARGET, alpha=0.4, ec="black", lw=1.2)
    ax.add_patch(rect)
    ax.text(target_x + target_w/2, target_h + 0.25, "真目标 (圆柱纵截面)",
            ha="center", fontsize=12, color=C_TARGET, fontweight='bold')

    # 关键点 (6个, 2列错开避免与云重叠)
    # 左侧列 (紧贴目标): K1上、K3中、K5下
    # 右侧列 (靠近云团位置): K2上、K4中、K6下
    keypoints = [
        (target_x + target_w, 2.4, "K1"), # 紧贴目标 上
        (4.2, 2.4, "K2"), # 靠近云1 上
        (target_x + target_w, 1.5, "K3"), # 紧贴目标 中
        (4.2, 1.5, "K4"), # 靠近云2 中
        (target_x + target_w, 0.6, "K5"), # 紧贴目标 下
        (4.2, 0.6, "K6"), # 靠近云3 下
    ]
    for (x_, y_, lab) in keypoints:
        ax.scatter([x_], [y_], c=C_ACCENT, s=55, zorder=6, edgecolor='black', linewidth=0.6)
        # 标注: 左列放在更左、右列放在更右, 避免被云挡住
        if x_ < 1.5:
            ax.text(x_-0.25, y_, lab, fontsize=11, fontweight='bold', va='center', ha='right')
        else:
            ax.text(x_+0.30, y_, lab, fontsize=11, fontweight='bold', va='center', ha='left')

    # 视线 (导弹→关键点, 灰色虚线)
    missile_x = 9.5
    missile_y = 1.5
    ax.scatter([missile_x], [missile_y], c=C_MISSILE, marker="^", s=220, zorder=5)
    ax.text(missile_x+0.2, missile_y+0.15, "导弹 M", fontsize=12, color=C_MISSILE, fontweight='bold')
    for (x_, y_, lab) in keypoints:
        ax.plot([missile_x, x_], [missile_y, y_], color="gray", linewidth=0.4, alpha=0.25, linestyle=':')

```

```

# 三朵云（垂直方向分开更多，避免重叠）
# 云1：挡上半 K1, K2
c1 = plt.Circle((5.0, 2.4), 0.6, color=C_SMOKE, alpha=0.6, ec=C_DRONE, lw=1.5)
ax.add_patch(c1)
ax.text(5.0, 3.30, "云 1\n(挡 K1, K2)", ha="center", fontsize=10, color=C_DRONE, fontweight='bold')

# 云2：挡中段 K3, K4
c2 = plt.Circle((5.0, 1.5), 0.6, color="#5DADE2", alpha=0.6, ec="#1B4F72", lw=1.5)
ax.add_patch(c2)
ax.text(5.0, 2.40, "云 2\n(挡 K3, K4)", ha="center", fontsize=10, color="#1B4F72", fontweight='bold')

# 云3：挡下半 K5, K6
c3 = plt.Circle((5.0, 0.6), 0.6, color="#48C9B0", alpha=0.6, ec="#117A65", lw=1.5)
ax.add_patch(c3)
ax.text(5.0, 0.0, "云 3\n(挡 K5, K6)", ha="center", fontsize=10, color="#117A65", fontweight='bold')

# 视线遮挡（彩色粗线）—— 导弹到被各云挡住的视线
blocked_map = {"K1":C_DRONE, "K2":C_DRONE, "K3": "#1B4F72", "K4": "#1B4F72", "K5": "#117A65", "K6": "#117A65"}
for (x_, y_, lab) in keypoints:
    col = blocked_map[lab]
    ax.plot([missile_x, x_], [missile_y, y_], color=col, linewidth=2.0, alpha=0.7)

# "被挡住"标记（绿色圈）
for (x_, y_, _) in keypoints:
    ax.scatter([x_], [y_], s=320, facecolor='none', edgecolor='green', linewidth=2.0, zorder=4)

# 公式框
ax.text(0.5, 3.95, r"完全遮蔽  $\Leftrightarrow$   $\forall$  关键点  $K_i$ : 至少一朵云挡住对应视线",
        fontsize=11, fontweight='bold',
        bbox=dict(boxstyle="round,pad=0.4", facecolor="#FFF8E1", edgecolor=C_ACCENT, lw=1.0))

# 视线方向箭头
ax.annotate("", xy=(missile_x-0.3, missile_y), xytext=(missile_x+1.0, missile_y),
            arrowprops=dict(arrowstyle="->", color="black", lw=1.2))
ax.text(missile_x-1.3, missile_y+0.25, "视线方向", fontsize=10, color="black")

ax.set_xlim(-1.0, 11)
ax.set_ylim(-0.7, 4.2)
ax.set_aspect("equal")
ax.set_xlabel("沿视线方向 (m, 等比示意)", fontsize=11)
ax.set_ylabel("目标高度方向 (m, 等比示意)", fontsize=11)
ax.set_title("多机多云团协同（互补）遮蔽机理示意", fontsize=13, pad=10)
ax.legend(handles=[
    plt.scatter([0], [0], c=C_MISSILE, marker="^", s=100, label="导弹"),
    plt.scatter([0], [0], c=C_TARGET, marker="s", s=100, label="真目标（截面）"),
    plt.scatter([0], [0], c=C_ACCENT, s=50, label="关键点  $K_i$ "),
    plt.scatter([0], [0], s=200, facecolor='none', edgecolor='green', lw=2, label="被云挡住"),
], loc="lower right", frameon=True, fontsize=9)
ax.grid(True, alpha=0.3)
fig.tight_layout()
fig.savefig(out_path("paper_fig_3_multi_cloud.png"), dpi=180)
plt.close(fig)
print(" fig_multi_cloud OK")

# =====
# 图4 问题1距离时序
# =====
def fig_pl_timeline():
    drone_init = cfg.DRONES_INIT[0]
    theta = cfg.P1_DRONE_THETA
    speed = cfg.P1_DRONE_SPEED
    tr = cfg.P1_RELEASE_TIME

```

```

td = cfg.P1_DETONATION_DELAY
te = tr + td
te_end = te + cfg.EFFECTIVE_DURATION
ts = np.arange(0, 15, 0.001)
M_pos = np.array([missile_position(t, 0) for t in ts])
dist_mt = np.linalg.norm(M_pos - cfg.TARGET_CENTER, axis=1)
direction = np.array([np.cos(theta), np.sin(theta), 0.0])
active = (ts >= te) & (ts <= te_end)
smoke_pos = np.full((len(ts), 3), np.nan)
if np.any(active):
    FY_rel = drone_init + speed * direction * tr
    bx = FY_rel[0] + speed * direction[0] * td
    by = FY_rel[1] + speed * direction[1] * td
    bh = FY_rel[2] - 0.5 * cfg.G * td**2
    t_a = ts[active]
    smoke_pos[active, 0] = bx
    smoke_pos[active, 1] = by
    smoke_pos[active, 2] = bh - cfg.SMOKE_SINK_SPEED * (t_a - te)
dist_ms = np.linalg.norm(M_pos - smoke_pos, axis=1)
valid = ~np.isnan(dist_ms)
i_min = np.nanargmin(dist_ms)
t_min, d_min = ts[i_min], dist_ms[i_min]
fig, ax = plt.subplots(figsize=(8, 4.5))
ax.plot(ts, dist_mt, color=C_MISSILE, linewidth=1.6, label="导弹-真目标距离")
ax.plot(ts, dist_ms, color=C_DRONE, linewidth=1.6, label="导弹-云-伪距离")
ax.axhline(cfg.EFFECTIVE_RADIUS, color=C_ACCENT, linewidth=1.0, linestyle=":",
            label=f"有效遮蔽半径 {cfg.EFFECTIVE_RADIUS:.0f} m")
ax.axvline(te, color="black", linewidth=0.8, linestyle="--", alpha=0.6)
ax.text(te, ax.get_ylim()[1]*0.92, f"起爆 t={te:.1f}s", fontsize=8, va="top")
ax.set_xlabel("t (s)"); ax.set_ylabel("距离 (m)")
ax.set_title(f"问题1距离时序: release={tr}s, delay={td}s "
             f"(下沉{cfg.SMOKE_SINK_SPEED}m/s, 速度{cfg.DRONE_SPEED_MIN:.0f}-{cfg.DRONE_SPEED_MAX:.0f}m/s)",
             fontsize=10)
ax.legend(loc="center right", frameon=False, fontsize=9)
axins = ax.inset_axes([0.10, 0.14, 0.30, 0.42])
win = (ts > t_min - 1.5) & (ts < t_min + 1.5)
axins.plot(ts[win], dist_ms[win], color=C_DRONE, linewidth=1.4)
axins.axhline(cfg.EFFECTIVE_RADIUS, color=C_ACCENT, linewidth=1.0, linestyle=":")
axins.scatter([t_min], [d_min], color=C_DRONE, s=18, zorder=5)
axins.annotate(f"最近 {d_min:.2f} m\n(阈值{cfg.EFFECTIVE_RADIUS:.0f}m, 差{d_min-cfg.EFFECTIVE_RADIUS:+.2f}m)",
               xy=(t_min, d_min), xytext=(0.55, 0.75), textcoords="axes fraction",
               fontsize=7.5, ha="left",
               arrowprops=dict(arrowstyle="-", color="#666666", linewidth=0.6))
axins.set_xlabel("t (s)", fontsize=7)
axins.tick_params(labelsize=7)
axins.grid(alpha=0.25, linewidth=0.4)
fig.tight_layout()
fig.savefig(out_path("paper_fig_4_p1_timeline.png"), dpi=180)
plt.close(fig)
print(" fig_p1_timeline OK")

# =====
# 图5 烟雾弹投放→平抛→起爆→下沉 全流程
# =====
def fig_bomb_trajectory():
    drone_init = np.array([17800., 0., 1800.])
    theta = np.radians(178.2); v = 83.8
    tr = 0.30; td = 2.91
    direction = np.array([np.cos(theta), np.sin(theta), 0.0])
    FY_rel = drone_init + v * direction * tr
    t_drop = np.linspace(0, td, 60)
    drop_x = FY_rel[0] + v * direction[0] * t_drop
    drop_y = FY_rel[1] + v * direction[1] * t_drop

```



```

drop_z = FY_rel[2] - 0.5 * cfg.G * t_drop**2
burst = np.array([drop_x[-1], drop_y[-1], drop_z[-1]])
te = tr + td
t_sink = np.linspace(0, 20, 80)
sink_x = np.full_like(t_sink, burst[0])
sink_y = np.full_like(t_sink, burst[1])
sink_z = burst[2] - cfg.SMOKE_SINK_SPEED * t_sink
t_fly = np.linspace(0, tr+td+2, 50)
fly_x = drone_init[0] + v * direction[0] * t_fly
fly_y = drone_init[1] + v * direction[1] * t_fly
fly_z = np.full_like(t_fly, drone_init[2])
fig = plt.figure(figsize=(9, 6))
ax = fig.add_subplot(111, projection="3d")
ax.scatter([0],[0],[0], c=C_FAKE, marker="x", s=60, label="假目标")
r, h = cfg.TARGET_RADIUS, cfg.TARGET_HEIGHT
z_arr = np.linspace(0, h, 8); th_arr = np.linspace(0, 2*np.pi, 16)
Z, TH = np.meshgrid(z_arr, th_arr)
Xc = cfg.TARGET_CENTER[0] + r*np.cos(TH)
Yc = cfg.TARGET_CENTER[1] + r*np.sin(TH)
ax.plot_surface(Xc, Yc, Z, color=C_TARGET, alpha=0.4, edgecolor='none')
ax.plot(fly_x, fly_y, fly_z, color=C_DRONE, linewidth=1.2, label="FY1 轨迹")
ax.plot(drop_x, drop_y, drop_z, color=C_ACCENT, linewidth=1.6, label="烟雾弹平抛段")
ax.scatter([burst[0]], [burst[1]], [burst[2]], c="red", marker="*", s=120, label="起爆点")
ax.plot(sink_x, sink_y, sink_z, color=C_SMOKE, linewidth=1.6, label=f"云团下沉 (v_s={cfg.SMOKE_SINK_SPEED}m/s)")
for tt in [0, 5, 10, 15]:
    idx = int(tt / 20 * (len(t_sink)-1))
    cx, cy, cz = sink_x[idx], sink_y[idx], sink_z[idx]
    u_, v_ = np.meshgrid(np.linspace(0, np.pi, 8), np.linspace(0, 2*np.pi, 12))
    xs = cx + cfg.EFFECTIVE_RADIUS * np.sin(u_) * np.cos(v_)
    ys = cy + cfg.EFFECTIVE_RADIUS * np.sin(u_) * np.sin(v_)
    zs = cz + cfg.EFFECTIVE_RADIUS * np.cos(u_)
    ax.plot_surface(xs, ys, zs, color=C_SMOKE, alpha=0.20, edgecolor='none')
ax.set_xlabel("X (m)"); ax.set_ylabel("Y (m)"); ax.set_zlabel("Z (m)")
ax.set_title("烟雾弹投放→平抛→起爆→下沉 全流程示意 (示例策略)", fontsize=12)
ax.legend(loc="upper left", frameon=False, fontsize=9)
ax.view_init(elev=20, azim=-60)
fig.tight_layout()
fig.savefig(out_path("paper_fig_5_bomb_trajectory.png"), dpi=180)
plt.close(fig)
print(" fig_bomb_trajectory OK")

```

```

# =====
# 图6 问题2最优策略
# =====

```

```

def fig_p2_strategy():
    drone_init = cfg.DRONES_INIT[0]
    theta = np.radians(178.2); v = 83.8
    tr = 0.30; td = 2.91
    te = tr + td
    direction = np.array([np.cos(theta), np.sin(theta), 0.0])
    FY_rel = drone_init + v * direction * tr
    burst = np.array([FY_rel[0] + v*direction[0]*td, FY_rel[1] + v*direction[1]*td, FY_rel[2] - 0.5*cfg.G*td**2])
    dt = 0.01
    ts = np.arange(te, te+cfg.EFFECTIVE_DURATION, dt)
    kp = get_target_keypoints(180, 5)
    mask = _bomb_coverage_mask(drone_init, theta, v, tr, td, 0, kp, dt, ts[-1]+1)
    i0 = int(te/dt); i1 = min(len(mask), i0 + int(cfg.EFFECTIVE_DURATION/dt))
    win = mask[i0:i1]
    fig, axes = plt.subplots(1, 2, figsize=(12, 5))
    ax = axes[0]
    ax.scatter(0, 0, c=C_FAKE, marker="x", s=100, label="假目标")
    th = np.linspace(0, 2*np.pi, 50)
    ax.plot(cfg.TARGET_CENTER[0]+cfg.TARGET_RADIUS*np.cos(th),

```

```

        cfg.TARGET_CENTER[1]+cfg.TARGET_RADIUS*np.sin(th), color=C_TARGET, label="真目标圆周")
ax.scatter([drone_init[0]], [drone_init[1]], c=C_DRONE, marker="o", s=80, label="FY1 起点")
ax.scatter([FY_rel[0]], [FY_rel[1]], c=C_ACCENT, marker="s", s=80, label="投放点")
ax.scatter([burst[0]], [burst[1]], c="red", marker="*", s=200, label="起爆点", zorder=5)
ax.annotate("", xy=(FY_rel[0], FY_rel[1]), xytext=(drone_init[0], drone_init[1]),
            arrowprops=dict(arrowstyle="->", color=C_DRONE, lw=1.4))
t_M = np.linspace(0, 30, 60)
M1 = np.array([missile_position(t, 0) for t in t_M])
ax.plot(M1[:,0], M1[:,1], color=C_MISSILE, linewidth=1.2, linestyle="--", alpha=0.7, label="M1 轨迹")
ax.set_xlim(16500, 18500); ax.set_ylim(-300, 300)
ax.set_xlabel("X (m)"); ax.set_ylabel("Y (m)")
ax.set_title("问题2最优策略 (俯视图, 聚焦真目标区域)", fontsize=11)
ax.legend(loc="upper right", frameon=False, fontsize=8)
ax.grid(True, alpha=0.3)
ax.set_aspect("equal", adjustable="datalim")
ax2 = axes[1]
ax2.fill_between(ts[:len(win)], 0, win.astype(float), step="mid",
                color=C_OK, alpha=0.7, label="有效遮蔽区间")
ax2.set_xlabel("t (s)"); ax2.set_ylabel("是否遮蔽 (0/1)")
ax2.set_title(f"问题2最优策略 遮蔽时序 (T_eff={win.sum()*dt:.3f}s)", fontsize=11)
ax2.set_ylim(-0.05, 1.1)
ax2.legend(loc="upper right", frameon=False, fontsize=9)
fig.tight_layout()
fig.savefig(out_path("paper_fig_6_p2_strategy.png"), dpi=180)
plt.close(fig)
print(" fig_p2_strategy OK")

# =====
# 图7 问题3 三弹时序
# =====
def fig_p3_timing():
    wb = openpyxl.load_workbook(os.path.join(os.path.dirname(__file__), "result1.xlsx"))
    ws = wb.active
    rows = list(ws.iter_rows(values_only=True))[1:]
    ts_release = [r[5] for r in rows]
    ts_deton = [r[5] + r[6] for r in rows]
    drone_init = cfg.DRONES_INIT[0]
    theta = np.radians(178.247); v = 83.81
    kp = get_target_keypoints(180, 5)
    fig, ax = plt.subplots(figsize=(10, 4.5))
    for i, (r, d) in enumerate(zip(ts_release, ts_deton)):
        m = _bomb_coverage_mask(drone_init, theta, v, r, d-r, 0, kp, 0.01, d+cfg.EFFECTIVE_DURATION+1)
        i0 = int((d)/0.01); i1 = min(len(m), i0+int(cfg.EFFECTIVE_DURATION/0.01))
        win = m[i0:i1]
        ts = np.arange(d, d+cfg.EFFECTIVE_DURATION, 0.01)[:len(win)]
        ax.fill_between(ts, i+0.1, i+0.9, where=win, step="mid",
                      color=C_OK, alpha=0.7)
        ax.text(d, i+0.5, f"弹{i+1}\n起爆{d:.2f}s",
                ha="center", va="center", fontsize=8)
    ax.set_xlabel("t (s)"); ax.set_ylabel("烟幕弹编号")
    ax.set_yticks([0.5, 1.5, 2.5])
    ax.set_yticklabels(["弹1", "弹2", "弹3"])
    ax.set_title("问题3: 三枚烟幕弹各自的起爆-下沉时序 (绿色=有效遮蔽区间)", fontsize=11)
    ax.set_xlim(0, 30)
    fig.tight_layout()
    fig.savefig(out_path("paper_fig_7_p3_timing.png"), dpi=180)
    plt.close(fig)
    print(" fig_p3_timing OK")

# =====
# 图8 PSQ收敛曲线
# =====

```

```

def fig_pso_convergence():
    from pso import PSO
    from problem2 import Problem2Objective
    kp = get_target_keypoints(180, 5)
    obj = Problem2Objective(cfg.DRONES_INIT[0], kp, dt=cfg.SEARCH_DT)
    bounds = [(2.73, 3.53), (cfg.DRONE_SPEED_MIN, cfg.DRONE_SPEED_MAX),
              (0.0, 15.0), (0.0, 6.0)]
    pso = PSO(obj, bounds, n_particles=80, max_iter=60, maximize=True, verbose=False)
    _, f_opt = pso.optimize()
    hist = pso.history
    values = [-h for h in hist]
    fig, ax = plt.subplots(figsize=(7, 4))
    ax.plot(np.arange(len(values)), values, color=C_DRONE, linewidth=1.4)
    ax.set_xlabel("迭代次数"); ax.set_ylabel("最优目标值 (s)")
    ax.set_title(f"问题2 PSO 收敛曲线 (最终 T_eff={f_opt:.3f}s)", fontsize=11)
    ax.grid(True, alpha=0.3)
    fig.tight_layout()
    fig.savefig(out_path("paper_fig_8_pso_conv.png"), dpi=180)
    plt.close(fig)
    print(" fig_pso_convergence OK")

# =====
# 图9 关键点采样数目收敛性
# 对比"远离视锥边界"与"贴近视锥边界"两种策略的 n_c 敏感性,
# 暴露 n_c 仅在后者上影响判断的精度下限。
# =====
def fig_keypoint_convergence():
    drone_init = cfg.DRONES_INIT[0]
    n_c_list = [4, 8, 12, 20, 30, 48, 72, 120, 200, 360, 540, 720]
    # 策略A: 最优 (远离视锥边界)
    theta_a, v_a, tr_a, td_a = np.radians(178.2), 83.8, 0.30, 2.91
    vals_a = []
    # 策略B: 贴近视锥边界 (theta 偏 0.2°)
    theta_b, v_b, tr_b, td_b = np.radians(178.0), 83.8, 0.30, 2.91
    vals_b = []
    for n_c in n_c_list:
        kp = get_target_keypoints(n_c, max(1, n_c//36))
        T_a = simulate_single_bomb(drone_init, theta_a, v_a, tr_a, td_a, 0, kp, 0.01, 30.0)
        T_b = simulate_single_bomb(drone_init, theta_b, v_b, tr_b, td_b, 0, kp, 0.01, 30.0)
        vals_a.append(T_a); vals_b.append(T_b)
    fig, ax = plt.subplots(figsize=(7, 4))
    ax.plot(n_c_list, vals_a, "o-", color=C_DRONE, linewidth=1.4, markersize=6,
            label="策略A: 最优 (θ=178.2°, 远离视锥边界)")
    ax.plot(n_c_list, vals_b, "s-", color=C_ACCENT, linewidth=1.4, markersize=6,
            label="策略B: 边界 (θ=178.0°, 贴近视锥边界)")
    ax.axvline(36, color="#888888", linewidth=0.8, linestyle="--", alpha=0.7,
              label="n_c=36 阈值参考线")
    ax.set_xscale("log")
    ax.set_xlabel("每圆周采样点数 $n_c$ (log)")
    ax.set_ylabel("有效遮蔽时长 $T_{eff}$ (s)")
    ax.set_title("关键点采样数目对遮蔽时长的影响 (问题 2)", fontsize=11)
    ax.legend(frameon=False, fontsize=8, loc="lower right")
    ax.grid(True, alpha=0.3, which="both")
    fig.tight_layout()
    fig.savefig(out_path("paper_fig_9_kp_convergence.png"), dpi=180)
    plt.close(fig)
    print(" fig_keypoint_convergence OK")
    return list(zip(n_c_list, vals_a)), list(zip(n_c_list, vals_b))

# =====
# 图10 时间步长收敛性
# =====

```

```

def fig_dt_convergence():
    theta = np.radians(178.2); v = 83.8
    tr = 0.30; td = 2.91
    drone_init = cfg.DRONES_INIT[0]
    kp = get_target_keypoints(180, 5)
    dt_list = [0.05, 0.02, 0.01, 0.005, 0.002, 0.001, 0.0005]
    vals = []
    for dt in dt_list:
        T = simulate_single_bomb(drone_init, theta, v, tr, td, 0, kp, dt, 30.0)
        vals.append(T)
    fig, ax = plt.subplots(figsize=(7, 4))
    ax.plot(dt_list, vals, "s-", color=C_DRONE, linewidth=1.4, markersize=6)
    ax.set_xscale("log")
    ax.set_xlabel("时间步长  $\Delta t$  (s, log)"); ax.set_ylabel("有效遮蔽时长  $T_{eff}$  (s)")
    ax.set_title("时间步长对遮蔽时长的收敛性 (问题2)", fontsize=11)
    ax.grid(True, alpha=0.3, which="both")
    fig.tight_layout()
    fig.savefig(out_path("paper_fig_10_dt_convergence.png"), dpi=180)
    plt.close(fig)
    print(" fig_dt_convergence OK")
    return list(zip(dt_list, vals))

# =====
# 图10b 侧面层数  $n_l$  收敛性 (验证  $n_l$  不影响  $T_{eff}$ )
# =====
def fig_nl_convergence():
    """ $n_l$  扫描: 对最优策略与两个边界策略, 固定  $n_c=180$ 、扫描  $n_l$ """
    drone_init = cfg.DRONES_INIT[0]
    n_l_list = [0, 1, 2, 3, 5, 7, 10, 15, 20]
    curves = {}
    for label, theta_deg in [("A:  $\theta=178.2^\circ$  (最优)", 178.2),
                             ("B:  $\theta=178.0^\circ$  (边界)", 178.0),
                             ("C:  $\theta=178.5^\circ$  (边界)", 178.5)]:
        vals = []
        for n_l in n_l_list:
            kp = get_target_keypoints(180, n_l)
            T = simulate_single_bomb(drone_init, np.radians(theta_deg), 83.8, 0.30, 2.91, 0, kp, 0.01, 30.0)
            vals.append(T)
        curves[label] = vals
    fig, ax = plt.subplots(figsize=(7, 4))
    colors = [C_DRONE, C_ACCENT, "#117A65"]
    for (label, vals), col in zip(curves.items(), colors):
        ax.plot(n_l_list, vals, "o-", color=col, linewidth=1.4, markersize=6, label=label)
    ax.set_xlabel("侧面层数  $n_l$  (固定  $n_c=180$ )")
    ax.set_ylabel("有效遮蔽时长  $T_{eff}$  (s)")
    ax.set_title("侧面层数  $n_l$  对有效遮蔽时长的影响 (问题 2)", fontsize=11)
    ax.legend(frameon=False, fontsize=9)
    ax.grid(True, alpha=0.3)
    fig.tight_layout()
    fig.savefig(out_path("paper_fig_10b_nl_convergence.png"), dpi=180)
    plt.close(fig)
    print(" fig_nl_convergence OK")
    return curves

# =====
# 图11 云图下沉速度影响
# =====
def fig_sink_speed():
    theta = np.radians(178.2); v = 83.8
    tr = 0.30; td = 2.91
    drone_init = cfg.DRONES_INIT[0]
    kp = get_target_keypoints(180, 5)

```

```

vs_list = [1.5, 2.0, 2.5, 3.0, 3.5, 4.0, 5.0]
vals = []
for vs in vs_list:
    old = cfg.SMOKE_SINK_SPEED
    cfg.SMOKE_SINK_SPEED = vs
    T = simulate_single_bomb(drone_init, theta, v, tr, td, 0, kp, 0.01, 30.0)
    vals.append(T)
    cfg.SMOKE_SINK_SPEED = old
fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(vs_list, vals, "o-", color=C_DRONE, linewidth=1.4, markersize=6)
ax.axvline(2.5, color=C_ACCENT, linewidth=0.8, linestyle="--", alpha=0.7, label="本题 v_s=2.5")
ax.axvline(3.0, color="#888888", linewidth=0.8, linestyle="--", alpha=0.7, label="对照 v_s=3.0")
ax.set_xlabel("云团下沉速度 v_s (m/s)"); ax.set_ylabel("有效遮蔽时长 T_eff (s)")
ax.set_title("云团下沉速度对遮蔽时长的影响 (问题2)", fontsize=11)
ax.legend(frameon=False, fontsize=9)
ax.grid(True, alpha=0.3)
fig.tight_layout()
fig.savefig(out_path("paper_fig_11_sink_speed.png"), dpi=180)
plt.close(fig)
print(" fig_sink_speed OK")
return list(zip(vs_list, vals))

# =====
# 图12 无人机速度上限影响
# =====
def fig_drone_speed():
    """无人机速度上限敏感性：每个 v_max 下若原始最优 v=83.8 ≤ v_max 则保持 v=83.8，
    否则取 v=v_max 并扫描；展示"上限收窄"对最优解的影响。"""
    theta = np.radians(178.2)
    tr = 0.30; td = 2.91
    drone_init = cfg.DRONES_INIT[0]
    kp = get_target_keypoints(180, 5)
    v_opt = 83.8 # 问题2原始最优速度
    vmax_list = [85, 90, 100, 110, 120, 130, 140, 150]
    vals = []
    for vmax in vmax_list:
        v_use = min(v_opt, vmax)
        T = simulate_single_bomb(drone_init, theta, v_use, tr, td, 0, kp, 0.01, 30.0)
        vals.append(T)
    fig, ax = plt.subplots(figsize=(7, 4))
    ax.plot(vmax_list, vals, "o-", color=C_DRONE, linewidth=1.4, markersize=6)
    ax.axvline(120, color=C_ACCENT, linewidth=0.8, linestyle="--", alpha=0.7, label="本题 v_max=120")
    ax.set_xlabel("无人机速度上限 v_max (m/s)"); ax.set_ylabel("有效遮蔽时长 T_eff (s)")
    ax.set_title("无人机速度上限对遮蔽时长的影响 (问题2, v=min(83.8, v_max))", fontsize=11)
    ax.legend(frameon=False, fontsize=9)
    ax.grid(True, alpha=0.3)
    fig.tight_layout()
    fig.savefig(out_path("paper_fig_12_drone_speed.png"), dpi=180)
    plt.close(fig)
    print(" fig_drone_speed OK")
    return list(zip(vmax_list, vals))

# =====
# 图A1 投放时刻敏感性 (问题2最优解附近扰动)
# =====
def fig_release_time():
    theta = np.radians(178.2); v = 83.8; td = 2.91
    drone_init = cfg.DRONES_INIT[0]
    kp = get_target_keypoints(180, 5)
    tr_list = [0.05, 0.10, 0.20, 0.30, 0.40, 0.50, 0.60, 0.80, 1.00, 1.50]
    vals = []
    for tr in tr_list:

```

```

        T = simulate_single_bomb(drone_init, theta, v, tr, td, 0, kp, 0.01, 30.0)
        vals.append(T)
    fig, ax = plt.subplots(figsize=(7, 4))
    ax.plot(tr_list, vals, "o-", color=C_DRONE, linewidth=1.4, markersize=6)
    ax.axvline(0.30, color=C_ACCENT, linewidth=0.8, linestyle="--", alpha=0.7, label="最优 tr=0.30 s")
    ax.set_xlabel("投放时刻 $t_r$ (s)")
    ax.set_ylabel("有效遮蔽时长 $T_{eff}$ (s)")
    ax.set_title("投放时刻 $t_r$ 对有效遮蔽时长的影响 (问题2)", fontsize=11)
    ax.legend(frameon=False, fontsize=9)
    ax.grid(True, alpha=0.3)
    fig.tight_layout()
    fig.savefig(out_path("paper_fig_a1_release_time.png"), dpi=180)
    plt.close(fig)
    print(" fig_release_time OK")
    return list(zip(tr_list, vals))

# =====
# 图A2 起爆延时敏感性 (问题2最优解附近扰动)
# =====
def fig_detonation_delay():
    theta = np.radians(178.2); v = 83.8; tr = 0.30
    drone_init = cfg.DRONES_INIT[0]
    kp = get_target_keypoints(180, 5)
    td_list = [1.5, 2.0, 2.5, 2.8, 2.91, 3.0, 3.2, 3.5, 4.0, 5.0]
    vals = []
    for td in td_list:
        T = simulate_single_bomb(drone_init, theta, v, tr, td, 0, kp, 0.01, 30.0)
        vals.append(T)
    fig, ax = plt.subplots(figsize=(7, 4))
    ax.plot(td_list, vals, "s-", color=C_DRONE, linewidth=1.4, markersize=6)
    ax.axvline(2.91, color=C_ACCENT, linewidth=0.8, linestyle="--", alpha=0.7, label="最优 td=2.91 s")
    ax.set_xlabel("起爆延时 $t_d$ (s)")
    ax.set_ylabel("有效遮蔽时长 $T_{eff}$ (s)")
    ax.set_title("起爆延时 $t_d$ 对有效遮蔽时长的影响 (问题2)", fontsize=11)
    ax.legend(frameon=False, fontsize=9)
    ax.grid(True, alpha=0.3)
    fig.tight_layout()
    fig.savefig(out_path("paper_fig_a2_detonation_delay.png"), dpi=180)
    plt.close(fig)
    print(" fig_detonation_delay OK")
    return list(zip(td_list, vals))

# =====
# 图A3 算法稳定性: 问题2 上重复运行 PSO 多次
# =====
def fig_algorithm_stability():
    from pso import PSO
    from problem2 import Problem20Objective
    kp = get_target_keypoints(180, 5)
    obj = Problem20Objective(cfg.DRONES_INIT[0], kp, dt=cfg.SEARCH_DT)
    bounds = [(2.73, 3.53), (cfg.DRONE_SPEED_MIN, cfg.DRONE_SPEED_MAX),
              (0.0, 15.0), (0.0, 6.0)]
    n_runs = 8
    finals = []
    for run_id in range(n_runs):
        np.random.seed(run_id * 17 + 1) # 主进程内确定性
        pso = PSO(obj, bounds, n_particles=80, max_iter=60, maximize=True,
                  verbose=False, n_workers=1) # 单进程让 seed 可控
        _, f_opt = pso.optimize()
        finals.append(f_opt)
    fig, ax = plt.subplots(figsize=(6.5, 4))
    ax.bar(range(1, n_runs+1), finals, color=C_DRONE, alpha=0.75, edgecolor='black', linewidth=0.4)

```

```

ax.axhline(np.mean(finals), color=C_ACCENT, linewidth=1.0, linestyle="--",
            label=f"均值 {np.mean(finals):.3f} s")
ax.axhline(np.max(finals), color=C_OK, linewidth=1.0, linestyle=":",
            label=f"最大值 {np.max(finals):.3f} s")
ax.set_xlabel("运行序号 (不同随机种子)")
ax.set_ylabel("最优 $T_{eff}$ (s)")
ax.set_title(f"PSO 在问题 2 上的稳定性 ({n_runs} 次独立运行)", fontsize=11)
ax.legend(frameon=False, fontsize=9)
ax.grid(True, alpha=0.3, axis='y')
ax.set_ylim(min(finals) - 0.05, max(finals) + 0.05)
fig.tight_layout()
fig.savefig(out_path("paper_fig_a3_pso_stability.png"), dpi=180)
plt.close(fig)
print(" fig_algorithm_stability OK")
return finals

# =====
# 图A4 三种算法收敛曲线对比 (PSO/GA/SA 在问题2 上同条件实测)
# =====
def fig_algo_comparison():
    """同条件同评估预算下 PSO / GA / SA 收敛曲线"""
    from pso import PSO
    from problem2 import Problem2Objective
    from ga_sa_baselines import run_ga, run_sa
    kp = get_target_keypoints(180, 5)
    obj = Problem2Objective(cfg.DRONES_INIT[0], kp, dt=cfg.SEARCH_DT)
    bounds = [(2.73, 3.53), (cfg.DRONE_SPEED_MIN, cfg.DRONE_SPEED_MAX),
              (0.0, 15.0), (0.0, 6.0)]
    n_eval_budget = 6000 # 公平对比: 每个算法最多 6000 次评估
    # PSO: 80 粒子 × 75 代 ≈ 6000
    np.random.seed(42)
    pso = PSO(obj, bounds, n_particles=80, max_iter=75, maximize=True,
              verbose=False, n_workers=1)
    _ = pso.optimize()
    pso_curve = [-h for h in pso.history]
    # GA / SA 用统一的评估预算
    ga_curve = run_ga(obj, bounds, n_eval_budget=n_eval_budget, seed=42)
    sa_curve = run_sa(obj, bounds, n_eval_budget=n_eval_budget, seed=42)
    fig, ax = plt.subplots(figsize=(7, 4))
    ax.plot(np.arange(len(pso_curve))*80, pso_curve, color=C_DRONE, linewidth=1.4, label="PSO")
    ax.plot(np.arange(len(ga_curve)), ga_curve, color=C_ACCENT, linewidth=1.4, label="GA (实数编码)")
    ax.plot(np.arange(len(sa_curve)), sa_curve, color="#117A65", linewidth=1.4, label="SA (指数降温)")
    ax.set_xlabel("函数评估次数")
    ax.set_ylabel("当前最优 $T_{eff}$ (s)")
    ax.set_title("PSO / GA / SA 在问题 2 上的收敛曲线 (相同评估预算 6000 次)", fontsize=11)
    ax.legend(frameon=False, fontsize=9)
    ax.grid(True, alpha=0.3)
    fig.tight_layout()
    fig.savefig(out_path("paper_fig_a4_algo_compare.png"), dpi=180)
    plt.close(fig)
    print(" fig_algo_comparison OK")
    return {"pso": pso_curve[-1], "ga": ga_curve[-1], "sa": sa_curve[-1]}

# =====
# 图13 问题4 三机协同策略 (去掉云团球避免3D重叠, 只画航迹+起爆点)
# =====
def fig_p4_strategy():
    wb = openpyxl.load_workbook(os.path.join(os.path.dirname(__file__), "result2.xlsx"))
    ws = wb.active
    rows = list(ws.iter_rows(values_only=True))[1:]
    fig = plt.figure(figsize=(10, 6.5))
    ax = fig.add_subplot(111, projection="3d")

```

```

M = cfg.MISSILES_INIT
D = cfg.DRONES_INIT
ax.scatter(M[0,0], M[0,1], M[0,2], c=C_MISSILE, marker="^", s=120, label="M1 (来袭)")
ax.scatter(D[:3,0], D[:3,1], D[:3,2], c=C_DRONE, marker="o", s=80, label="FY1-3 (起点)")
ax.scatter(*cfg.FAKE_TARGET, c=C_FAKE, marker="x", s=80, label="假目标(原点)")
# 真目标 (浅色)
r, h = cfg.TARGET_RADIUS, cfg.TARGET_HEIGHT
z_arr = np.linspace(0, h, 6); th_arr = np.linspace(0, 2*np.pi, 12)
Z, TH = np.meshgrid(z_arr, th_arr)
Xc = cfg.TARGET_CENTER[0] + r*np.cos(TH)
Yc = cfg.TARGET_CENTER[1] + r*np.sin(TH)
ax.plot_surface(Xc, Yc, Z, color=C_TARGET, alpha=0.3, edgecolor='none')
colors = [C_DRONE, "#48C9B0", "#D68910"]
for i, (r_, col) in enumerate(zip(rows, colors)):
    theta = r_[1]; v = r_[3]; tr = r_[4]; td = r_[5]
    direction = np.array([np.cos(theta), np.sin(theta), 0.0])
    rel = D[i] + v * direction * tr
    burst = np.array([rel[0]+v*direction[0]*td, rel[1]+v*direction[1]*td, rel[2]-0.5*cfg.G*td**2])
    t_fly = np.linspace(0, tr+td+0.5, 30)
    fx = D[i,0] + v*direction[0]*t_fly
    fy = D[i,1] + v*direction[1]*t_fly
    fz = np.full_like(t_fly, D[i,2])
    ax.plot(fx, fy, fz, color=col, linewidth=1.4, alpha=0.85, label=f"FY{i+1} 轨迹")
    # 起爆点 (带标签)
    ax.scatter([burst[0]], [burst[1]], [burst[2]], c=col, marker="*", s=180, zorder=5)
    ax.text(burst[0], burst[1], burst[2]+30, f"FY{i+1}起爆", color=col, fontsize=9, fontweight='bold')
# M1 轨迹
t_M = np.linspace(0, 25, 50)
M1 = np.array([missile_position(t, 0) for t in t_M])
ax.plot(M1[:,0], M1[:,1], M1[:,2], color=C_MISSILE, linewidth=1.4, linestyle="--", alpha=0.7, label="M1 飞行轨迹")
ax.set_xlabel("X (m)", fontsize=10); ax.set_ylabel("Y (m)", fontsize=10); ax.set_zlabel("Z (m)", fontsize=10)
ax.set_title("问题4: 三机协同最优策略 (轨迹+起爆点, 不画云团球以避免遮挡)", fontsize=12)
ax.legend(loc="upper left", frameon=True, fontsize=8)
ax.view_init(elev=22, azim=-55)
fig.tight_layout()
fig.savefig(out_path("paper_fig_13_p4_strategy.png"), dpi=180)
plt.close(fig)
print(" fig_p4_strategy OK")

# =====
# 图14 问题5 五机多弹策略 (去掉云团球, 俯视+侧视双视角)
# =====
def fig_p5_strategy():
    wb = openpyxl.load_workbook(os.path.join(os.path.dirname(__file__), "result3.xlsx"))
    ws = wb.active
    rows = list(ws.iter_rows(values_only=True))[1:]
    # 双图: 左=俯视图 (X-Y), 右=侧视图 (X-Z)
    fig, axes = plt.subplots(1, 2, figsize=(14, 5.5))
    M = cfg.MISSILES_INIT
    D = cfg.DRONES_INIT
    # 颜色: 按目标导弹
    missile_color = {0: C_MISSILE, 1: "#1B4F72", 2: "#117A65"}
    missile_label = {0: "M1", 1: "M2", 2: "M3"}
    # === 左图: 俯视 (X-Y) ===
    ax = axes[0]
    ax.scatter(0, 0, c=C_FAKE, marker="x", s=120, label="假目标")
    th = np.linspace(0, 2*np.pi, 50)
    ax.plot(cfg.TARGET_CENTER[0]+cfg.TARGET_RADIUS*np.cos(th),
            cfg.TARGET_CENTER[1]+cfg.TARGET_RADIUS*np.sin(th),
            color=C_TARGET, linewidth=1.5, label="真目标")
    # 3枚导弹
    for k in range(3):
        t_M = np.linspace(0, 30, 40)

```



```

Mk = np.array([missile_position(t, k) for t in t_M])
ax.plot(Mk[:,0], Mk[:,1], color=C_MISSILE, linewidth=1.0, linestyle="--", alpha=0.4)
ax.scatter([Mk[0,0]], [Mk[0,1]], c=C_MISSILE, marker="^", s=80)
ax.text(Mk[0,0]+200, Mk[0,1], f"M{k+1}", color=C_MISSILE, fontsize=10, fontweight='bold')

# 5架无人机起爆点 (按目标着色)
for r_ in rows:
    di = int(r_[0][2]) - 1
    theta = r_[1]; v = r_[3]; tr = r_[6]; td = r_[7]
    mi = int(r_[5][1]) - 1
    direction = np.array([np.cos(theta), np.sin(theta), 0.0])
    rel = D[di] + v * direction * tr
    burst = np.array([rel[0]+v*direction[0]*td, rel[1]+v*direction[1]*td])
    col = missile_color.get(mi, C_DRONE)
    ax.scatter([burst[0]], [burst[1]], c=col, marker="*", s=100, zorder=5)
ax.set_xlabel("X (m)"); ax.set_ylabel("Y (m)")
ax.set_title("问题5 起爆点分布 (俯视, X-Y)", fontsize=11)
ax.grid(True, alpha=0.3)
ax.set_aspect("equal", adjustable="datalim")
ax.legend(loc="upper right", frameon=True, fontsize=8)

# === 右图: 侧视 (X-Z) ===
ax = axes[1]
ax.scatter(0, 0, c=C_FAKE, marker="x", s=120, label="假目标")
# 真目标 (侧面投影: 矩形)
rect = plt.Rectangle((cfg.TARGET_CENTER[0]-cfg.TARGET_RADIUS, 0),
                    2*cfg.TARGET_RADIUS, cfg.TARGET_HEIGHT,
                    color=C_TARGET, alpha=0.5, label="真目标")
ax.add_patch(rect)
# 3枚导弹轨迹
for k in range(3):
    t_M = np.linspace(0, 30, 40)
    Mk = np.array([missile_position(t, k) for t in t_M])
    ax.plot(Mk[:,0], Mk[:,2], color=C_MISSILE, linewidth=1.0, linestyle="--", alpha=0.4)

# 5架无人机起爆点 (按目标着色)
for r_ in rows:
    di = int(r_[0][2]) - 1
    theta = r_[1]; v = r_[3]; tr = r_[6]; td = r_[7]
    mi = int(r_[5][1]) - 1
    direction = np.array([np.cos(theta), np.sin(theta), 0.0])
    rel = D[di] + v * direction * tr
    burst = np.array([rel[0]+v*direction[0]*td,
                    rel[2]-0.5*cfg.G*td**2 - cfg.SMOKE_SINK_SPEED*0]) # 起爆点Z
    col = missile_color.get(mi, C_DRONE)
    ax.scatter([burst[0]], [burst[1]], c=col, marker="*", s=100, zorder=5)

# 图例
handles = [plt.scatter([0], [0], c=C_MISSILE, marker="*", s=100, label=f"指向 {missile_label[0]} 的起爆点"),
            plt.scatter([0], [0], c="#1B4F72", marker="*", s=100, label=f"指向 {missile_label[1]} 的起爆点"),
            plt.scatter([0], [0], c="#117A65", marker="*", s=100, label=f"指向 {missile_label[2]} 的起爆点")]
ax.legend(handles=handles, loc="upper right", frameon=True, fontsize=8)
ax.set_xlabel("X (m)"); ax.set_ylabel("Z (m)")
ax.set_title("问题5 起爆点分布 (侧视, X-Z)", fontsize=11)
ax.grid(True, alpha=0.3)
ax.set_xlim(0, 21000)
ax.set_ylim(0, 2500)

fig.suptitle("问题5: 五机多弹多导弹协同最优策略起爆点分布", fontsize=13, y=1.02)
fig.tight_layout()
fig.savefig(out_path("paper_fig_14_p5_strategy.png"), dpi=180)
plt.close(fig)
print(" fig_p5_strategy OK")

```

```

# =====
# 入口

```

```

# =====
if __name__ == "__main__":
    print("=== 生成论文配图 ===")
    fig_scenario()
    fig_view_cone()
    fig_multi_cloud()
    fig_p1_timeline()
    fig_bomb_trajectory()
    fig_p2_strategy()
    fig_p3_timing()
    fig_pso_convergence()
    fig_keypoint_convergence()
    fig_dt_convergence()
    fig_nl_convergence()
    fig_sink_speed()
    fig_drone_speed()
    fig_p4_strategy()
    fig_p5_strategy()

    # 敏感性增强章节新增图
    data_kp = fig_release_time()
    data_td = fig_detonation_delay()
    finals = fig_algorithm_stability()
    compare = fig_algo_comparison()
    print("---")
    print(f"PSO 稳定性: mean={np.mean(finals):.4f} std={np.std(finals):.4f} max={np.max(finals):.4f}")
    print(f"算法对比(同等预算6000次评估): PSO={compare['pso']:.3f} GA={compare['ga']:.3f} SA={compare['sa']:.3f}")
    print("=== 全部完成 ===")

```